

# **Операционные системы**

**Лекции**

5 семестр

Лектор: Рязанова Наталья Юрьевна  
МГТУ им. Н. Э. Баумана

GitHub: [malyinik](#), [dqqa](#)

2026 г.

# СОДЕРЖАНИЕ

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>История создания ЭВМ</b>                  | <b>7</b>  |
| 1.1      | Принципы фон Неймана                         | 7         |
| 1.2      | Первые вычислительные машины                 | 8         |
| 1.3      | Ассемблер                                    | 8         |
| 1.4      | Компания IBM и появление операционных систем | 8         |
| 1.5      | Третье поколение вычислительных машин        | 9         |
| 1.5.1    | Канальная архитектура IBM 360                | 9         |
| 1.5.2    | Система прерываний                           | 9         |
| 1.5.3    | Пакетная обработка                           | 10        |
| 1.6      | Четвёртое поколение ЭВМ                      | 11        |
| <b>2</b> | <b>Unix</b>                                  | <b>12</b> |
| 2.1      | История создания Unix                        | 12        |
| 2.2      | Основные концепции Unix                      | 13        |
| 2.2.1    | Процессы и файлы                             | 13        |
| 2.2.2    | Язык программирования Си                     | 13        |
| 2.2.3    | Файловая система                             | 13        |
| <b>3</b> | <b>POSIX</b>                                 | <b>14</b> |
| 3.1      | История стандартизации                       | 14        |
| 3.2      | Ричард Столлмен и движение свободного ПО     | 14        |
| <b>4</b> | <b>Структура операционной системы</b>        | <b>14</b> |
| 4.1      | Ресурсы вычислительной системы               | 15        |
| 4.2      | Реентерабельные коды                         | 15        |
| 4.3      | Два взгляда на операционную систему          | 15        |
| <b>5</b> | <b>Управление процессорами</b>               | <b>17</b> |
| 5.1      | Диаграмма состояний процесса                 | 17        |
| 5.1.1    | Порождение                                   | 17        |
| 5.1.2    | Создание процесса и выделение дескриптора    | 17        |
| 5.1.3    | Загрузка в память                            | 17        |
| 5.1.4    | Состояние готовности (Ready)                 | 18        |
| 5.1.5    | Состояние выполнения (Running)               | 18        |
| 5.1.6    | Состояние завершения (Terminated)            | 18        |
| 5.1.7    | Состояние блокировки (Blocked)               | 18        |
| 5.1.8    | Системы разделения времени                   | 18        |
| 5.2      | Алгоритмы планирования (scheduling)          | 18        |
| 5.2.1    | Классификация алгоритмов                     | 19        |
| 5.2.2    | Планировщик и диспетчер                      | 20        |
| 5.2.3    | Определение длины кванта                     | 21        |
| 5.2.4    | Приоритетное планирование                    | 21        |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 5.2.5    | Многоуровневые очереди с обратной связью .....     | 22        |
| 5.2.6    | Анализ времени ожидания и накладных расходов ..... | 22        |
| 5.3      | Планирование в многопроцессорных системах .....    | 23        |
| 5.3.1    | Processor Affinity (сродство процессов) .....      | 23        |
| 5.4      | Классы планирования в Linux .....                  | 24        |
| 5.5      | Работа с очередями процессов .....                 | 24        |
| <b>6</b> | <b>Переключение в режим ядра .....</b>             | <b>25</b> |
| 6.1      | Режим работы процессора в Unix .....               | 25        |
| 6.2      | События, переводящие систему в режим ядра .....    | 26        |
| 6.3      | Диаграмма состояний процессов Unix. ....           | 27        |
| <b>7</b> | <b>Потоки .....</b>                                | <b>28</b> |
| 7.1      | Базовое представление о потоках .....              | 28        |
| 7.1.1    | Характерные черты процесса .....                   | 28        |
| 7.1.2    | Определение потока .....                           | 28        |
| 7.1.3    | Однопоточная и многопоточная модели .....          | 28        |
| 7.1.4    | Стеки потоков .....                                | 29        |
| 7.1.5    | Типы потоков .....                                 | 30        |
| 7.2      | Мотивация поддержки потоков .....                  | 31        |
| 7.3      | Преимущества мультипоточности (потоки ядра) .....  | 31        |
| 7.4      | Проблемы многопоточного программирования .....     | 32        |
| 7.5      | Типы параллелизма .....                            | 33        |
| 7.6      | Многопоточные модели (Multithreading Models) ..... | 34        |
| 7.7      | Библиотеки потоков .....                           | 36        |
| 7.8      | Lightweight Processes .....                        | 36        |
| <b>8</b> | <b>Взаимодействие параллельных процессов .....</b> | <b>38</b> |
| 8.1      | Уровни наблюдения .....                            | 38        |
| 8.2      | Проблемы параллельного выполнения .....            | 38        |
| 8.2.1    | Пример с банковским счётом .....                   | 39        |
| 8.2.2    | Критические секции и разделяемые ресурсы .....     | 41        |
| 8.3      | Методы взаимoisключения .....                      | 41        |
| 8.4      | Программный способ взаимoisключения .....          | 42        |
| 8.4.1    | Первая попытка с флагами .....                     | 42        |
| 8.4.2    | Алгоритм Деккера .....                             | 43        |
| 8.4.3    | Алгоритм Bakery .....                              | 45        |
| 8.5      | Аппаратный способ взаимoisключения .....           | 45        |
| 8.5.1    | Команда test_and_set .....                         | 45        |
| 8.5.2    | Spin lock .....                                    | 46        |
| <b>9</b> | <b>Семафоры Дейкстры .....</b>                     | <b>47</b> |
| 9.1      | Определение семафора .....                         | 47        |
| 9.2      | Реализация семафоров .....                         | 47        |

|           |                                                                                   |           |
|-----------|-----------------------------------------------------------------------------------|-----------|
| 9.3       | Взаимоисключение с помощью семафоров .....                                        | 48        |
| 9.4       | Задачи синхронизации .....                                                        | 49        |
| 9.4.1     | Задача производства — потребления (Producer — Consumer) ....                      | 49        |
| 9.4.2     | Задача обедающих философов .....                                                  | 50        |
| 9.4.3     | Решение с помощью наборов семафоров .....                                         | 51        |
| 9.5       | Задача «Булочная» Лампорта (Lamport's Bakery Algorithm) .....                     | 52        |
| 9.6       | Современные примитивы синхронизации .....                                         | 54        |
| 9.6.1     | Отличия от семафоров .....                                                        | 54        |
| <b>10</b> | <b>Мониторы Хоара .....</b>                                                       | <b>55</b> |
| 10.1      | Определение и структура монитора .....                                            | 55        |
| 10.2      | Простой монитор .....                                                             | 56        |
| 10.3      | Монитор «Кольцевой буфер» .....                                                   | 57        |
| 10.3.1    | Постановка задачи .....                                                           | 57        |
| 10.4      | Монитор читатели-писатели .....                                                   | 58        |
| 10.4.1    | Работа монитора .....                                                             | 60        |
| 10.5      | Задача Подписчик-Издатель .....                                                   | 61        |
| 10.6      | Задача «Проблема спящего парикмахера» .....                                       | 61        |
| 10.7      | Охрана (что-то дополнительное, от Дейкстры) .....                                 | 62        |
| 10.8      | Рандеву (свидание) .....                                                          | 62        |
| 10.9      | Взаимодействие процессов в распределённых системах (больше чисто про время) ..... | 65        |
| 10.9.1    | Проблема физического времени .....                                                | 66        |
| 10.10     | NTP .....                                                                         | 67        |
| 10.10.1   | Timestamp (алгоритм Лампорта отметки времени) .....                               | 68        |
|           | Пример системы, где не используется алгоритм .....                                | 69        |
|           | .....                                                                             | 70        |
| 10.10.2   | Алгоритм векторных часов .....                                                    | 70        |
| 10.11     | Алгоритмы и подходы взаимодействия процессов в распределённых системах .....      | 70        |
| 10.11.1   | Централизованный алгоритм .....                                                   | 70        |
| 10.11.2   | Распределённый алгоритм .....                                                     | 72        |
| 10.11.3   | Token Ring .....                                                                  | 72        |
| 10.12     | Транзакции .....                                                                  | 73        |
| 10.12.1   | Вторая фаза .....                                                                 | 75        |
|           | Первая фаза .....                                                                 | 75        |
|           | Вторая фаза .....                                                                 | 75        |
| 10.13     | Тупики .....                                                                      | 75        |
| 10.13.1   | Условия возникновения тупика .....                                                | 78        |
| 10.13.2   | Методы борьбы с тупиками .....                                                    | 78        |
| 10.13.3   | Способы исключения тупиков .....                                                  | 79        |
|           | Опережающее требование .....                                                      | 79        |



|                                                               |            |
|---------------------------------------------------------------|------------|
| Иерархическое распределение .....                             | 79         |
| .....                                                         | 80         |
| 10.13.4 Алгоритм банкира .....                                | 80         |
| Алгоритм Дейкстры (сам непосредственно) .....                 | 82         |
| 10.13.5 Обнаружение тупиков .....                             | 82         |
| 10.13.6 Алгоритм Хабермана .....                              | 86         |
| Алгоритм проверки безопасного состояния .....                 | 88         |
| 10.13.7 Алгоритм Chandy–Misra–Haas .....                      | 91         |
| 10.14 Транзакции .....                                        | 92         |
| 10.14.1 Алгоритм ожидание-смерть (wait-die) .....             | 92         |
| 10.14.2 Алгоритм ожидание-ранение (wound-wait) .....          | 92         |
| 10.15 что .....                                               | 92         |
| Статические разделы .....                                     | 93         |
| Динамические разделы .....                                    | 93         |
| 10.16 Управление памятью страницами по запросу .....          | 96         |
| 10.17 .....                                                   | 96         |
| 10.17.1 Прямое отображение .....                              | 96         |
| 10.17.2 Ассоциативное отображение .....                       | 97         |
| 10.17.3 Ассоциативно-прямое отображение .....                 | 97         |
| 10.17.4 Схема с гиперстраницами .....                         | 98         |
| 10.18 Замещение страниц .....                                 | 98         |
| 10.18.1 Замещение случайной страницы .....                    | 98         |
| 10.18.2 Алгоритм FIFO .....                                   | 99         |
| 10.18.3 Алгоритм LRU .....                                    | 99         |
| 10.18.4 Алгоритм LFU .....                                    | 100        |
| 10.18.5 Алгоритм NUR .....                                    | 100        |
| 10.19 Теория рабочего множества .....                         | 101        |
| 10.19.1 hit rate .....                                        | 102        |
| 10.20 Какая-то тема про что .....                             | 103        |
| 10.20.1 Официальное заявление на 10.12.2025 🚀 .....           | 103        |
| 10.21 Управление сегментами по запросу .....                  | 103        |
| 10.22 .....                                                   | 107        |
| 10.23 Ядра .....                                              | 108        |
| 10.24 Микроядра .....                                         | 108        |
| 10.25 Операционная система Mach .....                         | 110        |
| <b>11 Системы реального времени (RTOS, Realtime OS) .....</b> | <b>111</b> |

# ОПРЕДЕЛЕНИЯ

|                   |                                   |     |
|-------------------|-----------------------------------|-----|
| Определение 1.1   | Компьютер .....                   | 7   |
| Определение 1.2   | Пакет .....                       | 10  |
| Определение 1.3   | Терминал .....                    | 11  |
| Определение 1.4   | Квант .....                       | 11  |
| Определение 2.1   | Процесс .....                     | 13  |
| Определение 2.2   | Файл .....                        | 13  |
| Определение 4.1   | Операционная система .....        | 14  |
| Определение 4.2   | Реентерабельные коды .....        | 15  |
| Определение 5.1   | Планировщик .....                 | 20  |
| Определение 5.2   | Диспетчеризация .....             | 20  |
| Определение 6.1   | Процесс с точки зрения Unix ..... | 25  |
| Определение 7.1   | Поток .....                       | 28  |
| Определение 7.2   | Lightweight Processes .....       | 36  |
| Определение 9.1   | Семафор .....                     | 47  |
| Определение 9.2   | Лексикографический порядок .....  | 54  |
| Определение 10.1  | Монитор .....                     | 55  |
| Определение 10.2  | Солнечный день .....              | 66  |
| Определение 10.3  | Атомная секунда .....             | 66  |
| Определение 10.4  | NTP .....                         | 67  |
| Определение 10.5  | Тупик / тупиковая ситуация .....  | 77  |
| Определение 10.9  | Матрица заявок .....              | 87  |
| Определение 10.10 | Матрица распределений .....       | 87  |
| Определение 11.1  | Реальное время в ОС .....         | 111 |

# 1 История создания ЭВМ

## Определение 1.1 (Компьютер)

Компьютер — это программно управляемое устройство.



- Эта идея была реализована Чарльзом Бэббиджем.
- Первая вычислительная машина появилась не раньше 1845 года.
- Изначально основной задачей было выполнение вычислений автоматически.

## 1.1 Принципы фон Неймана

- 1) Универсальная вычислительная машина должна содержать несколько основных устройств:
    - 1.1) Арифметики
    - 1.2) Памяти
    - 1.3) Управления
    - 1.4) Связи с оператором Мы хотим ... (выполнялся автоматически)
  - 2) Необходимо, чтобы машина могла запоминать некоторым образом не только цифровую информацию (данные), которые требуются для вычислений, но так же и команды, управляющие вычислениями.
  - 3) Если команды машине представить в виде числового кода, если машина сможет каким либо образом отличать число от команды, то память можно использовать для хранения как чисел, так и команд (instruction) Это можно назвать принципом хранения команды
  - 4) Помимо памяти для команд должно ещё в составе машины существовать устройство, способное выполнять команды. Будем называть это **устройством управления** или **управляющим устройством**.
  - 5) Поскольку машина называется вычислительной, в ней должен быть вычислительный орган, способный вычислять (складывать, вычитать, ...)
  - 6) Должно существовать устройство ввода-вывода, с помощью которого осуществляется связь между оператором и машиной. Машина должна работать с двоичными числами.

Первые машины пытались сделать с десятичной системой счисления, но это оказалось проблематично.
- Процессор собственной памяти не имеет, а обменивается с оперативным запоминающим устройством. Это привело к необходимости адресации, то есть все данные в машине адресуются.
  - Ячейки памяти должны иметь уникальные адреса, которые указываются в соответствующих командах. Обращения к командам и данным выполняются по адресу.

Следующим шагом в реализации машины явилось то, что из команд исключили адрес следующей команды. Это привело к тому, что программа в память загружается последовательные адреса. Это потребовало включить счётчик команд (program counter). Считывание каждой команды приводит к увеличению содержимого счётчика команд на ...

Первые машины работали в абсолютных адресах. Это фиксированные адреса памяти, которые были известны оператору. Он оперировал ими вручную.

## 1.2 Первые вычислительные машины

Команда — последовательность 0 и 1.

Первые машины не имели никакой операционной системы, то есть не имели управляющего ПО. Они управлялись коммутационными панелями. Эксплуатировать машину могли только те, кто её создали.

Компьютером является процессор и оперативная память. В память загружается программа, а дальше она начинает выполняться автоматически.

- В задачи оператора входило выполнение определённых действий, связанных с «пакетами». Для ввода данных в машины использовались перфокарты.
- Оператор должен был загрузить в машину информацию.

## 1.3 Ассемблер

- Идея ассемблера была в том, чтобы записать команду в удобном для запоминания виде.
- Это позволяло работать быстрее.
- Это потребовало разработки компилятора с ассемблером.
- Каждый раз писать вычисления  $\cos$ ,  $\sin$  было неудобно, поэтому были созданы библиотеки математических функций.

## 1.4 Компания IBM и появление операционных систем

- Машина первого класса предназначалась для чтения данных с перфокарт в магнитный барабан, который дальше переносился на другой носитель. После окончания вычислений с магнитного барабана информацию нужно было распечатать.
- Все эти переносы делал человек, но для автоматизации перехода с одного задания на другое потребовалось специальное управляющее ПО, которое получило название **операционная система**, так как частично заменяла работу оператора.
- Чтобы управлять этим процессом, был написан специальный язык — **язык управления заданиями**. В результате пакет, который должен был сформулировать программист.
- Специальное ПО могло переключаться на выполнение следующей задачи. На начальном этапе развития этого ПО, программа выполнялась от начала до конца.

Все машины до сих пор строятся по архитектуре фон Неймана.

## 1.5 Третье поколение вычислительных машин

После 1960 года началась эпоха микросхем. Это третье поколение вычислительных машины.

Именно в машинах третьего поколения появилась полноценная архитектура вычислительных машин. Архитектура фон Неймана концептуальная, а в IBM 360 появилась аппаратная архитектура.

### 1.5.1 Канальная архитектура IBM 360

В основу этой архитектуры положен **принцип распараллеливания функций**.

- Чтобы сократить накладные расходы, в состав IBM 360 были введены специальные устройства, процессоры, которые получили название — каналы. Устройство IBM 360 было построено на основе канальной архитектуры.
- Канал, получив канальную программу, начинал управлять внешнее устройство. Процессор от внешнего устройства отключался и мог перейти на выполнение другой работы.

### 1.5.2 Система прерываний

- Появилась полноценная система прерываний, которая соответствует современной системе.
- Если операциями ввода-вывода управляет канал, то необходимо сообщить процессору, что операция завершена. Для этого были включены аппаратные прерывания от внешних устройств.

Два типа каналов:

- 1) Мультиплексорный — позволяет обслуживать
- 2) Селекторный

#### ✱ Claude

Мультиплексорный канал переключается между несколькими медленными устройствами (терминалы, принтеры), обслуживая их почти одновременно. Селекторный канал работает с одним быстрым устройством (диск, магнитная лента) от начала до конца операции ввода-вывода.

В наших компьютерах шинная архитектура.

**Контроллерами** принято называть устройства, входящие в состав компьютера, а **адаптерами** — устройства на материнской плате.

По завершении операции ввода-вывода формируется сигнал прерывания, который приводит к простейшей схеме на контроллер прерывания. Эти прерывания необ-

ходимо обслуживать. Этим занимаются обработчики прерываний. Обработчики прерывания входят в драйвера внешних устройств.

В систему прерываний<sup>1</sup> были включены:

1) Системный вызовы или программные прерывания.

Нам лучше называть это именно системными вызовами, потому что это более ёмкое название.

2) Исключения

3) Аппаратные прерывания. Их принято делить на

3.1) Прерывания системного таймера.

3.2) Прерывания от устройств ввода-вывода.

3.3) Прерывания действия оператора.

Про аппаратные прерывания[new]. Каналы – программируемые, задача - управлять работой внешних устройств, освободить процессор от непроизводительного управления внешними устройствами (внешние - механические, значит медленные). Что значит процессор управляет? Значит, что процессор, например, ждет от внешнего устройства данные (запросил их) и он будет опрашивать готовность устройства, например, клавиатуры, когда же запрошенные данные окажутся в буфере данной клавиатуры. Т.е. будет тратить кванты процессорного времени на опрос внешнего устройства и делать другую работу не может. Выполняется непроизводительная работа. Поэтому реализовано распараллеливание функций и управление устройствами передано каналам (канальная архитектура). В результате появилась группа аппаратных прерываний от устройств ввода вывода. При этом прерывание от устройств ввода вывода информирует процессор о **завершении** операции ввода вывода.

Также включены прерывания действий оператора.

IBM 360 — система мультипрограммной пакетной обработки, т.е. в памяти одновременно большое количество программ и система переключается между ними.

IBM 360 серия машин, разной мощности и разного состава. В результате покупатель мог купить машин которая наиболее полно соответствовала его потребностям. В СССР — ЕС ЭВМ (единая серия).

История развития и создания ЭВМ в СССР от Рязановой.

### 1.5.3 Пакетная обработка

#### Определение 1.2 (Пакет)

Пакет — это



---

<sup>1</sup>Под interrupt понимается именно аппаратное прерывание.

Пакет — это набор заданий (программ), объединённых вместе для последовательного выполнения на компьютере без вмешательства оператора. Пакет включает в себя саму программу, данные для неё, и управляющие директивы на языке управления заданиями (JCL), которые указывают системе, какие ресурсы нужны программе и что с ней делать.

В ранних системах пакетной обработки оператор физически собирал несколько перфокарт с разными заданиями в один пакет, загружал его в машину, и система автоматически выполняла все задания одно за другим, что значительно повышало эффективность использования компьютера по сравнению с ручной загрузкой каждого задания отдельно.

Основой работы любого компьютера и любой системы являются прерывания.

В современном мире аппаратные вызовы считают «злом», которые происходят очень часто. Важным аспектом является сетевое взаимодействие, а это огромное количество прерываний. Каждый приходящий пакет — это прерывание.

## 1.6 Четвёртое поколение ЭВМ

Появление «больших» интегральных схем, совершенствовались в сторону создания больших по насыщенности элементами интегральных схем. Разработка системы Maltics.

У программиста была потребность интерактивно участвовать в редактировании кода.

- Появилось больше по насыщенности интегральных схем
- Возможность включения в интегральную схему привело к тому, что в наших компьютерах одна сверхбольшая ИС, что дает большее быстроедействие.
- Появились внешние устройства
- Появились мониторы, позволяющие оперативно выводить на экран информацию (строки, буквенно-цифровой интерфейс)

### Определение 1.3 (Терминал)

Терминал — это совокупность монитора и клавиатуры.



Maltics. Разработчики UNIX работали в коллективе, который разрабатывал машину и ОС Maltics. Система создавалась как обучающая система (система разделения времени, sharing time system). К центральному вычислителю удаленно (по тел кабелю) подключены терминалы, за каждым терминалом — пользователь. Каждый пользователь должен работать с машиной так, будто сидит за ней один.

Квантование процессорного времени — решение.

## Определение 1.4 (Квант)

Квант — это отрезок времени.



Время процессора делится на кванты и каждой задаче выделяется квант. Это потребовало изменения операционной системы, изменения алгоритмов планирования выполнения работы.

### ✱ Claude

Квантование процессорного времени — это основа систем разделения времени (time-sharing). Без квантования один процесс мог бы занять процессор надолго, и другие процессы ждали бы своей очереди слишком долго. С квантованием каждый процесс получает небольшой отрезок процессорного времени (например, 10-100 миллисекунд), после чего процессор автоматически переключается на следующий процесс в очереди. Когда квант истекает, процесс снимается с выполнения и помещается обратно в очередь готовых процессов, даже если он ещё не завершил свою работу.

Благодаря такому быстрому переключению между процессами создаётся иллюзия, что все программы выполняются одновременно. Пользователь за терминалом не замечает, что процессор на самом деле переключается между десятками процессов, потому что каждый процесс получает процессорное время достаточно часто. Это и есть квазипараллельное выполнение — на уровне отдельных команд процессы выполняются последовательно, но с точки зрения пользователя они работают параллельно.

## 2 Unix

### 2.1 История создания Unix

В 1964 компания MIT создала операционную систему CTSS (Compatible Time Sharing System, совместимая система разделения времени), для IBM 7094. До этого системы могли выполнять только обработки пакетов. Появились системы разделения времени, время делится, каждой задаче — квант. Одна из них — **Multics** (Multiplexed Information and Computing Service), 1965 год.

(на РТР7 чтобы опробовать свои идеи. они описали идеи, которые вложили в разработку системы. было описание в статье система разд времени юникс. )

Они описали идеи, которые вложили в разработку своей системы.

Unix — универсальная многопользовательская интерактивная операционная система (полноценная версия была написана для РТР11). Для обеспечения работы большого числа пользователей, которые сидели за удалёнными терминалами.

Система предлагает ряд функциональных возможностей, редко встречающихся даже в более крупных операционных системах, в том числе:

1) Иерархическую файловую систему, включающую отсоединяемые тома



- 2) Совместимые файлы, устройства и межпроцессный (от слова процесс) обмен данными
- 3) Возможность запускать асинхронные процессы
- 4) Системный командный язык, выбираемый пользователем
- 5) Более 100 подсистем, включающих десятки языков

## 2.2 Основные концепции Unix

### 2.2.1 Процессы и файлы

На первом месте поставили файловую систему. (Начали разработку юникс с разработки файловой системы)

Разработчики ввели ряд базовых абстракций, самой главной из них является процесс.

#### Определение 2.1 (Процесс)

Процесс — это программа в стадии выполнения.



#### ! Основная парадигма Unix

В Unix всё — файл.

Устройство тоже файл. Эта идея позволила решить все задачи компактно, при том структурным кодом.

**Linux** — Unix-подобная операционная система, в которой реализованы все парадигмы.

### 2.2.2 Язык программирования Си

Важным моментом разработки Unix стало создание языка Си. Его написали для себя, а не как коммерческий. Разработчики вложили в этот язык то, что им было нужно. Си считают языком системного программирования. В нём есть явное определение адреса и возможность непосредственного обращения к нему.

### 2.2.3 Файловая система

#### Определение 2.2 (Файл)

Файл — любая поименованная совокупность данных.



Любая поименованная последовательность байт, хранящаяся во вторичной памяти. Задача — долговременное хранение данных на энергонезависимых носителях. С точки зрения полной работы операционной системы, файлы участвуют в управлении памятью.

- Файлы предназначены для долговременного хранения данных на энергонезависимом носителе.
- Файлы участвуют в управлении памятью.

Заметили, что пользователю удобнее использовать дерево каталогов (иерархическую файловую систему).

Работа в интернете – включение сетевой подсистемы в наши системы.

В результате РТР11 с открытыми кодами ядра были доставлены в учебные ведущие центры Америки. Успех Windows - в убойных приложениях (ворд, эксель, аксес). Си завоевал мир за счет своего ФАНТАСТИЧЕСКОГО качества.

## **3 POSIX**

### **3.1 История стандартизации**

Portable Operating System Interface based on Unix.

BSD — Berkeley System Distribution. Другая версия – Unix System5/ Операционные системы стали значительно отличаться друг от друга набором системных вызовов. В результате ПО для BSD нельзя было выполнять на System5 и наоборот.

Был создан стандарт, а операционные системы, которые учитывают этот стандарт, стали называть POSIX-совместимыми.

Этот стандарт для Maintaining Compatibility Between OS (Поддержание совместимости между Операционными Системами).

### **3.2 Ричард Столлмен и движение свободного ПО**

Ему принадлежит идея создания открытого и независимого ПО, то есть доступного для любого программиста, — GNU (от английского GNU is not Unix). Над GNU он начал работать с сентября 1983 года. Им был разработан текстовый редактор Emacs. Всё это в рамках идеи создания ПО с открытыми исходными кодами. Но при этом ему не хватало операционной системы. И в 1991 году Линус Торвальдс начал работать над ОС под этой лицензией и создал GNU/Linux (сейчас просто Linux).

## **4 Структура операционной системы**

ОС оперирует процессами, файлами, потоками (все абстракции). Вся ОС построена на наборе соответствующих абстракций. Процесс – основная абстракция ЛЮБОЙ операционной системы.

#### Определение 4.1 (Операционная система)

Операционная система — комплекс программ, которые совместно управляют ресурсами вычислительной системы и процессами, которые используют эти ресурсы.



### 4.1 Ресурсы вычислительной системы

К ресурсам вычислительной системы относятся:

- 1) Процессорное время
- 2) Объём оперативной памяти
- 3) Устройства ввода/вывода
- 4) Запоминающие устройства
- 5) Кольца защиты (можно включить, у мальтекс - 16, интел - 4, в системах общего назначения, обычно 0 и 3)
- 6) Коды самой операционной системы
- 7) Данные (различные таблицы и т.п)

При этом важно, что ОС должна иметь реентерабельные коды или чистые (коды чистых процедур).

### 4.2 Реентерабельные коды

В теле процедуры можно изменить переменные. В реентабельных процедурах переменных нет, нельзя ничего изменить (тело процедуры / код изменить нельзя)

#### Определение 4.2 (Реентерабельные коды)

Реентерабельные процедуры — это процедуры, которые не модифицируют сами себя. Из них вынесены переменные и сосредотачиваются в системных таблицах. В результате процессы, заинтересованные в выполнении одной и той же процедуры, получающие в своё распоряжение эту процедуру, могут находиться в разных точках кода, поскольку процедура чистая.



### 4.3 Два взгляда на операционную систему

Процесс — программа во время выполнения, основная абстракция системы, более того - единица декомпозиции системы.

Именно процессом выделяются ресурсы. Поток не является «самостоятельной» единицей. Поток стал единицей диспечеризации. Потокам выделяется процессорное время. Потоки разделяют адресное пространство процесса, своего адресного пространства не имеют. Запрашивают ресурсы потоки, а владельцем ресурса является процесс. За время своей жизни процессы запрашивают различные ресурсы, система должна эти запросы удовлетворять.

Существует два взгляда на ОС:

- Inside (professional взгляд) — взгляд изнутри, основная задача ОС — управление ресурсами и выделение этих ресурсов процессам.
- Outside (для обычного пользователь) - ОС — обычный интерфейс.

## 5 Управление процессорами

Это планирование и диспетчеризация. (Программу пишем для решения определенной задачи).

Процесс за время своей жизни переходит из одного состояния в другое. Это определяется базовой диаграммой состояния процесса.

### 5.1 Диаграмма состояний процесса

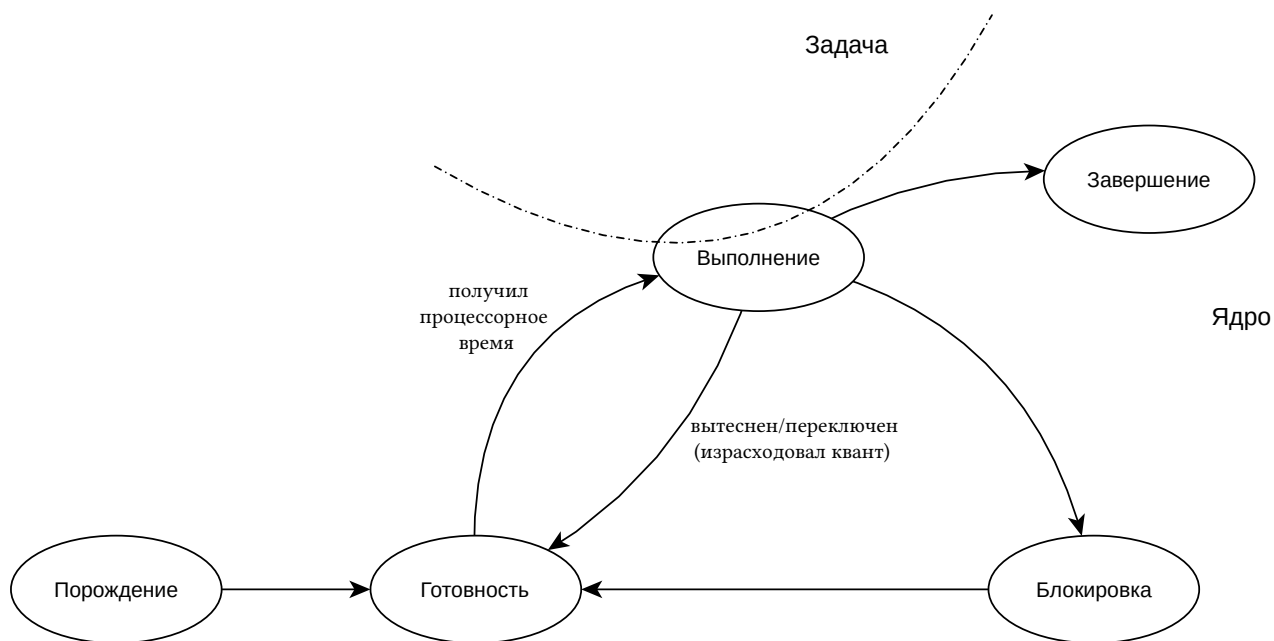


Рис. 1. Диаграмма состояний процессов

#### 5.1.1 Порождение

Состояние, когда собственно запускается программа на порождение.

#### 5.1.2 Создание процесса и выделение дескриптора

Прежде чем процесс начнет выполняться, он должен быть идентифицирован в системе и ему должен был выделен дескриптор, то есть описатель, это структура ядра. В современных версиях Linux эта структура занимает 11-12 страниц. Он нужен системе, чтобы управлять процессом и выделять процессу запрошенные им ресурсы.

После того, как процессу выделили дескриптор, процесс все еще в состоянии порождения.

#### 5.1.3 Загрузка в память

Загружается в память, должен быть выделен основной ресурс – память.

Согласно архитектуре фон-Неймана весь выполняемый код должен находиться в ОЗУ, поэтому процессу должна быть выделена память.

#### **5.1.4 Состояние готовности (Ready)**

После этого процесс переходит в состояние готовности (Ready to Run, ready list). Ready list не просто список, а очередь готовых процессов. В состоянии готовности одновременно может находиться много процессов, которые конкурируют за процессорное время.

Выделяют ready list и рангуют.

В этой очереди находятся процессы, получившие все необходимые ресурсы, кроме последнего – процессорного времени.

#### **5.1.5 Состояние выполнения (Running)**

Из состояния готовности процесс может перейти в состояние выполнения. Для этого процессу необходимо процессорное время, ведь именно процессор выполняет код.

На самом деле система не различает Run, Runnable состояния процесса. Внутри различается, с точки зрения состояния процесса и информации о процессе. В поле state это все обозначается буквой R, Run or Runnable(в макосях просто rannable).

#### **5.1.6 Состояние завершения (Terminated)**

Из состояния выполнения процесс может перейти в состояние завершения. Это очень важное состояние. В этом состоянии у процесса отбираются все ресурсы и возвращаются в пул свободных ресурсов системы.

#### **5.1.7 Состояние блокировки (Blocked)**

И так же в системах с распараллеливанием функций из состояния выполнения процесс может перейти в состояние блокировки. Если или процесс запросил ввод/вывод и тогда он будет блокирован, пока устройство не завершит операцию ввода/вывода, или запросил ресурс системы, который не может быть получен сразу.

После того как процесс получает необходимый для продолжения ресурс, он переходит в состояние готовности.

#### **5.1.8 Системы разделения времени**

Когда появились системы, которые могли выводить на экран текстовую информацию, появилась возможность управлять непосредственно программой через терминал. Это и есть системы разделения времени.

Поскольку время делится между пользователями, это делается это путем квантования процессорного времени.

Алгоритмы планирования в этих типах систем будут отличаться.

### **5.2 Алгоритмы планирования (scheduling)**

Алгоритмы планирования рассматриваются с точки зрения типов операционных систем. В мультипрограммных системах пакетной обработки.

Оценка эффективности машинок – количество выполненных заданий в единицу времени.

### 5.2.1 Классификация алгоритмов

Классификация алгоритмов:

#### 1) Планирование без переключения (1)

Алгоритм FCFS (в современных системах). First Come - First Served (по принципу FIFO). Без переключения, без приоритетов, без вытеснения, процесс выполняется от начала до конца.

Процессы за время своей работы могут запросить ввод/вывод. Тогда процессы будут блокированы и не будут находиться в очереди готовых процессов {тут сказала, что внесла что-то новое}.

#### 2) Планирование с переключением

Необходимо выполнять планирование выполнения процессов, при этом необходимо учитывать, что процессы получают только отрезок процессорного времени и израсходовав квант, процесс должен быть снят с выполнения, чтобы квант мог быть предоставлен другому процессу в очереди. Это планирование с переключением (процессор переключается между процессами)

Алгоритм RR (Round Robin), первый для систем разделения времени. Используется для систем разделения времени. Здесь добавляется линия, обозначающая что процесс потерял квант (процессор в результате того, что исчерпал квант). Без приоритетов.

Супостаты называют это вытеснением, но мы назовем это переключением. «Они убогие, они не различают.»

#### 3) Планирование без приоритетов

#### 4) Планирование с приоритетами

##### 4.1) С вытеснением (3)

Алгоритм SRT (Shortest Remaining Time, Наименьшее Оставшееся Время). Процесс мог быть вытеснен с выполнения, если в очень готовых процессов поступил процесс с меньшим оценочным временем выполнения, чем время, оставшееся выполняющемуся процессу до завершения. Он вытеснялся в очередь готовых процессов и из его оценочного времени вычиталось полученное им процессорное время. Но еще более усиленное бесконечное откладывание (не объяснила подробно).

##### 4.2) Без вытеснения (2)

Алгоритм SJF (Shortest Job First), наикратчайшее задание первое. Должна быть априорная (до опыта) информация о времени выполнения задания. В резуль-

тате возникла ситуация: задания, которым требуется много процессорного времени, постоянно откладывались (потому что преимущество получали короткие задания, это приоритетное планирование). Возникла ситуация, которая получила название «бесконечное откладывание». Приоритет можно было купить, заплатив деньги (просто ее замечание о былых деньгах).

HRRN (Highest Response Ratio Next) (4). Был разработан для того чтобы убрать негативные последствия откладывания, здесь приоритет процесса вычисляется по формуле

$$P = \frac{t_w + t_s}{t_s}$$

$t_s$  — запрошенное время выполнения

$t_w$  — время простоя в очереди готовых процессора.

В результате учитывается время простоя и приоритет процесса увеличивается и в конечном счете он получит процессорное время.

### 5.2.2 Планировщик и диспетчер

Планирование затрагивает не только вопрос выделения процессорного времени. Может также затрагивать вопрос выделения любого дефицитного ресурса в системе. Но прежде всего по отношению выделения процессу процессорного времени.

#### Определение 5.1 (Планировщик)

Планировщик — программа, отвечающая за порядок доступа процессов к ресурсу. И если речь идёт о процессорном времени, отвечает за то, в каком порядке процессы будут получать процессорное время.



Кроме планировщиков в системе присутствует диспетчер. В некоторых системах планировщик и диспетчер объединены.

#### Определение 5.2 (Диспетчеризация)

Диспетчеризация — непосредственное выделение процессорного времени процессу.



У планировщика есть определенные задачи, которые он должен выполнять и характеристики «хорошего» планировщика.

Количество выполненных заданий в единицу времени служит мерой эффективности работы компьютера, поэтому эффективнее всего выполнять короткие задачи.

Необходимо выполнять планирование выполнения процессов, при этом необходимо учитывать, что процессы получают только отрезок процессорного времени, и



израсходовав квант, процесс должен быть снят с выполнения, чтобы квант мог быть предоставлен другому процессу в очереди. Это планирование с переключением (процессор переключается между процессами).

### 5.2.3 Определение длины кванта

Когда возникла задача переключения, возникла и задача определения длины кванта. Если взять квант очень небольшим, это приведет к частым переключениям, это значит, что необходимо постоянно переключать контекст процесса. Это можно сопоставить с вызовом подпрограммы или переходом на выполнение кода обработчика прерывания (в коде 8 прерывания).

Выполнение программы прерывается системой, начинает выполняться код обработчика. После того как код обработчика завершился, необходимо восстановить выполнение прерванной программы. Для этого в 8 прерывании мы видели сохранение точки возврата и флага, но если идёт переключение между задачами, то обработчик начнется с сохранения 4-х регистров, а заканчивается восстановлением содержимого этих 4-ех регистров. Если речь идёт о переключении контекстов, то необходимо восстановить полный контекст (содержит аппаратный контекст, т.е. информацию о регистрах, и информацию о ресурсах процесса). Переключение полного контекста это очень большая задача, большие накладные расходы. Поэтому если квант слишком маленький, то огромное время процессора будет тратиться на переключение полного контекста.

Как разработчики решают эту задачу? Квант выбирается таким образом чтобы большее количество заданий за выделенный квант успевало или завершиться, или выдать запрос на ввод/вывод. Процессы очень часто запрашивают ввод/вывод, поэтому именно он (хотя в системе большое количество других ресурсов).

«Это свеженькое! (про формулу)»

$$\text{Накладные расходы по переключению контекстов} = \frac{c}{Q + c}$$

$c$  — время переключения контекста, оно оценивается в системе.

$Q$  — квант.

### 5.2.4 Приоритетное планирование

В современных системах используется приоритетное планирование. Важно, поскольку системы (особенно общего назначения) выполняют разные процессы, вплоть до процессов реального времени. Учитывать особенности алгоритма можно с помощью приоритетов.

Приоритеты:

### 1) Статические

Назначаются процессу до начала его выполнения и в процессе выполнения не меняются.

### 2) Динамические

Меняются во время его выполнения.

Multilevel Queues (Многоуровневые Очереди). В системе имеется ряд очередей, в которых процессы крутятся по RR. Приоритет процессу назначается изначально (или реального времени, или системы, или интерактивный), мы не видим здесь пересчёта.

Для ОС Windows декларируется 256 очередей, которые выполняются по алгоритму RR.

Во второй части первой ЛР будет затрагиваться этот вопрос. Для процессов учитывается время простоя в очереди готовых процессов.

!!!добавить из лабы данных про планирование

### 5.2.5 Многоуровневые очереди с обратной связью

Классическим алгоритмом является алгоритм (многоуровневые очереди — Дейтел), реализуется не как совокупность отдельных очередей по алгоритму RR, а как связанных между собой очередей.

В системе несколько очередей разного уровня, по FIFO, а последняя по RR. В первую очередь поступают процессы вновь созданные или завершившие ожидание (оидание завершения ввода/вывода) ввода вывода. Если процесс не успел завершиться или выполнить запрос ввода вывода за выделенный ему квант, он перемещается в следующую более низко-приоритетную очередь и так до тех пор пока не окажется в последней самой низко приоритетной очереди.

Вычислительные задачи чаще всего оказываются с самым низким приоритетом. А интерактивные постоянно запрашивают ввод/вывод.

### 5.2.6 Анализ времени ожидания и накладных расходов

Сравним кванты размером 10(short quantum) и 100 мс (long quantum), без учета блокировок (запросов ввода/вывода).

Скажем, у нас выполняется 10 процессов. Тогда 10-й процесс сможет начать выполняться только примерно через 1 секунду. Почему? Каждый процесс выполняется 100 миллисекунд, при этом 5 миллисекунд занимает переключение контекста. так 10-й процесс будет ждать 945 миллисекунд.

Если квант 10 миллисекунд, то время ожидания сократится в 20 раз, но с другой стороны, если квант 10 миллисекунд, то  $\frac{5}{15} = 33.3\%$ , то есть накладные расходы достигнут 1/3 процессорного времени. В то время как для кванта 100 миллисекунд это будет только 4%.

С одной стороны, короткий квант приводит к большим накладным расходам на переключение, длинный квант приведёт к увеличению времени ожидания процесса в очереди на выполнение. Это идеализированная ситуация, когда процессы не запрашивают другие ресурсы, разрешение на ввода/вывод, не блокируются и т.п.

## 5.3 Планирование в многопроцессорных системах

### 5.3.1 Processor Affinity (сродство процессов)

У нас системы многопроцессорные. Это значит, что в наших системах процессы могут выполняться реально параллельно, на разных ядрах. Для многопроцессорных систем вводится понятие Processor Affinity (сродство процессов). Под этим понимается аспект планирования в многопроцессорной системе, в которой планировщик отслеживает на каком процессоре выполняется процесс.

«Решила расширять материал, до этого в курсе игнорировалось.»

Имеется две формы Processor Affinity:

- 1) Hard Affinity — когда процесс выполняется на таком же процессоре. Это подход максимальных усилий, то есть требует дополнительных расходов. При этом планировщик пытается спланировать процесс таким образом, чтобы его можно было перемещать на разные процессоры

#### ✱ Claude

Hard Affinity — это жесткая привязка процесса к конкретному процессору (или набору процессоров). Процесс может выполняться только на указанных ядрах и не может быть перемещён планировщиком на другие процессоры, даже если они свободны. Устанавливается программистом явно (например, через системный вызов `sched_setaffinity` в Linux).

Это противоположность Soft Affinity, где планировщик старается держать процесс на одном процессоре для эффективности, но может переместить его при необходимости (например, для балансировки нагрузки).

- 2) Soft Affinity

Существует два подхода:

- 2.1) Push migration — это когда ОС проверяет загрузку, то есть количество процессов в очереди на выполнения для каждого процессора периодически. Если имеется дисбаланс, то какие-то процессы перемещаются с одного процессора на другой.
- 2.2) Pull migration — миграция по запросу. Если планировщик обнаруживает, что на данном процессоре в очереди на выполнение нет процессов, т.е. очередь пуста, то совершаются «набеги» на очереди других процессоров с целью переместить процессы в свою пустую очередь.

`ps -ajx` позволяет посмотреть потоки ядра на каждый процессор.

В Линукс используется оба способа.

## 5.4 Классы планирования в Linux

Так же вводится понятие, как Scheduling Classes (классы планирования). Linux предполагает использование разных планировщиков.

Определено два класса планирования:

- 1) `sched_fair` (справедливое планирование) — планировщик, используемый в Linux по умолчанию.
  - Он учитывает полученное процессом процессорное время, то есть выполняется динамический пересчёт приоритетов процессов в зависимости от полученного процессом процессорного времени. Процессы, получившие больше процессорного времени, их приоритет понижается.
  - В современных системах учитывается время простоя в очереди готовых процессов. В этом случае чем дольше процесс простаивал в очереди, тем выше становится его приоритет. Такой подход изначально был заложен в Unix, поэтому отбрасывать время ожидания мы не можем.
- 2) `sched_rt` (real-time) — планировщик, выполняющийся для процессов с наивысшим приоритетом по алгоритму RR.

Алгоритмы планирования называются *policy*.

При этом:

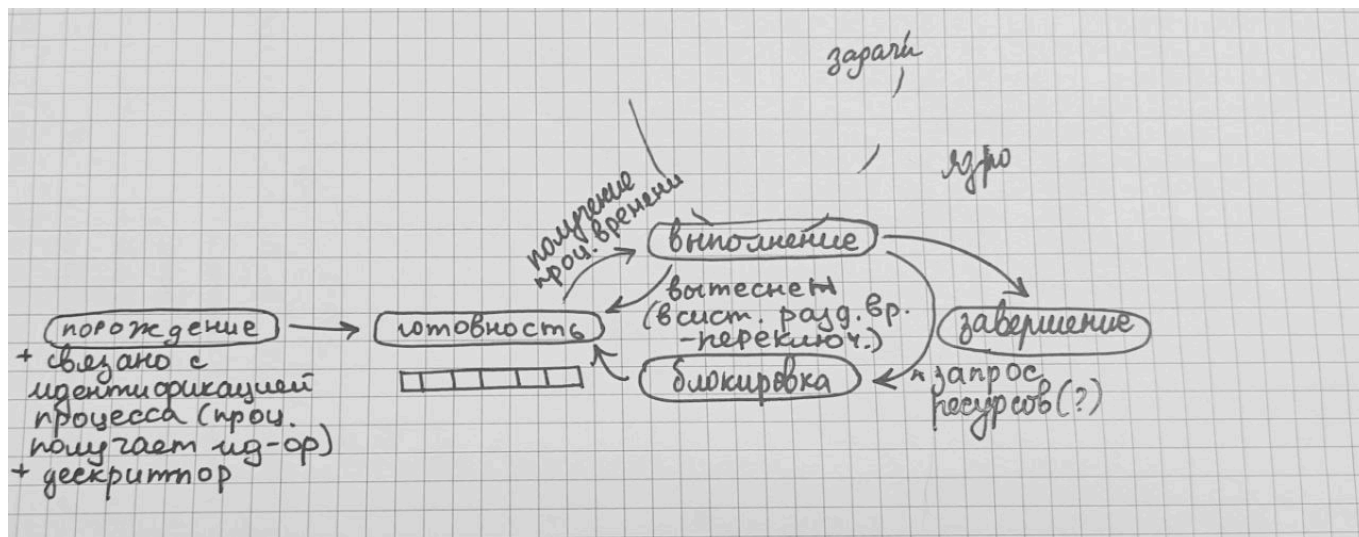
- FIFO не предполагает переключение, процесс выполняется от начала до конца.
- RR предполагает переключение по истечению кванта.

## 5.5 Работа с очередями процессов

Процессы — динамические объекты, это (таблица процессов) затратно, поэтому любая система работает всегда со связными (причем двусвязными) списками. К номеру прилагается дескриптор. Процесс должен получить основной ресурс (после порождения) — оперативную память. Сегодня можно сказать, что процесс должен получить минимально необходимый объем памяти для начала выполнения. После того, как он получил память, он переходит в очередь на выполнение (или очередь готовых процессов, Runnable - процесс, который может выполняться, готовый («готовенький по русски»)). В мультизадачных ОС в этой очереди находится большое количество процессов. В зависимости от того, какое у нас планирование, процесс, скажем, первый из этой очереди получает процессорное время, то есть переходит в состояние выполнения. При этом, как мы видели, ни MacOS, Ubuntu, не различают Runnable и running, системе главное поставить процесс в очередь, а получение процессорного времени это уже диспетчеризация, нижний уровень.

### Важно

Нужно понимать, что процесс выполняется только тогда, когда у него есть процессорное время.



Если процесс запрашивает дополнительные ресурсы, например, данные от устройств ввода или желает что-то вывести на устройства вывода, он блокируется в ожидании завершения ввода/вывода. Но внешние устройства – не единственный ресурс который может быть запрошен процессом. В системе очень много ресурсов. После завершения блокировки процесс возвращается в очередь готовых процессов. Из состояния выполнения он переходит в состояние завершения, которое очень важное для системы. При завершении процесса, ресурсы, занятые им, возвращаются в пул свободных ресурсов системы. В результате, эти ресурсы могут быть выделены другим процессам.

Из состояния выполнения процесс может вернуться в очередь готовых процессов для мультипрограммных систем пакетной обработки с приоритетным планированием, если в очередь поступил процесс с более высоким приоритетом выполнения в результате вытеснения. В системах разделения времени, то есть в системах, в которых процессорное время квантуется, есть переключение (это как раз тоже возврат процесса в очередь готовых процессов), если процесс израсходовал выделенный ему квант, не вызвал какой-то дополнительный ресурс, не обратился к внешнему устройству, соответственно вытеснен или потерял квант переключён.

## 6 Переключение в режим ядра

### 6.1 Режим работы процессора в Unix

Именно в Unix чётко введено понятие процесса.

#### Определение 6.1 (Процесс с точки зрения Unix)

Процесс часть времени выполняется в режиме «задача» и тогда он выполняет собственный код, а часть времени — в режиме код, и тогда он выполняет реентерабельный код ОС



Переменные реентерабельных процедур вынесены в системные таблицы.

Unix различает два режима (прозвучало в определении). Не значит, что в других системах так же.

### ✱ Claude

Unix различает два режима:

- 1) Режим задачи (user mode) — процесс выполняет собственный код приложения.
- 2) Режим ядра (kernel mode) — процесс выполняет реентерабельный код операционной системы.

Процессоры Intel поддерживают 4 кольца защиты. В истоках стоит ОС Multix. В процессорах Intel для разделения режимов выделено два разряда, отсюда 4 кольца защиты. Unix, Linux и Windows работают на двух уровнях привилегий: на 0 (ядро) и 3 (задачи).

## 6.2 События, переводящие систему в режим ядра

Фактически эта информация полностью коррелирует с системой прерываний. Три типа события, которые переводят систему из режима пользователя в режим ядра:

- 1) Системные вызовы (программные прерывания, Software Interrupts, Windows — traps, ловушки).

Это набор функций, которые предоставляются пользователю операционной системой, это не произвольные функции (говорили в POSIX).

- 2) Исключения (исключительные ситуации).

Исключительные ситуации в современных системах бывают двух типов:

2.1) Исправимые — страничные прерывания. После того, как оно обработано системой, программа продолжает выполняться.

2.2) Неисправимые — ошибки кода или системы (сбои). При них программа завершается, как мы говорим – аварийно.

- 3) Аппаратные прерывания

Принято различать три типа:

3.1) прерывания системного таймера (единственное периодическое прерывание системы)

3.2) прерывания от устройств ввода/вывода

3.3) прерывания от действий оператора





## 7 Потоки

Все современные системы поддерживают потоки.

### 7.1 Базовое представление о потоках

#### 7.1.1 Характерные черты процесса

Процесс имеет две основные характерные черты:

1) Владение ресурсами — ownership (владелец)

Современный процесс владеет адресным пространством (виртуальным и защищёнными).

- Виртуальным, потому что современные системы имеют механизмы управления виртуальной памятью.

- Защищённым, потому что в любой мультизадачной системе адресное пространство процесса защищено. Это значит, что ни один процесс не может обратиться в адресное пространство другого процесса. Это связано с тем, что проще контролировать выход процесса за его собственное адресное пространство.

- Адресное пространство есть всегда, но время от времени процесс владеет физической памятью, устройствами ввода/вывода, файлами, объектами ядра.

2) Планирование и выполнение (Scheduling and Execution)

- Выполнение кода на потоках, поэтому поток стал единицей диспетчеризации.

#### 7.1.2 Определение потока

##### Определение 7.1 (Поток)

Поток — это непрерывная часть кода, которая может выполняться параллельно с другими частями кода.



Это может быть и приложение, и код оперативной системы. Часть кода — это и есть наполнение адресного пространства процесса. /Отсюда и следует, что/ поток собственного адресного пространства не имеет, а выполняется в адресном пространстве процесса. Именно поток становится единицей диспетчеризации, то есть именно он получает процессорное время, то есть потоки владеют аппаратным контекстом (чаще говорят, счётчиком команд). Потоки / выполняются, значит / запрашивают ресурсы, но владельцем ресурса остаётся процесс.

#### 7.1.3 Однопоточная и многопоточная модели

Принято разделять многопоточную и однопоточную модель процессов.

БУП - блок управления процессом



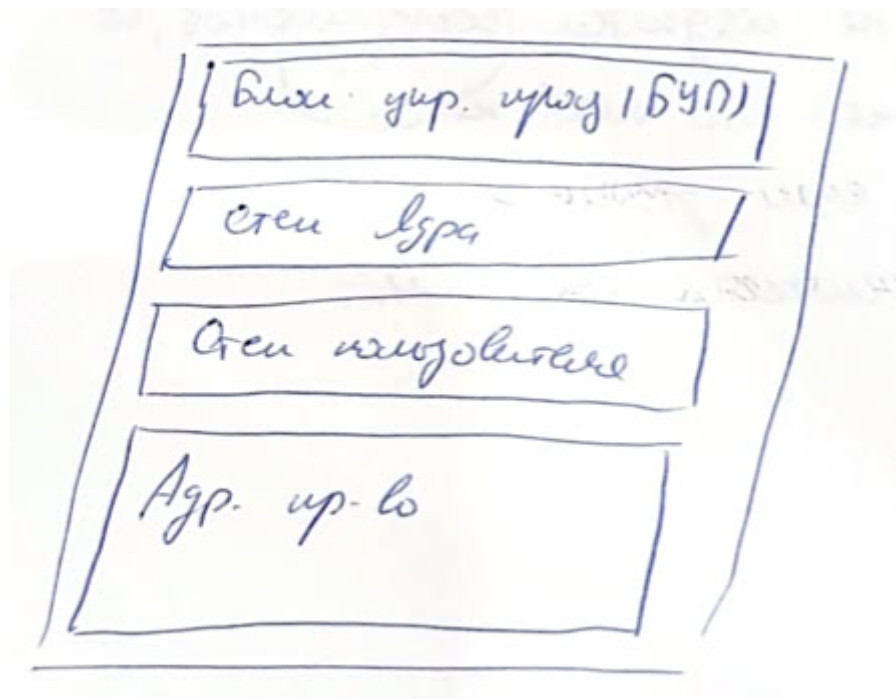


Рис. 2. Однопоточная модель процесса

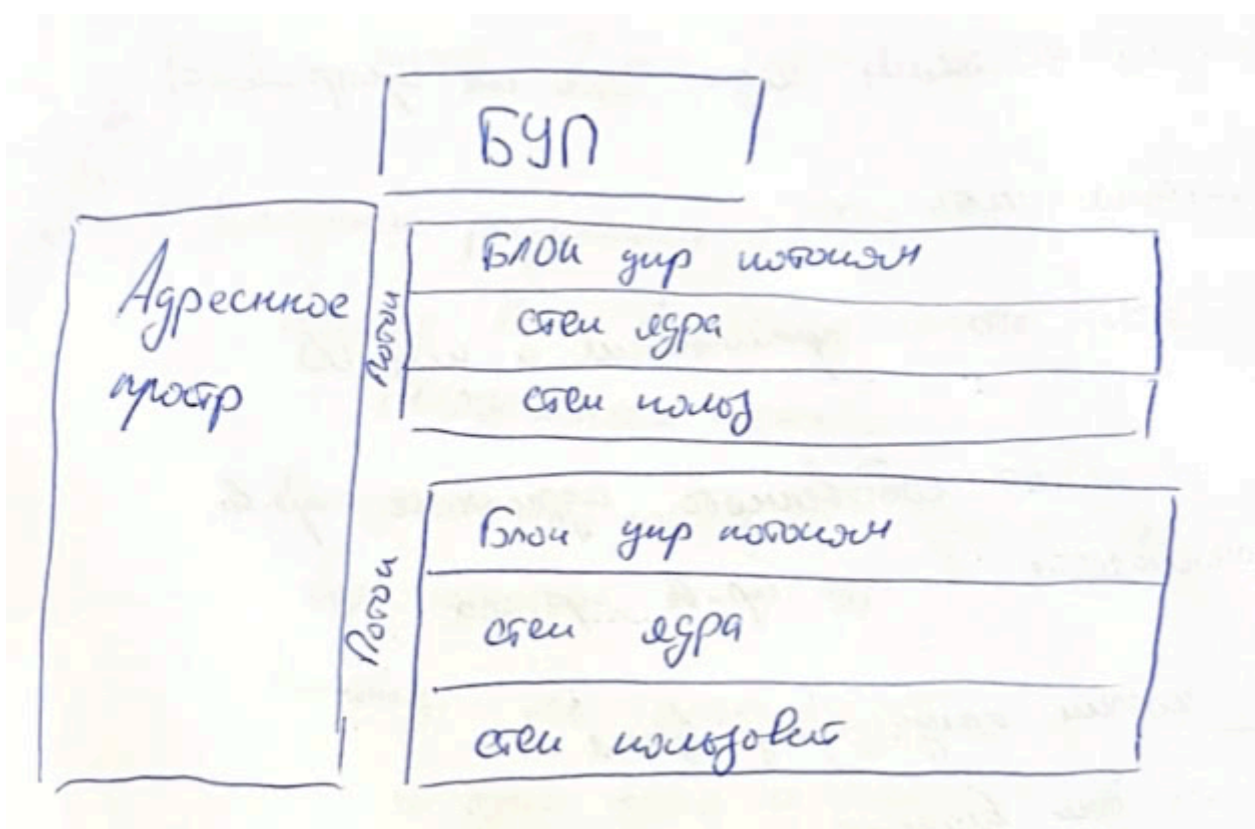


Рис. 3. Многопоточная модель процесса

#### 7.1.4 Стеки потоков

Каждый поток должен иметь стек ядра, который в наших системах назначается автоматически.

Для общности управления потоками, если в процессе не создаётся дополнительных потоков, то считается, что это главный поток. Если в процессе создаются дополнительные потоки, то сам процесс является главным потоком.

Каждый поток процесса будет иметь два стека:

1) Стек ядра

Он создаётся автоматически, это особенность, / можно сказать / конкретных операционных систем /, с которыми мы работаем/.

2) Стек пользователя

### 7.1.5 Типы потоков

Существует 2 типа:

1) Потоки уровня пользователя

- Определяются только в режиме пользователя.
- Ядру о них ничего не известно.
- Для управления должна существовать библиотека соответствующих функций уровня пользователя.
- Переключение между потоками одного процесса требует только переключение аппаратного контекста. Эта мысль стала очень привлекательной и разработчики Unix стали ее дорабатывать. Существуют библиотеки, доступные пользователю, которые предоставляют такие возможности. Все хорошо, до тех пор пока один из потоков не запросит ввод/вывод. Для этого система должна перейти в режим ядра, т.к. ни одна система не разрешает приложениям обращаться напрямую к устройствам ввода/вывода напрямую. Поэтому будет заблокирован целый процесс.

2) Потоки уровня ядра

- В программе определены потоки, но эти потоки являются потоками уровня ядра, то есть создаются системным вызовом, например, `pthread_create`. ( / напоминает, что / Потоки называются `thread` (нить), но в русской технической литературе — принято название поток. Назвали нитями, потому-что есть поток `stream`, но в английском связано с потоками данных.
- Здесь совсем другая ситуация. Ядру известны потоки. Оно переключает потоки. В очереди процессора стоят потоки. Процессору стоит смесь потоков (не сначала потоки одного процесса, потом другого и т.п.). Если у нас есть три программы (см. низ рисунка). Стоит понимать, если осуществляется переключение на поток того же процесса, переключается будет только аппаратный контекст, если на поток другого процесса — полный контекст.

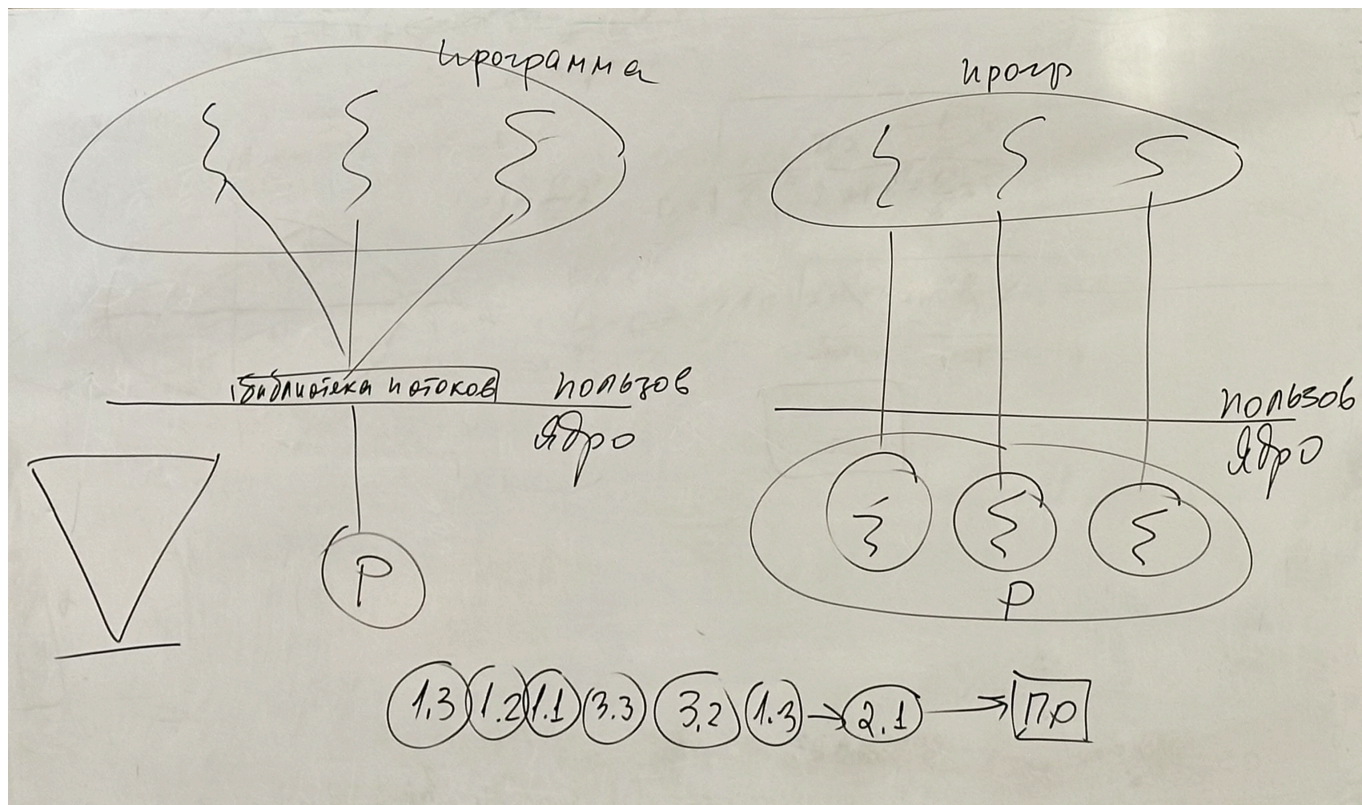


Рис. 4. ???

## 7.2 Мотивация поддержки потоков

Потоки очень полезны в тех случаях, когда процесс выполняет несколько задач, независимых друг от друга, и это особенно справедливо, когда одна задача должна блокироваться по каким-то причинам (наиболее часто — запрос ввода/вывода), при этом другие задачи могут продолжать выполняться. Это касается только потоков уровня ядра.

Классический пример такой программы — современные текстовые редакторы. В таких редакторах фоновый поток может проверять грамматику и правописание, в то время, как другой поток, например, загружает изображение из вторичной памяти или даже из интернета в современных системах; скажем, третий поток выполняет резервное копирование введенного текста, который был отредактирован с помощью потока, который предназначен для ввода текста. То есть современный текстовый редактор — многопоточное приложение.

В современном мире серверов и распределенных систем (сервер — service — обслуживать, процессы-клиенты обращаются к серверу за обслуживанием, таких процессов много) большинство серверов являются многопоточными. И каждый поток на таком сервере создается для обслуживания каждого клиента. Они не всегда должны быть многопоточными.

## 7.3 Преимущества мультипоточности (потоки ядра)

Benefits по-супостатски, по нашему — достоинства.

Речь идет о потоках ядра.

### 1) Отзывчивость

- Один поток может обеспечивать быструю реакцию на запросы пользователя, в то время как другие могут быть заблокированы или выполняться медленно, например, выполняя большой объём вычислений (см. приоритеты интерактивных процессов). Система должна обеспечивать определенный уровень отзывчивости.

### 2) Разделение ресурсов (sharing)

- Поскольку потоки разделяют адресное пространство процесса, они разделяют глобальные переменные и другие ресурсы процесса.
- Главное, что они выполняются в одном адресном пространстве и могут обращаться к глобальным переменным. Это достоинство в том смысле, что процессы, для того, чтобы взаимодействовать, должны использовать средства, которые им предоставляет ядро системы.

### 3) Экономичность

- Если переключение выполняется среди потоков одного процесса, то переключается только аппаратный контекст.
- При этом, поскольку наши системы являются многопроцессорными, то потоки одного процесса могут выполняться параллельно на разных процессорах, РЕАЛЬНО параллельно.
- Переключение аппаратного контекста занимает меньше времени.

### 4) Масштабируемость

- Мультипроцессорные архитектуры. Возможность реальной параллельности, что сокращает время выполнения или приложения, или (если это многопоточный сервер) время обслуживания клиентов.

/совместное исследование со студентами/ Создание потока — затратное действие в системе. То есть имеется в виду, что потоки режима ядра создаются в режиме ядра путем вызова соответствующих функций ядра и редактирования соответствующих структур ядра, то есть это затратное действие.

## 7.4 Проблемы многопоточного программирования

Они выделяют 5 проблем:

### 1) Выделение задач, которые могут выполняться параллельно в конкретном приложении.

- Это проблема, потому что правильное деление на задачи, которые могут выполняться параллельно являются ключом к успеху.
- Тупое объяснение от Рязановой: уже упоминалось, что потоки могут выполняться параллельно, в то время как один поток, запросив ввод/вывод, блокируется. Нет смысла заставлять все потоки запрашивать ввод/вывод, они все время от времени будут блокироваться.
- Нет, ?<sup>2</sup>

---

<sup>2</sup>Нет, потому что гладиолус.

## 2) Сбалансированность

- Поиск задач для параллельного выполнения, которые обеспечивают одни и те же затраты, то есть деление на потоки должно быть сбалансировано и в качестве потока не стоит выделять какую-то тривиальную задачу.
- Примеры плохие от Рязановой: функцией вызывается только один системный вывод, один вызов if в функции.

## 3) Разделение данных (Data Splitting)

- Имеется в виду работа потока с одними и теми же данными, то есть обращение к одним и тем же данным. Это требует специальных подходов, чтобы не терять данные, то есть не формировать ошибочные результаты.
- Это называется «взаимоисключение» / или «монопольный доступ к данным» /

## 4) Зависимость данных (Data Dependency)

- Если потоки обращаются к одним и тем же данным, то один поток изменяет их, а другие потоки заинтересованы в результатах этих изменений. Такая реализация требует синхронизации.
- Это «синхронизация»

## 5) Тестирование и отладка

- Это невозможно в многопоточных приложениях.
- Взаимосвязи между потоками могут быть очень сложными. Их можно выявить промежуточной печатью, причём очень скрупулёзно выполненной.

Монопольный доступ и синхронизация – разные вещи!

## 7.5 Типы параллелизма

Два глобальных типа:

### 1) Распараллеливание по данным (Data Parallelism)

- Используется наиболее широко.
- Примером могут служить алгоритмы машинной графики (трассировка лучей). Есть кадр, состоящий из пикселей. Выделяем фрагменты кадра и можем обрабатывать их параллельно.
- Какие-то / данные или их / части данных могут быть обработаны параллельно.

### 2) Распараллеливание кода (Task Parallelism)

- Самый понятный классический пример — текстовый редактор.
- Выделение задач, которые могут выполняться параллельно, не мешая друг другу, а именно поддерживая выполнение других задач, обеспечивая дополнительный сервис приложения, не замедляя его выполнение в целом.
- Это сложная задача.



## 7.6 Многопоточные модели (Multithreading Models)

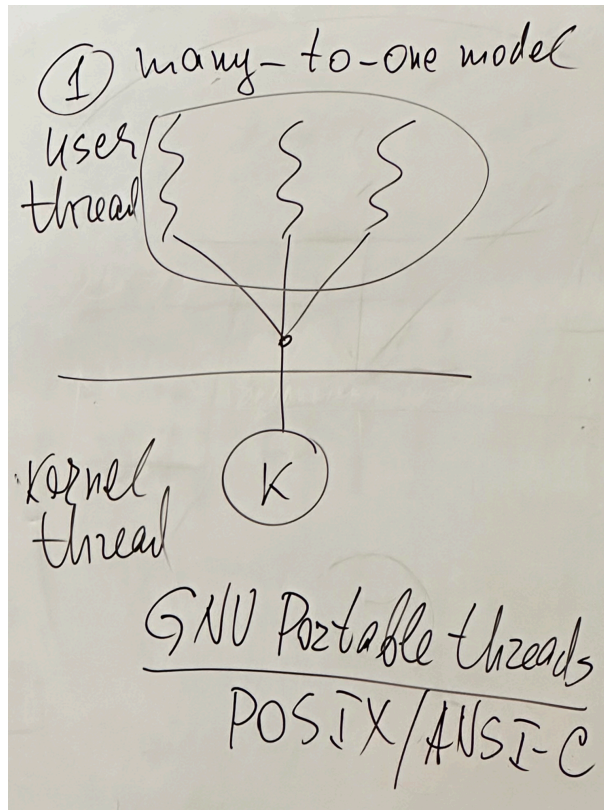


Рис. 5. Many-to-one

Для работы с потоками уровня пользователя, необходимо иметь библиотеку уровня пользователя, которая предоставляет функции переключения потоков, предоставления потокам ресурсов, завершения потоков. Проблема в том, что ядро ничего не знает о потоках уровня пользователя и выполняет то что здесь названо потоком ядра. Дело в том, что есть такая ОС как solaris, в ней реализованы все типы потоков.

GNU Portable threads — это потоки POSIX/ANSI-C уровня пользователя

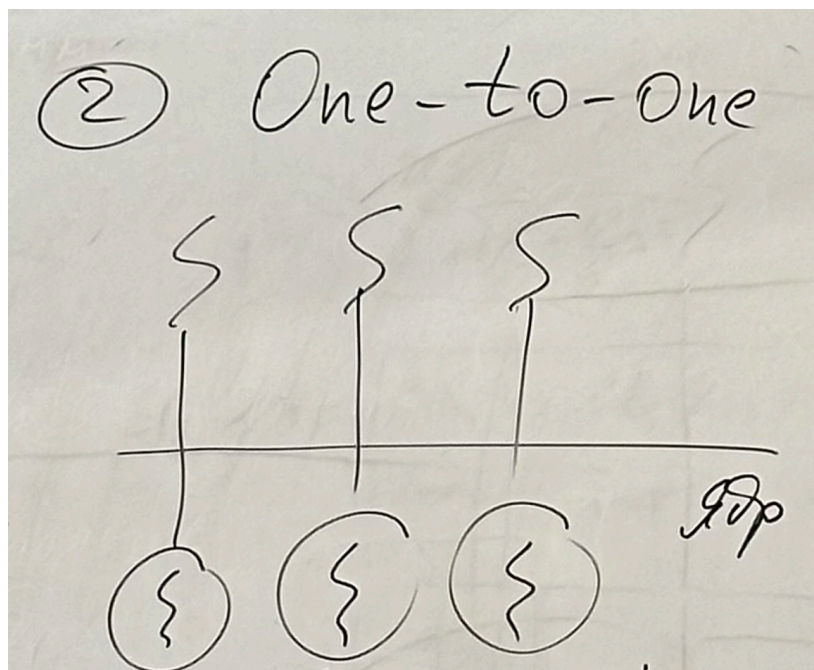


Рис. 6. One-to-one

Это уже потоки ядра, в этой модели каждому пользовательскому потоку соответствует поток ядра. Тут предполагаются потоки разных процессов. Windows, Linux реализуют эту модель.

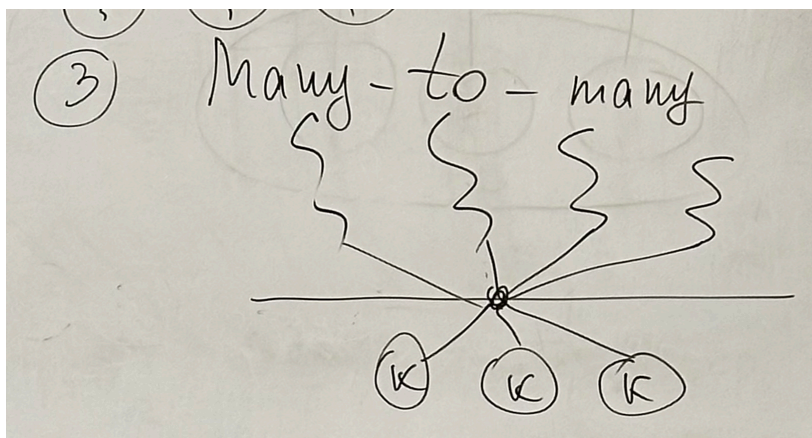


Рис. 7. Many-to-many

- Мультиплицирует любое количество потоков уровня пользователя на равное или меньшее количество потоков ядра.
- Сколько потоков ядра выделять для каждого приложения, определяется системой.

Есть модели, расширяющие 3-ю модель и учитывающие 2-ую.

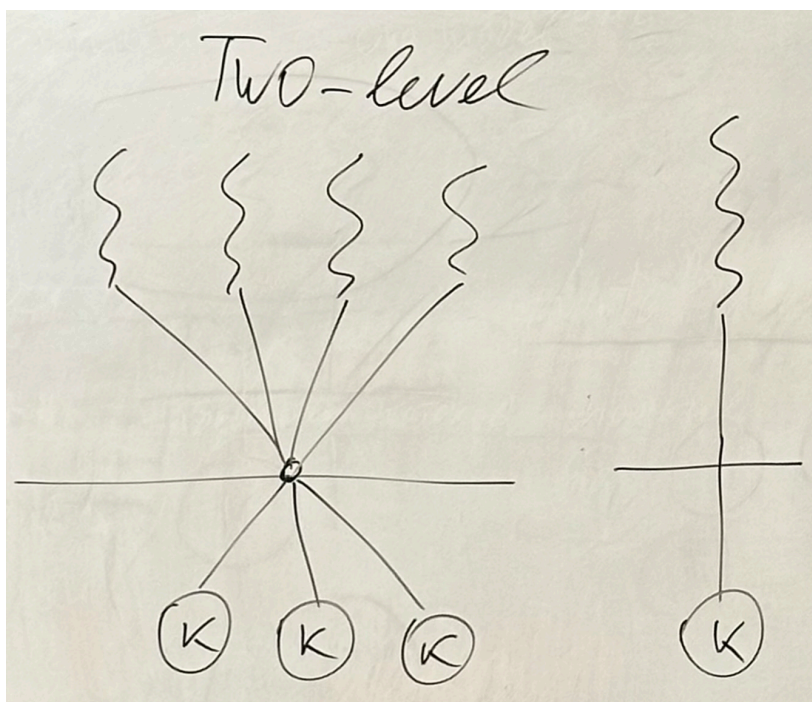


Рис. 8. Two-level, двухуровневая модель

Система в зависимости от ситуации может поддерживать как Many-to-many модель, определять количество потоков ядра, которые могут быть созданы для конкретного приложения, то есть не учитывать количество потоков, которые создаются в приложении, а учитывать, так сказать, свои возможности, так и one-to-one модель

Характерно для Solaris.

### ✱ Claude

Solaris — это Unix-подобная операционная система, разработанная компанией Sun Microsystems (позже приобретённой Oracle). Она была создана в начале 1990-х годов и стала одной из самых важных коммерческих версий Unix.

В контексте многопоточности Solaris имеет особое значение, потому что именно в этой системе была впервые реализована и детально описана двухуровневая модель потоков (two-level model). Solaris поддерживала как потоки уровня пользователя, так и потоки уровня ядра, и могла гибко отображать их друг на друга в зависимости от потребностей приложения.

Многие концепции и решения, впервые реализованные в Solaris (включая модели потоков, планирование в многопроцессорных системах, механизмы синхронизации), позже были заимствованы другими операционными системами, включая Linux.

## 7.7 Библиотеки потоков

Для потоков ядра система предоставляет API, системные вызовы по-другому. В коде ядра Linux всё собрано в определённой библиотеке. Библиотеки такие могут быть уровня ядра, а могут быть уровня пользователя.

3 основные библиотеки:

- 1) pthreads (POSIX Threads)
  - Определяет как библиотеку уровня пользователя, так и уровня ядра
  - Зависит от системы
- 2) win32 threads (kernel)
  - Библиотека уровня ядра
  - Зависит от системы
- 3) java threads
  - Надо учитывать, что Java в основном выполняется на виртуальной машине JVM (Java Virtual Machine)
  - Зависит от JVM.

Системный вызов POSIX — pthread\_create. В Windows — CreateThread

## 7.8 Lightweight Processes

### Определение 7.2 (Lightweight Processes)

Lightweight Processes — / можно сказать, что это / абстракция для разработчиков для реализации процессов





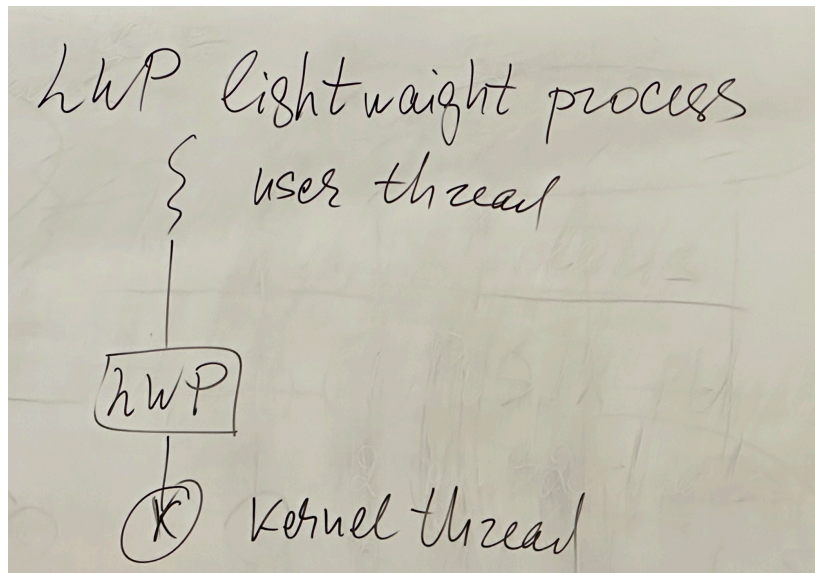


Рис. 9. Диаграмма работы lightweight processes

Все о чем говорим, в системе реализовано в виде структур. Дополнительная промежуточная структура может быть проинициализирована заранее в системе. В итоге создаваемый поток связывается с этой проинициализированной структурой, а она в свою очередь с соответствующей структурой, описывающей поток на уровне ядра.

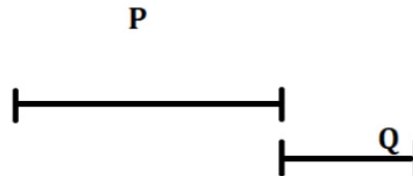
## 8 Взаимодействие параллельных процессов

### 8.1 Уровни наблюдения

Выделяют три варианта

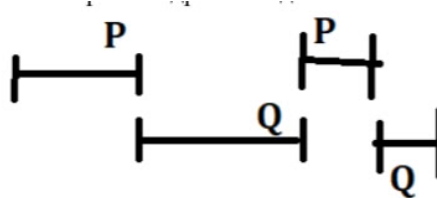
#### 1) Последовательное выполнение

- Сначала выполняется первая программа от начала до конца, после этого начинает вторая от начала до конца и так далее.



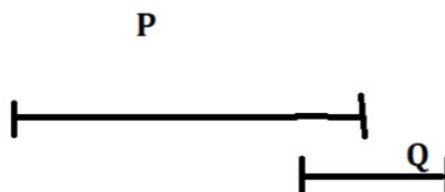
#### 2) Квазипараллельное выполнение

- Связан с квантованием процессорного времени
- Для первой программы выделен квант, он истёк. Квант выделяется для второй программы и так далее.
- На уровне команд это последовательное выполнение, но пользователю кажется, что оно параллельное, отсюда и название.
- Это характерно для однопроцессорных систем (но в какие-то моменты может быть и в многоядерных системах).



#### 3) Реальная параллельность

- Выполнение кодов совпадает по времени.
- 



### 8.2 Проблемы параллельного выполнения

При этом это рассуждение проводится для того, что проблемы, связанные с взаимодействием параллельных процессов, будут одни и те же для квазипараллельного выполнения и реальной параллельности.

### 8.2.1 Пример с банковским счётом

Рассмотрим пример. Два филиала банка или фирмы в конце дня должны перечислить на один и тот же счёт выручку. Нас не интересует интерфейс, а интересует только зачисление выручки.

(возможно в этом году было другое)

| P1                | P2                               |
|-------------------|----------------------------------|
| $S := S + D1$     | $S := S + D2$                    |
|                   | mov                              |
|                   | add                              |
|                   | потерял квант                    |
| получил квант     |                                  |
| mov               |                                  |
| add               |                                  |
| загрузил в память |                                  |
|                   | получил квант, загрузил в память |
| $S = S + D1$      | $S = S + D2$                     |

...Квант потерян. Сохраняется аппаратный контекст, т.е. сохраняется значение аккумулятора (AX). Начинает выполняться действие от другого банка, выручку перевели в регистр, в аккумулятор записали значение суммы, сложили, записали в память, квант потерян. Квант получает второй филиал. Он из памяти уже все прочитал. Программа выполняется последовательно, в аккумуляторе уже сумма, осталось записать ее в ячейку памяти по адресу, в результате выручка потеряна.

Два процесса выполняют они и те же действия. Это можно представить следующим образом:

|   |                |  |                |
|---|----------------|--|----------------|
| 1 | P1: ...        |  | P2: ...        |
| 2 | mov eax, myvar |  | mov eax, myvar |
| 3 | inc eax        |  | inc, eax       |
| 4 | mov myvar, eax |  | mov myvar, eax |

Листинг 1.

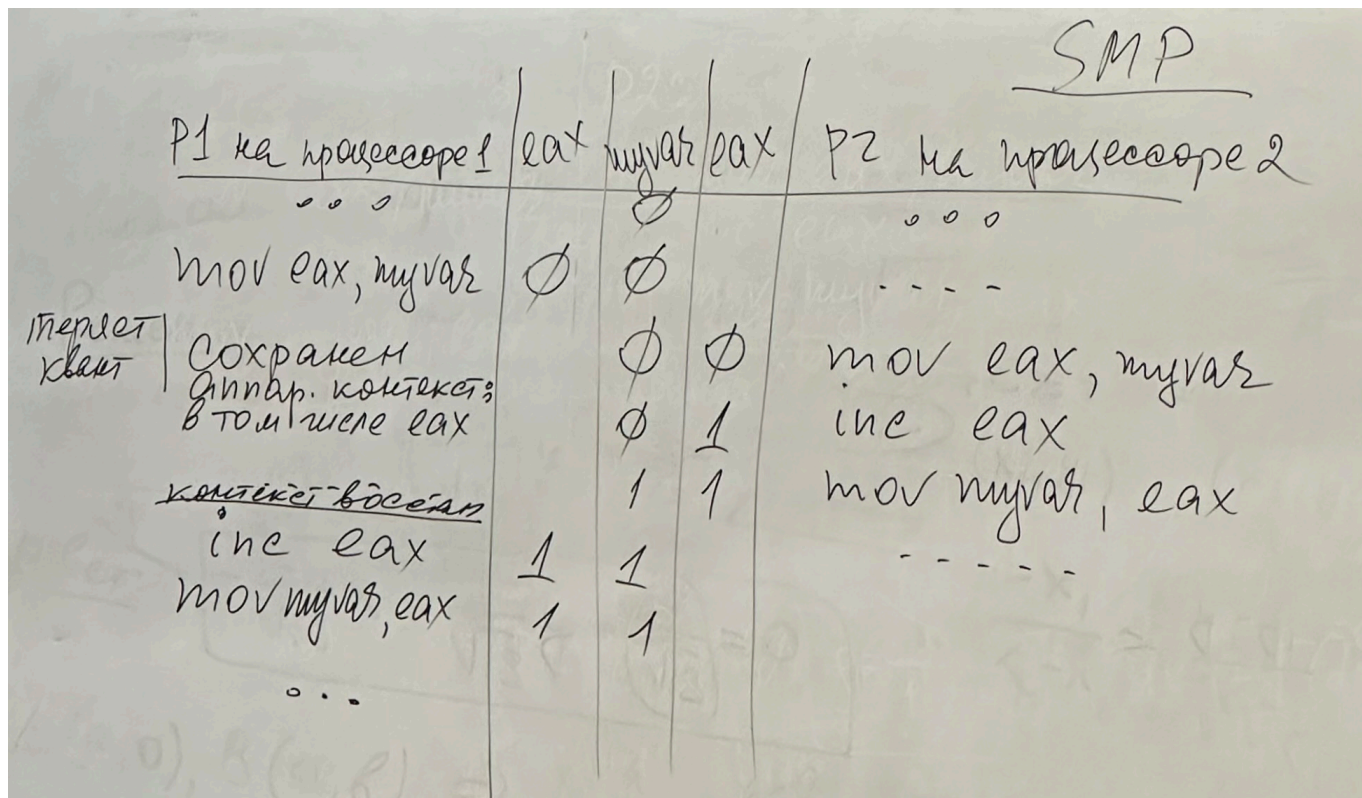


Рис. 13. Диаграмма выполнения на разных процессорах

- 1) Пусть первый процесс получает квант первым и выполняет команду и помещает переменную muvar в регистр EAX. Первый процесс теряет квант.
- 2) Второй получает процессорное время несколько позже, выполняет команды и переходит к другой работе.
- 3) Когда P1 потерял квант, был сохранен аппаратный контекст, в том числе содержимое EAX. Получив квант, P1 продолжает выполнение программы. После этого записывает EAX в muvar.
- 4) В итоге мы потеряли единицу  $\Rightarrow$  потеряли данные, а это самое страшное в программировании

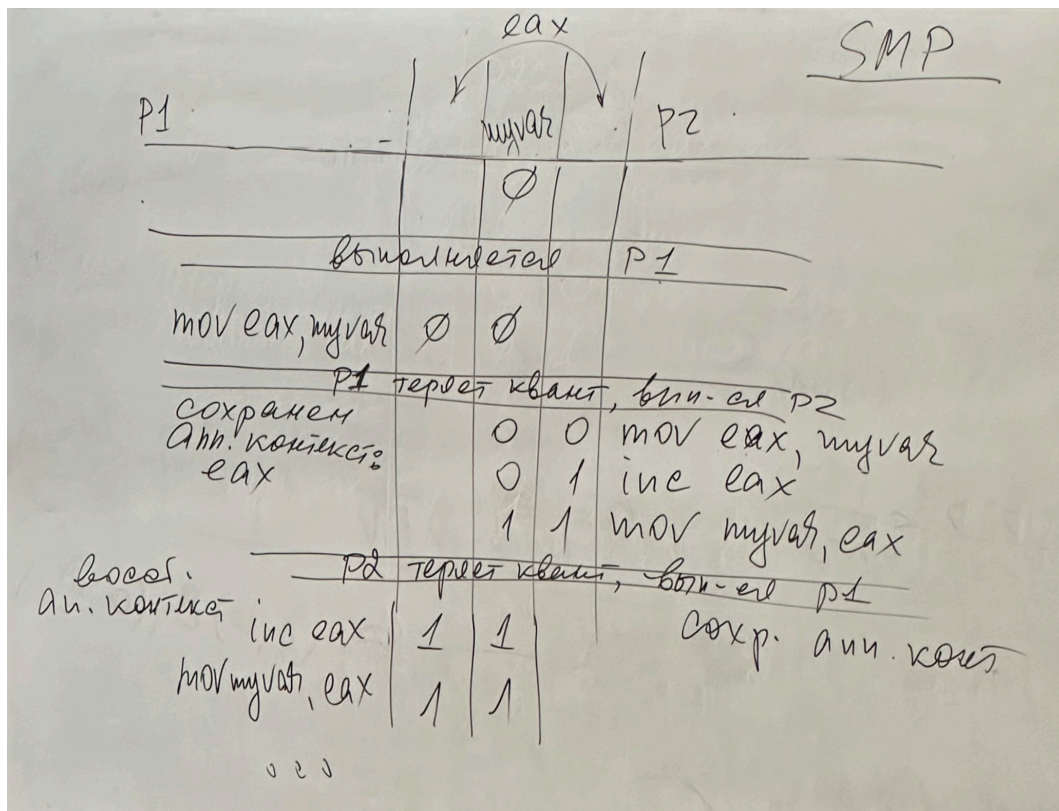


Рис. 14. Диаграмма для однопроцессорных машин

### 8.2.2 Критические секции и разделяемые ресурсы

Критическая секция — `mov eax, muvar; inc eax; mov muvar, eax` («вот этот вот код»)

Переменная `muvar` в данном случае называется разделяемой.

Чтобы не терять данные, работа с разделяемыми переменными или, в общем виде, с разделяемыми ресурсами должна выполняться в режиме монопольного доступа. Это значит, что если процесс зашёл в свой критический участок по разделяемой переменной, другой процесс в этот же критический участок по той же разделяемой переменной войти не должен.

Монопольный доступ обеспечивается методом взаимного исключения, то есть если процесс находится в критическом участке, то другой процесс в этот же участок войти не сможет. Обеспечивается соответствующими средствами взаимного исключения.

### 8.3 Методы взаимного исключения

- 1) Программный способ
- 2) Аппаратный способ
- 3) С помощью семафоров
- 4) С помощью монитора

## 8.4 Программный способ взаимного исключения

### 8.4.1 Первая попытка с флагами

Это попытка обеспечить монопольный доступ процесса к разделяемым переменным чисто программным способом. Решение выполняется с помощью флагов.

```
1 fl1, fl2 : integer
2 P1: while (1)
3   {
4     while(fl2);
5     fl1 = 1;
6     CR1;
7     fl2 = 0;
8     PR1;
9   }
10 P2: while (1)
11   {
12     while(fl1);
13     fl1 = 1;
14     CR2;
15     fl2 = 0;
16     PR2;
17   }
18
19 main:
20   fl1 = 0
21   fl2 = 0
22   pasbegin P1; P;
23   pasend;
24   end;
```

Листинг 2. program №1

Пусть инициативу перехватил второй процесс (процессорное время). Вошел в цикл вайл и обнаружен что флаг 1 процесса сброшен зашел в крит секцию, установил флаг и потерял квант. Процессор 1 получил квант, обнаружил флаг, будет находится в цикле ожидания весь квант. Второй выполнил, сбросил. Все хорошо. Но что если второй зашел в крит секцию, но не успел установить флаг? Первый также обнаружил что флаг второго сброшен, выполнил действие (установил сбросил). P2 уже не проверяет флаг первого. Опять потеряли данные. Подход не всегда работает.

Пусть процессы сразу устанавливают свой флаг до цикла проверки флага.

```
1 fl1, fl2 : integer
2 P1: while (1)
3   {
```



```

4      fl1 = 1;
5      while(fl2);
6          CR1;
7      fl2 = 0;
8      PR1;
9  }
10 P2: while (1)
11 {
12     fl2 = 1;
13     while(fl1);
14     CR2;
15     fl2 = 0;
16     PR2;
17 }
18
19 main:
20     fl1 = 0
21     fl2 = 0
22     pasbegin P1; P;
23     pasend;
24     end;

```

Листинг 3. program №1 после изменений

- Второй процесс установил свой флаг и потерял квант.
- Первый получил квант, установил флаг и начал крутиться в цикле.
- Он потратил квант. Квант получил второй процесс.
- Так как флаг 1 процесса уже установлен, он начнет крутиться в цикле.
- И так по очереди. Процессы попали в тупик.
- Итог: флаги не работают в таком простейшем варианте.

#### 8.4.2 Алгоритм Деккера

Задача решена только для двух параллельных процессов с помощью дополнительной переменной «чья очередь».

```

1 fl1, fl2m: logical;
2 queue : int;
3 P1: while (1)
4 {
5     fl1 = true;
6     while(fl2);
7     {
8         if (queue == 2) then
9         {
10             fl1 = false;

```



```

11      while (queue == 2)
12          fl1 = true;
13      }
14  }
15  CR1;
16  fl1 = false;
17  queue = 2;
18  PR1;
19 P2: while (1)
20     {
21         fl2 = true;
22         while(fl1);
23         {
24             if (queue == 1) then
25                 {
26                     f2 = false;
27                     while (queue == 1)
28                         fl2 = true;
29                 }
30         }
31         CR2;
32         fl2 = false;
33         queue = 1;
34         PR2;

```

Листинг 4.

- Проблема тупика решена за счет введения дополнительной переменной queue и если процесс обнаруживает что установлен флаг другого процесса, он проверяет, чья очередь. Если очередь второго процессора, то он (первый) свой флаг сбрасывает. Сбросил свой флаг он переходит в цикл ожидания по переменной queue. Когда выходит из своей критической секции, сбрасывает свой флаг, и устанавливает что следующим в критическую секцию войдет другой процесс.
- За счет того, что введена переменная чья очередь и за счет проверки этой переменной, процесс имеет возможность сбросить свой флаг если очередь другого процесса, но о всё равно входит в очередь ожидания, когда другой процесс, выйдя из своей критической секции, передаст инициативу другому процессу.

Но решена только для 2 процессов, ничего возня с флагами ничего не дала.

Мы увидели, что такое решение приводит к проверке значений переменных в цикле, то есть процессорное время тратится непроизводительно на проверку флагов. Это называется **активным ожиданием на процессоре, неэффективная трата процессорного времени**.

Есть и другое решение. «Деккер решает задачу достаточно сложным способом».

### 8.4.3 Алгоритм Bakery

Решение основывается на очереди.

## 8.5 Аппаратный способ взаимного исключения

### 8.5.1 Команда test\_and\_set

Начиная с машины IBM370 (с операционной системы, с системы команд), во всех архитектурах присутствует аппаратная команда `test_and_set(a, b);`.

- Это неделимая команда, её выполнение прервать нельзя.
- Она как правило (в архитектурах различных могут быть нюансы) читает значение логической переменной `a` и устанавливает `b = истина`.

```
1 P1: while (1)
2     {
3         C1 = 1;
4         while (C1)
5             test_and_set(C1, fl);
6         CR1;
7         fl = 0;
8         PR1;
9     }
10 P2: while (1)
11     {
12         C2 = 1;
13         while (C2)
14             test_and_set(C2, fl);
15         CR2;
16         fl = 0;
17         PR2;
18     }
19 fl = 0;
20 pasbegin
21 P1, P2'
22 pasend
```

- Логическая переменная `fl` имеет значение «истина», когда любой из процессов находится в своём критическом участке, иначе ложь.
- Рассмотрим когда P1 хочет войти в свой критический участок, а P2 уже находится в своем крит. участке
  - 1) P1 устанавливает в C1 единицу и входит в цикл проверки переменной `fl` командой `test_and_set`
  - 2) Так как P2 находится в критическом участке, то значение `fl = 1`
  - 3) Команда `test_and_set` обнаруживает этот факт и устанавливает в C1 единицу

- 4) Таким образом P1 находится в цикле активного ожидания, пока P2 не выйдет из своего критического участка.
  - 5) Когда процесс выходит из критического участка и устанавливает в `fl = false` то скорее всего другой процесс сможет перехватить инициативу, установив в `fl true`
- Данный способ практически исключает захват критической секции одним и тем же процессом.

`test_and_set` лежит в основе `spin_lock` и семафоров.

### 8.5.2 Spin lock

`test_and_set` есть в макрокоманде `spin_lock`, которые активно используются в ядрах ОС, в ядрах Linux и Windows.

`test_and_set` используются для захвата в ядре критических секций, некоторые части кода ядра не могут блокироваться, на них можно устанавливать только спинлоки, это активное ожидание на процессоре. (ЭТО ПРИМЕЧАНИЕ ДЛЯ ВЕРСИИ ПРОСТО C `while (test_and_set(*c) != 0)`)

Означает, что процессорное время используется неэффективно, плохо для системы.

Вот такой вариант лучше:

```
1 void spin_lock(spin_lock_t *c)
2 {
3     while (test_and_set(*c) != 0)
4         /* ресурс занят */
5         while (*c != 0);
6 }
7
8 void spin_unlock(spin_lock_t *c)
9 {
10     *c = 0;
11 }
```

Листинг 6.

- `test_and_set` приводит к занятию шины одним процессом или потоком и, следовательно, к снижению отзывчивости системы.
- Если ресурс занят, то дальше выполняется простая проверка `c`, и её можно прервать. То есть захват на длительное время локальной шины памяти не выполняется.

## 9 Семафоры Дейкстры

### 9.1 Определение семафора

Dijkstra EW.

Работа, связанная с семафорами и взаимоисключением была опубликована в 1965 году. Уже в 1965 году решение проблем взаимодействия параллельных процессов назрело, стало очень важным.

#### Определение 9.1 (Семафор)

Семафор — это неотрицательная защищённая переменная, на которой определены 2 операции:

- 1)  $P$  — Первая буква нидерландского слова «Proberen» (пытаться) — декремент, захват семафора. Можно выполнить только если  $S > 0$ . Иначе процесс блокируется в ожидании освобождения семафора.

$$S = S - 1 \quad S > 0$$

- 2)  $V$  — Первая буква нидерландского слова «Verhogen» (поднимать выше) — инкремент. Разблокирует процесс, ожидающий на семафоре.

$$S = S + 1$$

$S$  — Semaphore.



Семафоры бывают:

- 1) Бинарные (0 и 1), определены только две неделимые операции ( $P$  и  $V$ )
- 2) Считающие (целые положительные значения и 0)

### 9.2 Реализация семафоров

В своих работах Дейкстра обозначил именно этими буквами операции на семафоре.

Изменять семафор можно только двумя неделимыми операциями.

Дейкстра определил это, чтобы уйти от активного ожидания на процессоре, чтобы не тратить процессорное время на проверку флагов. Но за всё надо платить. От активного ожидания удалось избавиться, но за счёт того, что операции  $P$  и  $S$  являются системными вызовами и переводят систему в режим ядра. Только ядро может заблокировать и разблокировать процесс. Переход в режим ядра несёт расходы, связанные с переключением программного контекста.

Изменение положения нескольких семафоров при этом современные приложения как правило многопоточные, но даже в те времена выполнялось распараллеливание действий. Изменение в таких приложениях набора нескольких семафоров между собой не связанных могло привести к тупику.

## 9.3 Взаимоисключение с помощью семафоров

|    |              |
|----|--------------|
| 1  | S: Semaphore |
| 2  | P1:          |
| 3  | P(S)         |
| 4  | CR1          |
| 5  | V(S)         |
| 6  | PR1          |
| 7  | P2:          |
| 8  | P(S)         |
| 9  | CR2          |
| 10 | V(S)         |
| 11 | PR2          |

Листинг 7. Взаимоисключение с помощью семафоров

P2 получил инициативу, выполнил декремент, P1 получив инициативу, будет блокирован, пока P2 не инкрементирует S. Это взаимодействие с использованием бинарных семафоров.

|    |                   |
|----|-------------------|
| 1  | S1, S2: Semaphore |
| 2  | P1:               |
| 3  | P(S1)             |
| 4  | P(S2)             |
| 5  | CR1               |
| 6  | P(S2)             |
| 7  | P(S1)             |
| 8  | PR1               |
| 9  | P2:               |
| 10 | P(S2)             |
| 11 | P(S1)             |
| 12 | CR2               |
| 13 | P(S1)             |
| 14 | P(S2)             |
| 15 | PR2               |

Листинг 8. Пример взаимной блокировки процессов (deadlock)

Процесс изменяет несколько семафоров. 2 семафора. В обоих семафорах нуждаются оба процесса. Каждый из процессов захватил семафор, но для продолжения работы нужно захватить ещё один семафор, который занят другим процессом. Никто свой семафор освободить не может, это тупик.

В современных системах определены **наборы** считающих семафоров. Семантически наборы определены в ОС как массивы.

## 9.4 Задачи синхронизации

### 9.4.1 Задача производства — потребления (Producer — Consumer)

Есть 2 типа процессов:

- 1) Процессы-производители, которые производят единицу каких-то данных и помещают их в ячейки буфера / хранилища, т.е. производители могут только производить и помещать произведенные данные в буфер.
- 2) Процессы-потребители, которые могут только выбирать данные из буфера. В результате ячейка освобождается, и второй раз из этой же ячейки ничего взять невозможно.

Решение задачи предложил Дейкстра. Его решение основывается на 3 семафорах: двух считающих и одном бинарном.

- Считающие семафоры: `Sempty`, `Sfull`
- Бинарные семафор: `Sbinary`

```
1 // Начальные значения:
2 //   Sempty  = N
3 //   Sfull   = 0
4 //   Sbinary = 1
5 producer:
6   P(Sempty)
7   P(Sbinary)
8   N = N + 1
9   V(Sbinary)
10  V(Sfull)
11
12 consumer:
13  P(Sfull)
14  P(Sbinary)
15  N = N - 1
16  V(Sbinary)
17  V(Sempty)
```

- 1) Производитель пытается декрементировать семафор `empty` (проверка наличия свободных ячеек). Если ему это удалось, он пытается декрементировать (захватить) бинарный семафор для монопольного доступа к буферу, увеличивает количество занятых ячеек буфера, поскольку ему удалось положить что-то в ячейку буфера, после этого освобождает бинарный семафор и инкрементирует семафор `sfull`.
- 2) Потребитель пытается декрементировать `sfull` (проверка наличия заполненных ячеек). Если удалось, пытается захватить бинарный семафор, если удалось, освобождает ячейку, освобождает бинарный семафор и инкрементирует `empty`.

Producer будет блокирован на `sempty`, если нет свободных ячеек в буфере. Он не может ни в одну ячейку не может положить произведённые данные. Он будет блокирован, пока Consumer не освободит хотя бы одну ячейку буфера. Если Producer удалось положить данные в ячейку буфера, он декрементирует количество свободных ячеек. Когда он выходит из своей критической секции, он увеличивает количество заполненных ячеек буфера.

Бинарный семафор обеспечивает монопольный доступ к буферу.

Consumer наоборот будет блокирован, если нет заполненных ячеек в буфере. Если есть хотя бы одна заполненная ячейка, Consumer декрементирует семафор `sfull`, захватывает семафор, декрементирует количество заполненных ячеек, освобождает семафор и инкрементирует `Sempty` количество свободных ячеек буфера.

Смысл в том что процессы являются асинхронными, то есть процессы выполняются с собственной скоростью. Предсказать, когда процесс придёт в свою определённую точку невозможно. Поэтому возможна ситуация когда потребители быстрее освобождают ячейку буфера, чем производители их заполняют, то есть возможны ситуация когда все ячейки буфера пустые, тогда потребитель будет ждать производителей.

И наоборот, возможна ситуация, когда производители работают быстрее потребителей, заполняют все ячейки буфера, и вынуждены ждать, когда потребители освободят все ячейки буфера. Таким образом происходит синхронизация работы производителей и потребителей.

Несмотря на то, что тут три семафора, тут тупик не возникнет, поскольку такая задача грамотно решённая и она очень показательна именно с точки зрения, что тут используются считающие семафоры.

#### **9.4.2 Задача обедающих философов**

В современных операционных системах реализованы наборы считающих семафоров. Работу наборов продемонстрирует задача обедающих философов, которую также сформулировал Дейкстра

На круглом столе стоят 5 тарелок, между которым лежит по одному прибору. Философ может поесть только если ему удалось захватить сразу два прибора: левый и правый. Иначе он будет «голодать». Он думает некоторое время, потом пытается поесть. То есть пытается захватить приборы, захватив, ест. После этого освобождает приборы, опять какое-то время думает.

Существуют 3 варианта действия философов:



- 1) Каждый философ пытается захватить сразу два прибора: левый и правый. Если ему удаётся, он может начать есть. Поев, он освобождает сразу оба прибора.
- 2) Каждый философ сначала захватывает сначала правый прибор. Если ему это удалось, захватывает левый.
- 3) Каждый философ пытается захватить левый прибор. Если удалось, захватить, начинает есть. Если не удалось — освобождает левый.

Первый вариант может привести к «голодной смерти». Умереть может только 1 философ.

Скажем, сначала начальный момент времени второму и пя удалось захватить оба прибора. Соответственно, первый, третий и четвёртый захватить оба прибора уже не могут. Поев, скажем, второй, освобождает оба прибора. Значит, третий может захватить оба прибора. Второй, первый, четвёртый захватить не мог. Если третий освободит, то, скажем, второй опять может захватить. Четвёртый не может, потому что пятый удерживает. Понятно? Если пятая освобождает, то четвёртый может захватить. Поев, освободит пятый может захватить. Значит, в итоге вот он первый, он никак не может захватить. То второй удерживает, то пятый.

Второй вариант. Каждому удаётся взять правые приборы, левые взять не могут. Всё расхватили. Тупик, блокируют друг друга.

И третий вариант. Взяли правый прибор, не могут взять левый, кладут правый. Опять взяли правый, не могут взять левый, кладут. Захваты, освобождения одних и тех же ресурсов.

Первый вариант - это бесконечное откладывание. Процесс не может захватить необходимые ему ресурсы. Второй вариант - это тупик. И третий вариант - захват и освобождения одних и тех же ресурсов. Работа выполняется системой, но полезные действия при этом отсутствуют. Ничего полезного не делается, всё тратится на захват и освобождение одних и тех же ресурсов.

#### 9.4.3 Решение с помощью наборов семафоров

Все философы действуют, как вы видели, единообразно. Одинаковые действия выполняют. Соответственно, подробно мы запишем действие пятого. Ну, можно по-простому написать.

```
1 chops: array[1..5] of semaphore;  
2 i: int  
3 begin  
4   i := 5  
5   repeat  
6     chops[i] := 1  
7     i := i - 1
```

```

8  until i = 0;
9  cobegin
10     1: var left, right:
11         begin
12             left := 1;
13             right := 5;
14             ...
15         end;
16     ...
17 5:
18     begin
19     var
20         left, right: 1..5
21     begin
22         left := 5;
23         right := 1;
24     repeat
25         // Размышляет
26         P(chops[left], chops[right]);
27         // Ест
28         V(chops[left], chops[right]);
29     forever
30     end; // 5
31 coend;
32 end;

```

Листинг 10. Какой-то алгоритм?

Мы видим, что захват и освобождение выполняется сразу на несколькими семафорами набора. Ну, для пятого философа, значит, для пятого и первого приборов сразу, захват левого и правого прибора.

### Важно

Основная особенность набора семафоров заключается в том, что одной неделимой операцией можно изменить все или часть семафоров набора.

## 9.5 Задача «Булочная» Лампорта (Lamport's Bakery Algorithm)

Решает задачу взаимного исключения для  $N$  параллельных процессов. Идею Лампорт заимствовал из принципа работы булочной. Каждому входящему в дверь булочной клиенту выдаётся листок с номером. Каждому вновь пришедшему клиенту выдаётся листок с номером на единицу больше всех выданных номеров. Продавец обслуживает клиентов в порядке полученных ими номеров.

При этом возможна ситуация когда в дверь булочной одновременно входят 2 клиента. Они должны получить один и тот же номер, но как их обслуживать, в каком порядке? Например, в соответствии с номером паспорта: у кого номер меньше, тот будет обслужен раньше.

```
1 var choosing: shared array[0..n-1] of boolean; // Массив флагов
2   number:   shared array[0..n-1] of integer; ...
3 repeat
4   choosing[i] := true;
5   number[i] := max(number[0], number[1], ..., number[n-1]) + 1;
6   choosing[i] := false;
7   for j := 0 to n-1 do begin
8     while choosing[j] do /* nothing */;
9     while number[j] <> 0 and
10       (number[j], j) < number[i], i) do
11       /* nothing */
12   end;
13 /* critical section */
14 number[i] := 0;
15 /* remainder section */
16 until false;
```

Листинг 11. ALGOL-подобный псевдокод

Под обслуживанием понимается код критической секции. Первыми обслуживаются клиенты с наименьшими номерами. обслуживание в порядке очереди. Расширение – есть информация о том, что текущий клиент уже получает обслуживаемое.

{ Причём здесь даже невозможно записать, сформализовать, что одновременно вошедшие в дверь клиенты получают один и тот же номер. По этой записи Макс + 1 они должны получить разные номера. Ну, ещё раз повторяю, Лампорт сформулировал, так как я продиктовала, формулировка Лампорта. Это как бы первая часть.}

- (Строки 4–6): Сначала входящий клиент получает максимальный номер из уже выданных (между true и false критический ресурс, разделяемый массив)
- (Строки 7–12): Затем он обслуживается в соответствии с полученным номером. При этом клиент будет ждать, пока клиенты с меньшими номерами не будут обслужены.
- (Строка 10): Был выбран процесс, который хочет войти в критическую секцию. Процесс  $P_i$  будет ждать пока имеются процессы с меньшими номерами. Если 2 процесса имеют один и тот же номер, то первым будет обслужен процесс с меньшим именем (с меньшим номером паспорта или каким-то ещё номером). Это отношение называется лексикографическим порядком (см. ниже).
- (Строка 8): Указывает на то, что если процесс выбирает номер, когда  $P_i$  пытается так же его найти,  $P_i$  ждёт до тех пор, пока предыдущий процесс не выберет номер.

- (Строка 14): Показывает, что  $P_i$  больше не заинтересован во вхождении в критическую секцию.

Если процесс не пытается войти в свой критический участок, то их номера равны нулю.

Здесь нет таких средств взаимoisключения, как семафоры. Это взаимoisключение, которое обеспечивается порядком обслуживания. Если взять расширение, которое мы видим в работе аптеки , когда клиенты обслуживаются в порядке очереди, но разными провизорами, там необходимо будет использовать средства взаимoisключения, несмотря на то, что они обслуживаются в порядке очереди. Если провизор один, то уже ничего не нужно.

### Определение 9.2 (Лексикографический порядок)

Запись  $(a, b) < (c, d)$  называется лексикографическим порядком.

Она истинна если  $a < c$  или  $a = c$ , но  $b < d$



## 9.6 Современные примитивы синхронизации

|        |        |          |                                                                         |
|--------|--------|----------|-------------------------------------------------------------------------|
| lock   | wait() | wait()   | pthread_mutex_lock()<br>pthread_mutex_trylock()<br>pthread_mutex_init() |
| unlock | post() | signal() | pthread_mutex_unlock()<br>pthread_mutex_destroy()                       |

Таблица 1. Чета

```

1 int x; // Разделяемая переменная
2
3 // Статическая инициализация
4 pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;
5
6 // ИЛИ Динамическая инициализация
7 pthread_mutex_init(&mut, NULL);
8
9 pthread_mutex_init();
10 pthread_mutex_lock(&mut);
11 // Действия с x (критическая секция)
12 pthread_mutex_unlock(&mut);

```

Листинг 12. Закодированный

### 9.6.1 Отличия от семафоров

От перечисленных средств семафоры отличаются тем, что у семафора нет хозяина. Мьютекс может освободить только тот процесс, который его захватил; это касается и

других средств взаимного исключения. Семафор же может освободить другой процесс, который, зная идентификатор, может получить дескриптор семафора.

В качестве отдельного средства взаимодействия процессов, рассматриваются сообщения и системные вызовы отправки и приёма сообщений. Сообщение — специальный метод обмена данными, соответственно, используются специальные системные вызовы.

Также, кроме перечисленных ранее способов взаимного исключения (программный, аппаратный, с помощью семафоров, мониторов), существует рандеву.

## 10 Мониторы Хоара

### 10.1 Определение и структура монитора

В 1974 году Хоар и примкнувший Хансен.

Мониторы являются средством обеспечения монопольного доступа более высокого уровня, чем те средства, что мы уже рассмотрели (спин-блокировки (test and set) и семафоры). Для того, чтобы структурировать средства взаимного исключения, были предложены мониторы.

#### Определение 10.1 (Монитор)

Монитор — высокоуровневое средство взаимного исключения<sup>3</sup>. Может быть языковой конструкцией, а может быть реализован средствами ОС. Представляет из себя набор защищенных данных и процедур, причем к данным монитора могут обращаться только процедуры монитора. Ключевое слово — монитор.



#### ✱ Claude

Взаимное исключение (mutual exclusion) решает проблему монопольного доступа. Оно отвечает на вопрос: «Как сделать так, чтобы только один процесс мог работать с разделяемым ресурсом в данный момент?» Это про защиту критической секции. Когда один процесс внутри — другие ждут снаружи.

Синхронизация (synchronization) решает проблему упорядочивания событий во времени. Она отвечает на вопрос: «Как сделать так, чтобы процесс А дождался, пока процесс В выполнит определённое действие?» Это про координацию действий процессов. Например, потребитель должен ждать, пока производитель что-то произведёт.

Теперь про мониторы. Сам по себе монитор как структура обеспечивает только взаимное исключение — только один процесс может находиться внутри монитора в любой момент времени. Это его базовая функция.

---

<sup>3</sup>Это не всегда средство синхронизации (как пишут в интернетах), всегда для взаимного исключения (обеспечения монопольного доступа).

Но когда к монитору добавляются примитивы `wait` и `signal`, работающие с переменными типа `condition`, тогда монитор становится ещё и средством синхронизации. Эти примитивы позволяют процессам ждать наступления определённых условий и сигнализировать другим процессам о том, что условие выполнено. Вот тогда монитор решает обе задачи — и взаимоисключение, и синхронизацию.

Мониторы появились раньше классов, которые так же состоят из данных и функций, которые могут обращаться к этим данным.

На классических мониторах определено 2 примитива:

- 1) `wait`
- 2) `signal`

Эти функции монитора определены на переменной типа условие (`conditional`).

## 10.2 Простой монитор

```
1 resource: monitor
2 var
3   busy: logical
4   x: conditional
5
6 procedure acquire;
7   begin
8     if busy then
9       wait(x);
10    busy := true;
11  end;
12
13 procedure release;
14   begin
15    busy := false;
16    signal(x);
17  end;
```

Листинг 13. Пример использования простого монитора

Чтобы обратиться к монитору, процессы вызывают процедуры (функции) монитора. Процесс, вызвавший функцию монитора, считается процессом, находящимся в мониторе. К монитору может быть организована очередь, таким образом, сам монитор является ресурсом.

В простом мониторе 2 процедуры (захватить (`acquire`) и релиз (`release`)), 2 простых логических переменных. Когда процессу нужен какой-то ресурс, он вызывает процедуру `acquire`

— Если ресурс занят, то он блокирован системным вызовом `wait` на переменной типа «условие».

- Если ресурс свободен, то он меняет значение переменной `busy` на `true`, обозначая тем самым, что ресурс занят.

Когда процесс желает освободить занимаемый ресурс, он вызывает процедуру `release`, освобождает т.е. сбрасывает значение `busy` в `false` и вызывает системный вызов `signal`, который просигнализирует, что ресурс освободился. Действие системного вызова сигнал связано с разблокировкой процесса, который находится в очереди к монитору. `wait` блокирует процесс, `signal` разблокирует.

Монитор называется простым, потому что он обеспечивает монопольное использование одного единственного ресурса многими параллельными процессами.

## 10.3 Монитор «Кольцевой буфер»

### 10.3.1 Постановка задачи

Второй монитор называется «кольцевым буфером». Он обеспечивает уже рассмотренную нами задачу производства-потребления. Для этой задачи характерно наличие двух типов процессов:

- 1) процессов-производителей, которые заполняют ячейки буфера;
- 2) процессов-потребителей, которые выбирают объекты из ячеек буфера.

Кольцевым называется, потому что этот буфер представлен в виде массива, и этот массив можно закольцевать, фактически сделав из него кольцо, и тогда он становится «безразмерным». На кольцевом буфере определены 2 переменные типа «условие»: `BufferFull` и `BufferEmpty`

```
1 resource: mutex;
2 var
3   bcircle: array[0..n-1] of <type>;
4   pos: 0..n; // Текущая позиция
5   j: 0..n-1; // Заполняемая позиция
6   k: 0..n-1; // Освобождаемая позиция
7   BufferFull, BufferEmpty: conditional;
8 procedure producer(data: <type>);
9   begin
10    if pos = n then wait(BufferEmpty);
11    bcircle[j] := data;
12    pos := pos + 1;
13    j := (j + 1) mod n;
14    signal(BufferFull);
15  end;
16 //          | -- "Паскалевское разыменование"
17 procedure consumer(var data: <type>);
18   begin
19    if pos = 0 then wait(BufferFull);
```



```

20    data := bcircle[k];
21    pos := pos - 1;
22    k := (k + 1) mod n;
23    signal(BufferEmpty);
24    end;
25    begin
26        pos := 0;
27        j := 0;
28        k := 0;
29    end.

```

Листинг 14. gg

Обеспечивает зацикливание массива  $j := (j + 1) \bmod n$ .

Процесс который желает положить что-то в хранилище, вызывает процедуру produce, соответственно, если позиция равна n, значит буфер заполнен и процесс перейдет в состояние блокировки на переменной BufferEmpty.

Если буфер не заполнен, то помещает в i позицию данные, увеличивает текущую позицию, выполняется зацикливание, посылается сигнал потребителю, что буфер заполнен. Таким образом, потребитель будет заполнен.

Процесс, желающий взять данные вызывает, процедуру consumer. Если текущая позиция 0, то в буфере нет заполненных ячеек. Соответственно процесс перейдет в состояние блокировки на переменной типа условие BufferFull.

Если в буфере есть, что брать, данные будут выбраны с k позиции, текущая позиция будет декрементирована и послан сигнал BufferEmpty процессу, который может быть заблокирован на полностью заполненном хранилище, процесс в результате будет разблокирован.

Задача реализуется во всех системах массовой продажи билетов. Прежде чем перейти к резервированию билеты, мы сначала указываем дату вылета, какие-то дополнительные условия. В итоге система высвечивает список этих рейсов на указанную дату. В это время мы действуем как читатели. Одновременно с нами эту же информацию может читать большое количество других людей. Читатели читают параллельно, они друг другу не мешают.

Когда мы выбрали нужный рейс, по нажатию кнопки «купить», нас переводит на другой интерфейс, на другой режим работы. Здесь уже предполагается, что мы можем зарезервировать какое-то место, то есть мы будем работать как писатель или редактор, указывая, что мы резервируем место. Тут уже необходимо обеспечить монопольный доступ.

## 10.4 Монитор читатели-писатели

В классическом мониторе Хоара 4 процедуры.



Характерно наличие двух типов процессов:

- 1) Процессы читатели, которые могут только читать данные. При этом читать данные они могут параллельно, они друг другу не мешают.
- 2) Процессы писатели, которые могут изменять данные, и поэтому они должны работать в режиме монопольного доступа.

### Типы процессов

- 1) Читатели могут читать **параллельно**, не мешая друг другу
- 2) Писатели, которые могут изменять данные, поэтому они должны работать в режиме монопольного доступа.

Но если речь идёт о какой-то базе данных, например, о БД системы массовой продажи билетов, то писатель работает только с отдельным полем базы данных. Только отдельное поле обозначается как критический ресурс, не вся база. Если вся БД будет рассматриваться как критический ресурс, никто никогда не сможет позволить себе купить билет.

```
1 resource: monitor;
2 var
3   nr: integer; // Читатели
4   wrt: logical; // Писатели
5   c_read, c_write: conditional;
6 procedure startread();
7 begin
8   if wrt or turn(c_write) then wait(c_read);
9   nr := nr + 1;
10  signal(c_read);
11 end;
12 procedure stopread();
13 begin
14   nr := nr - 1
15   if nr = 0 then signal(c_write);
16 end;
17 procedure startwrite();
18 begin
19   if nr > 0 or wrt then wait(c_write);
20   wrt := true;
21 end;
22 procedure stopwrite();
23 begin
24   wrt := false;
25   if turn(c_read) then signal(c_read)
26   else signal(c_write)
```

```
27 end;  
28 begin  
29   wr := 0;  
30   wrt := false;  
31 end.
```

Читатель может начать читать, если нет активного писателя и очередь писателей (очередь ждущих писателей) пуста. Если есть активный писатель и очередь ждущих писателей не пуста, то читатель переходит в состояние ожидания «можно читать». То есть он ждёт, потому что читать изменяемые данные нельзя. (что вы прочитаете? предыдущее значение или результирующие, неизвестно)

Если это условие не выполняется, то тогда увеличивается число активных читателей и посылается сигнал «можно читать». Это особенность монитора Хаара, так называемая цепная реакция читателей. Каждый читатель, который получил доступ к интересующим его данным, инициализирует работу следующего читателя. Когда читатель заканчивает чтения, он вызывает процедуру `stopRead`, число активных читателей декрементируется. И если нет больше активных читателей, то есть счётчик активных читателей равен нулю, то посылается сигнал, «можно писать». Писатель вызывает `StartWrite`.

Писатель может начать писать, если нет активных читателей и нет активного писателя. То есть, если есть активные читатели и или есть пишущий писатель, то писатель блокирован на переменный типа условия «можно писать». Если это условие не выполнено, то тогда писатель захватывает логическую переменную `wrt` и присваивает ей `true`.

Когда писатель заканчивает писать, он присваивает логической переменной `wrt` `false`, и проверяется, если есть очередь ждущих читателей / ждущие читатели, то посылается сигнал «можно читать», иначе — сигнал «можно писать».

Анализируя этот монитор, Дейтел обращает внимание, что монитор написан таким образом, чтобы исключить бесконечное откладывание читателей и писателей.

в `lr` - счётчик ждущих, активных читателей, ждущих писателей и 1 логическая переменная писатель.

#### 10.4.1 Работа монитора

Само существование функции `turn` предполагает соответствующих действий, то есть когда читатель вызывает `startread`, значит увеличивается число ждущих читателей, то есть процессов, которые хотят прочитать информацию. Если они успешно проходят этот условный оператор, то читатель из ждущего превращается в активного, то есть увеличивается число активных читателей, а ждущих — уменьшается. Когда читатель прочитал, то число активных уменьшается.

С писателями так же (`turn(c_write)`, `turn(c_read)`). Когда вызывает `start_write`, то увеличивает число ждущих писателей. Если писатель успешно прошёл условный оператор, захватил переменную `wrt`, то число ждущих писателей уменьшается. То есть мы работаем с тремя счётчиками и одной логической переменной {бинарной, по нашему, по сишному}.

## 10.5 Задача Подписчик-Издатель

Эта задача была описана не так давно. Посто АНОНСИРУЕТ для ИНТЕРЕСУЮЩИХСЯ

## 10.6 Задача «Проблема спящего парикмахера»

Эта задача была сформулирована Медником и Данованом.

Проблема спящего парикмахера, она подобна, а описанию, которое дано для обедующих философов. Во всех этих задачах рассматриваются получения процессами ресурсов в разных последовательностях.

Парикмахерская. У парикмахера есть рабочее место, то есть кресло, в котором находится клиент. Когда клиент смог занять кресло, парикмахер его обслуживает. Кроме этого рабочего места, есть «прихожая», в которой имеется несколько стульев, где клиенты могут ждать обслуживания. Возможна следующая ситуация: приходит клиент и первое, что он делает — он смотрит, свободно ли кресло, то есть свободен ли парикмахер. Если он свободен, клиент садится в кресло и парикмахер начинает обслуживать. Если занято — клиент переходит в прихожую и если есть свободное место садится ждать. Если свободного места нет, клиент уходит.

Здесь возможны следующие ситуации:

- Клиент пришёл и увидел, что парикмахер занят и отправился занимать стул, для того чтобы ждать когда парикмахер освободится. В это время парикмахер освободился, а когда он освобождается, он идёт в прихожую чтобы пригласить следующего клиента. Он освободился, пошёл в прихожую, то есть параллельно идут парикмахер и клиент.
- Но парикмахер оказался более шустрым, он пришёл и обнаружил, что прихожая пуста, то есть клиент ещё не успел дойти. В результате парикмахер возвращается на рабочее место и спит, а клиент пришёл в прихожую, сел на стул и начал ждать, когда его позовут. В результате ни тот, ни другой ничего не делают и ждут.

Это как раз проблема спящего парикмахера.

### Важно

Речь идёт о захвате и освобождении ресурса и об асинхронных процессах, то есть выполняющихся с собственной скоростью. В этом проблема спящего парикмахера.

### Проблема гонки в задаче спящего парикмахера:

Рассмотрим критическую ситуацию, которая может возникнуть при параллельном выполнении действий клиента и парикмахера.

Клиент заходит в парикмахерскую и видит, что парикмахер занят обслуживанием другого клиента. Клиент решает пройти в прихожую, чтобы занять стул в зале ожидания. Почти одновременно парикмахер заканчивает работу с предыдущим клиентом и направляется в прихожую, чтобы пригласить следующего клиента. Таким образом, парикмахер и новый клиент движутся навстречу друг другу параллельно.

Возникает гонка: кто быстрее? Если парикмахер окажется более шустрым и придёт в прихожую первым, он обнаружит её пустой — клиент ещё не успел прийти. Решив, что клиентов нет, парикмахер возвращается на своё рабочее место и засыпает в ожидании. Через мгновение клиент приходит в прихожую, садится на стул и начинает ждать, когда его позовут.

В результате возникает взаимная блокировка: парикмахер спит, думая что клиентов нет, а клиент сидит и ждёт, когда его позовут. Оба бездействуют, хотя один готов работать, а другой — получить услугу. Это классический пример проблемы синхронизации, требующей использования семафоров или других механизмов для координации действий параллельных процессов.

## 10.7 Охрана (что-то дополнительное, от Дейкстры)

{Это развитие задачи взаимосключения. Это языковая конструкция, то есть она действительно должна каким-то образом поддерживаться.}

$S_i$  – список охраняемых операторов,  $G_i$  – охрана

```

1 <охрана> → <список охраняемых операторов>
2 |— IF  $G_1 \rightarrow S_2$ 
3 |   □  $G_2 \rightarrow S_2$ 
4 |     ...
5 |   □  $G_n \rightarrow S_n$ 
6 |— FI
```

Более современная версия:

```

1 Bash if statement
2 examples (if then fi,
3 if then else fi, if elif
4 else if, nested if)
```

## 10.8 Рандеву (свидание)

Основной единицей языка Ада является задачи (task, задача). Задачи языка Ада связываются между собой при помощи с входов (entry).

```

1 task <имя> is
```

```
2 entry <идентификатор входа> (дискретный диапазон) (формальная часть);  
3 <другие объявления входов или фазы представления>  
4 end <имя>;
```

Когда какой-то процесс готов принять вызов, он выполняет команду ассерт (принять).

```
1 assert <идентификатор входа (выражение) формальная часть> do  
2   <последовательность операторов>  
3 end;
```

То есть, если вызов принят, то будет выполнена последовательность операторов. Причём результат выполнения данной последовательности будет получен одновременно обоими процессами, запросившими, запросившим рандеву и, собственно, принявшим рандеву и выполнившим эту определённую последовательность операторов. Основа рандеву – это **одновременное** получение параллельно выполняющимися процессами, то есть процессом запросившим рандеву, фактически запросившим обслуживание, и процессом, принявшим запрос, то есть обслуживавшим, результата выполнения последовательности операторов.

Для того, чтобы они одновременно могли получать результаты выполнения последовательности операторов, они должны быть синхронизированы. То есть если процесс принял рандеву в результате запроса другого процесса, они на время выполнения этого рандеву синхронизированы. Причём возможна такая ситуация, что процесс, которому нужно рандеву, пришёл в точку запроса этого рандеву раньше, чем процесс, который может принять это рандеву, дошёл до выполнения ассерт. В этом случае процесс запросивший рандеву будет блокирован в ожидании когда другой процесс дойдёт до точки ассерт. Если же другой процесс, в общем-то, уже выполнил ассерт, а другой процесс не запросил рандеву, фактически он будет ждать этого запроса. В этом особенность механизма рандеву.

Косвенно они выделяются в отдельный тип (америкосиками). При этом речь идёт об удалённом взаимодействии.

Процесс, который может принять запрос от другого процесса, может выбирать, поэтому в языке определена команда `select` (могу принять, могу не принять).

В механизме рандеву отсутствует симметрия, то есть одна задача может инициировать возможность установления рандеву, обратившись ко входу, а другая задача может принять или не принять вызов. При этом рандеву происходит только тогда, когда вторая задача принимает вызов.

Рассмотренные вопросы связаны с вызовом удалённых процедур. Этот механизм будем рассматривать при написании программы. При этом мы будем рассматривать первый способ реализации, когда вызывается удалённая же процедура, которая

фактически может выполняться на другом компьютере, то есть это уже реализация сетевого взаимодействия. Но механизм удалённых процедур реализован таким образом, что он закрывает от пользователя особенности сетевого взаимодействия, но взаимодействие всегда выполняется по модели клиент-сервер.

## ✳ Claude

### Что такое рандеву и почему это «свидание»

Рандеву (rendezvous) — это механизм синхронизации, при котором два процесса должны “встретиться” в определённой точке программы, как два человека на свидании. Название отражает суть: оба участника должны прийти в назначенное место и время, иначе встреча не состоится. Это отличается от семафоров или мониторов, где один процесс может оставить “сообщение” (изменить семафор) и уйти дальше. При рандеву оба процесса должны остановиться и дождаться друг друга.

### Ключевая особенность: асимметрия ролей

В рандеву всегда есть два участника с разными ролями. Один процесс является **вызывающим** (клиент) — он обращается к входу (entry) другого процесса, запрашивая рандеву. Второй процесс является **принимающим** (сервер) — он выполняет команду ассерт, соглашаясь на встречу. Это асимметричная схема: клиент не может принять вызов, а сервер не может вызвать сам себя.

Представь это как телефонный звонок: один человек звонит (вызывает), другой снимает трубку (принимает). Разговор (выполнение кода в ассерт) начнётся только когда оба готовы — один позвонил, другой ответил.

### Синхронизация и блокировка

Главное преимущество рандеву — автоматическая синхронизация. Когда рандеву установлено, оба процесса выполняют код внутри блока ассерт синхронно, то есть оба видят промежуточные результаты и оба получают финальный результат одновременно. Это гарантирует согласованность данных.

Но что если процессы приходят в точку рандеву в разное время? Тогда первый пришедший блокируется и ждёт второго. Если клиент вызвал entry, но сервер ещё не дошёл до ассерт — клиент ждёт. Если сервер выполнил ассерт, но никто не вызвал — сервер ждёт. Рандеву произойдёт только когда оба готовы, как два человека, ожидающих друг друга у входа в кафе.

### Команда select: возможность выбора

Команда select даёт серверу гибкость. Сервер может объявить несколько точек входа (entry) и с помощью select выбирать, какой вызов принять первым, или вообще отказаться от приёма, если выполнены определённые условия. Это как если бы человек мог выбирать, на какое из нескольких свиданий пойти, или вообще остаться дома, если погода плохая.

Механизм рандеву используется для удалённого взаимодействия между задачами, которые могут выполняться на разных процессорах или даже на разных машинах. Это делает язык Ада особенно подходящим для распределённых систем и систем реального времени, где требуется строгая координация между параллельными компонентами.

# 10.9 Взаимодействие процессов в распределённых системах (больше чисто про время)

Этот материал не совсем относится к ОС мейнфреймов.

Рассмотрим работу программы GNU Make.

В Unix принято большие программы разделять на несколько файлов. При этом понятно, что внесение изменений в одни файлы не требует внесения изменения в другие файлы. Если какие изменения внесены в один файл, этот файл необходимо перекомпилировать, но другие файлы перекомпилировать нет необходимости.

Программа Make после её запуска проверяет время последней модификации всех исходных файлов и всех объектных файлов программы. Если исходные файлы имеют время последней модификации больше, чем время модификации соответствующего ему объектного файла, то необходима перекомпиляция.

На каждом компьютере есть тактовый генератор (можем мерить время джифисами; джифис, но это не совсем тактовая частота, это как бы специально выделяемые импульсы). При этом это техническое устройство, и даже несмотря на огромную точность генератора, как в привычных нам наручных часах, часы на основе импульсов, сформированных на основе частоты генератора, могут спешить или отставать на какое-то количество импульсов.

Возможна следующая ситуация:

|    |                                |      |      |      |               |
|----|--------------------------------|------|------|------|---------------|
| 1  | <ИМЯ>.С                        |      |      |      |               |
| 2  | <ИМЯ.О>                        |      |      |      |               |
| 3  | -----#-----#-----#-----#-----> |      |      |      |               |
| 4  | 1224                           | 1224 | 1225 | 1226 | Время по лок. |
| 5  |                                |      |      |      | часам 1       |
| 6  | <ИМЯ.С>                        |      |      |      |               |
| 7  | -----#-----#-----#-----#-----> |      |      |      |               |
| 8  | 1222                           | 1223 | 1224 | 1225 |               |
| 9  |                                |      |      |      |               |
| 10 | Например:                      |      |      |      |               |
| 11 | <ИМЯ.С> -- 1224                |      |      |      |               |
| 12 | <ИМЯ.О> -- 1223                |      |      |      |               |
| 13 | -----                          |      |      |      |               |
| 14 | требуется перекомпиляция       |      |      |      |               |
| 15 | <ИМЯ.С> -- 1224                |      |      |      |               |
| 16 | <ИМЯ.О> -- 1225                |      |      |      |               |
| 17 | -----                          |      |      |      |               |
| 18 | не требуется перекомпиляция    |      |      |      |               |

Речь идёт о том, что исходный файл редактируется на одном компьютере. И для того, чтобы позволить программисту работать более эффективно, перекомпилируется на другом компьютере.

Время редактирования по локальным часам меньше времени предыдущего создания объектного файла. То есть файл перекомпилирован не будет.

### 10.9.1 Проблема физического времени

Реальное время – поясное время, по которому живем. Счётчик реального времени, который инкрементируется обработчиком восьмого прерывания, находится в области данных BIOS, то есть в RAM. И когда компьютер выключается информация там не сохраняется. Но когда мы включаем компьютер, то информация из счётчика, который находится в CMOS-микросхеме, копируется вот в этот счётчик реального времени. Так сделано, потому что микросхема является очень низкой энергоёмкой и питается от аккумуляторной батареи. Работаем с копией этих часов в памяти.

В распределённой системе каждый компьютер имеет собственный тактовый генератор, на импульсах которого работают часы реального времени.

Некоторые путают, потому что Unix традиционно имеет два типа часов. Они считали, что важно считать часы, которые отсчитывают время от начала работы системы (это одни часы), вторые – реального времени. Windows таких часов не имеют (наткнувшись надали все время).

$10^{-5}$  - точность генерации тиков. Если генерируется для таймера 60 прерываний в секунду, то в час будет сгенерировано 216.000 так называемых тиков.

Служба точного времени. Это очень важная служба. В каждой (не в каждой стране, только в развитых) страна реально имеется так называемая служба точного времени, которая финансируется государством.

#### Определение 10.2 (Солнечный день)

Интервал между двумя солнечными переходами называется солнечным днём.



Человечество всё время отслеживало эти моменты. Начиная с определённого времени отслеживало достаточно точно. В результате многочисленных исследований была определена средняя солнечная секунда. (min solar srcond(?))

В 1948 году понадобилось точно вычислять время, так как появились баллистические ракеты. Доставка атомных снарядов на дом. Были изобретены атомные часы и была определена атомная секунда.



### Определение 10.3 (Атомная секунда)

Это время за которое атом Цезия-133 ( $^{133}\text{Cs}$ ) совершает ровно 9 192 631 770 переходов.



Эти переходы можно зафиксировать прибором. В результате в мире около 50 лабораторий создали для себя атомные часы на Цезии-133. Было создано международное бюро, которое получая данные из этих лабораторий усредняет их результаты и выдаёт так называемое глобальное время по атомным часам. Это время получило название International Atomic Time.

В настоящее время, которое мы с вами имеем честь жить, соответственно, солнечный день увеличивается, соответственно, изменяется солнечная секунда, а атомная секунда остаётся такой же. Если не предпринять корректирующих действия, то полдень с использованием TAI будет наступать всё раньше и раньше. Для обывателей — всё равно, но для техники это имеет огромное значение. Международное бюро решило эту проблему, введя потерянные секунды (dip seconds). Всякий раз, когда разница между тайм и солнечной секунды возрастает до 800 микросекунд объявляется потерянное время. В результате мир перешёл на универсальное согласованное время (UTC — Coordinated Universal Time).

Заинтересованы:

- 1) Авиакомпании, перевозчики
- 2) Компании, поставляющие электроэнергию

## 10.10 NTP

### Определение 10.4 (NTP)

NTP — Network Time Protocol (протокол сетевого времени) — сетевой протокол для синхронизации внутренних часов компьютера с использованием сетей с переменной латентностью.



Мы все связаны глобальной сетью. Есть возможность синхронизировать свои локальные часы на компьютере с временем по глобальной сети. А когда не было такой глобальной сети, то скажем, у америкосов, у них была радиостанция коротковолновая, которая передавала сигналы точного времени. У нас можно набрать 650.

Существуют алгоритмы, позволяющие корректировать локальные часы в локальной сети. Как правило локальная сеть имеет выход в глобальную сеть, что чревато целым набором сложнейших проблем, но в итоге позволяет всё-таки передавать информацию.

- Алгоритм Кристиана
- Алгоритм Беркли

— Усредняющие алгоритмы.

Для этого необходимо каким-то образом получать сигналы точного времени или по сети, или ...

Отношения «случилось до» или «случилось после» — очень важные и не всегда требуют сигнала точного времени. Кроме тех привычных нам отношений больше/меньше и т.д. существуют отношения «случилось до» и «случилось после». Это так называемая **непротиворечивость данных** и в первую очередь это связано с распределёнными системами.

#### **10.10.1 Timestamp (алгоритм Лампорта отметки времени)**

Самым известным алгоритмом, который позволяет решить данную проблему, является алгоритм Лампорта отметки времени — timestamp. Чаще указывается в нашей литературе как логические часы.

Логические часы основаны на отметках времени, но речь не идёт об отметках физического времени. Смысл предлагаемого алгоритма в точной упорядоченности действий по отношению «случилось до/случилось после», чтобы точно определить какое действие произошло раньше.

Рассмотрим ситуацию, которая возможна в системе, в которой не реализуется алгоритм Лампорта. Скажем, у нас имеется три компьютера. В начальный момент времени у них показания одинаковые. Потом начинается расхождение локальных часов. Естественно, показана такая экстремальная ситуация. Как вы видите, здесь дискретность шесть, здесь восемь, здесь 10.

## Пример системы, где не используется алгоритм

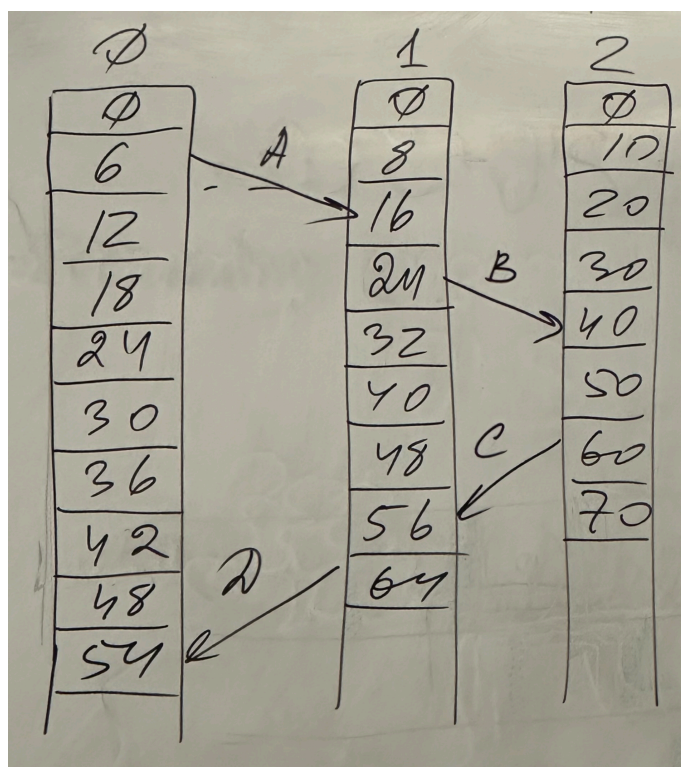


Рис. 15. Без timestamp

В какой-то момент времени с первого хоста присылается сообщение на другой хост, обозначим его как А. Соответственно, получив это сообщение, выполняются какие-то действия, формируется следующее сообщение, обозначим его как В.

В какой-то момент времени с первого хоста вызывается сообщение на другой хост «А». Получив это сообщение выполняются какие-то действия, формируется другое сообщение «Б». Видим, что сообщение с первого хоста было отправлено раньше, то есть было отправлено раньше времени приема. Время отправки сообщения меньше времени приёма с учётом времени передачи этого сообщения. В реальности не может быть так, что время отправки сообщения больше времени получения. Но всё меняется, когда третий хост отправляет а сообщение как бы в обратном порядке. Вот мы видим, что отправка сообщения произошла после приёма, то есть время отправки, больше времени получения, и это невозможно (на картинке). Дальше ещё хуже, мы никак не сможем с вами проверить, то ли сообщение получено, поскольку нет никакой опорной информации.

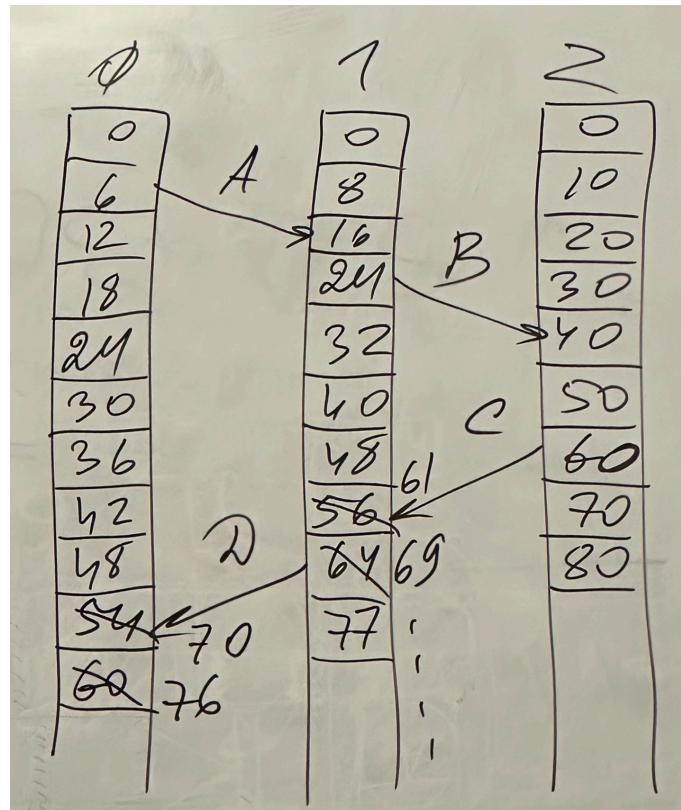


Рис. 16. C timestamp

Лампорт предложил следующее решение, которое не требует привязки к физическому времени, сложных технических устройств, приёма сигналов точного времени или каких-то дополнительных демонов. Он предложил вместе с посылаемыми сообщениями дополнительно отправлять время создания сообщения. Когда такое сообщение приходит на другой компьютер, заинтересованный процесс проверяет (сравнивает) время в пришедшем сообщении со своим временем. Если своё время меньше, чем время, которое пришло в сообщении, то выполняется коррекция и устанавливается на принимающей стороне время на единицу большее, чем время, которое пришло к сообщению.

#### 10.10.2 Алгоритм векторных часов

На каждом локальном компьютере создаётся так называемый вектор событий. И когда происходит событие, а приём сообщения - это событие, редактируется вектор событий и эти вектора позволяют реализовывать правильные отношения, которые называются случилось до, случилось после, что очень важно для правильной оценки ситуации, для анализа данных.

### 10.11 Алгоритмы и подходы взаимодействия процессов в распределенных системах

#### 10.11.1 Централизованный алгоритм

Алгоритмы взаимодействия параллельных процессов в распределенных системах.

Централизованный алгоритм мы дополним алгоритмом забияки.

Алгоритм строится на основе наличия в распределенной системе процесса-координатора. Нужен для того, чтобы регулировать вхождение процессов в их критические секции. Для определённости в качестве процесса-координатора выбирается процесс, который выполняется на компьютере с наибольшим сетевым номером. Процесс, желающий войти в критическую секцию посылает координатору сообщение-запрос, в котором он указывает идентификатор критической секции, в которую хочет войти. Координатор проверяет, не находится ли какой-то другой процесс в указанной критической секции. Если находится, то никакого сообщения-ответа не посылается. Тот процесс, который послал сообщение-запрос на вхождение в критическую секцию, будет находиться в состоянии ожидания ответа, то есть будет блокирован, пока не получит сообщение-разрешение. Таким образом реализуется взаимное исключение.

Можно, например, обратить внимание на то, что у нескольких процессов может возникнуть необходимость практически одновременного входа в одну и ту же критическую секцию. Соответственно, все они посылают сообщение-запрос. К какому процессу послать сообщение-разрешение? Данная ситуация наиболее логично будет решена по логическим часам. То есть в каждом сообщении запросе посылается время по локальным часам. И тот процесс, сообщение-запрос, который был сформирован раньше остальных, получает сообщение-разрешение, сообщение-ответ. Остальные сообщения будут поставлены в очередь данной критической секции. То же самое произойдёт, если критическая секция занята.

Прекрасно до тех пор, пока работает координатор, но мы понимаем, что возможные различные ситуации, и координатор может аварийно завершиться или получить сигнал kill. Чтобы система не потеряла работоспособность, алгоритм дополняется алгоритмом забияки (выбором нового координатора). Если какой-то процесс обнаружил отсутствие координатора (система основана на таймаутах, что, безусловно, не является гарантией, но это единственный инструмент, имеющийся в наличии у процессов).

Какой-то процесс обнаружил отсутствие координатора. Тогда он инициирует выборы координатора, рассылая всем взаимодействующим процессам сообщения-выборы с указанием собственного идентификатора (сетевого какого-то). Процесс, получивший такое сообщение выборы, сравнивает свой номер с номером, которым пришёл в этом сообщении. Если его номер больше, то он инициирует новые выборы. Теперь он рассылает сообщения, выборы и указывает свой идентификатор. В результате такой рассылки будет определён процесс с наибольшим сетевым номером, и он станет координатором. То есть работоспособность будет восстановлена.

Такой подход требует наличия на каждом хосте программного обеспечения обычного процесса, участвующего во взаимодействии с другими процессами, и координатора.

### 10.11.2 Распределённый алгоритм

В этом алгоритме нет координатора. Все процессы равноправны. Но для того, чтобы войти в свою критическую секцию, процесс должен разослать  $n - 1$  сообщение-запрос (то есть  $n$  взаимодействующих процессов). Процесс должен получить разрешение от  $n - 1$  взаимодействующего с ним процесса.

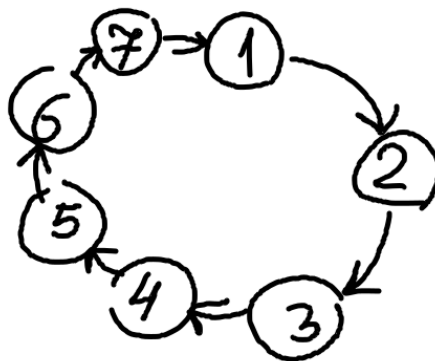
При этом возможно 3 ситуации:

- 1) Первое, когда процесс получатель сообщения запроса не находится и не собирается входить в данную критическую секцию, тогда он высылает сообщение-разрешение;
- 2) Вторая ситуация, когда процесс получатель сообщения запроса сам находится в данной критической секции. В этом случае он не отправляет сообщение-разрешение, а ставит сообщение-запрос в очередь к данной критической секции. Чтобы, когда он выйдет из критической секции, первому процессу, находящимся в такой очереди, отправить своё разрешение;
- 3) И третья ситуация, когда процесс, получивший сообщение-запрос, сам хочет войти в данную критическую секцию, но ещё не успел это сделать. Тогда он сравнивает время в сообщении-запроса со временем в своём сообщении, которое он разослал другим процессам. Если время пришедшем в запросе меньше (его запрос возник позже), то он посылает сообщение-разрешение. Если его запрос возник раньше, то он не посылает сообщение-разрешение и ставит пришедший запрос в очередь.

Другими словами, процесс может войти в соответствующую критическую секцию, получив  $n - 1$  сообщение разрешения.

### 10.11.3 Token Ring

Кольцо токенов. Token — маркер. Взаимодействующие процессы выстраиваются каким-то образом в некоторое логическое кольцо, при этом каждый процесс должен знать своего соседа.



По этому кольцу передаётся токен. Токенов будет столько, сколько имеется критических секций в данной системе взаимодействующих процессов. Если ни один из процессов не заинтересован во вхождении в соответствующую критическую секцию,



то токен с высокой скоростью будет передаваться от процесса к процессу, то есть «крутиться» в этом кольце. Также возможна ситуация, когда процесс, получивший токен, находится в данной критической секции и тогда он не отправляет этот токен дальше, удерживает его до тех пор, пока не выйдет из критической секции. Тогда он не отправляет этот токен дальше, удерживает его до тех пор, пока не выйдет из критической секции. Если он, получив токен, хочет войти в критическую секцию, значит, собственно, все предыдущие процессы в ней не находятся, он входит в эту критическую секцию, удерживая этот токен. Эти алгоритмы сравниваются с точки зрения надёжности выполнения такой системы процессов.

### Примечание

Таким образом, самым надёжным алгоритмом является централизованный алгоритм, но! дополненный алгоритмом забияки.

Процесс в распределённом алгоритме должен получить  $n-1$  разрешение, и он может не получить, если завершился хотя бы один из взаимодействующих процессов. Значит, каким-то образом надо восстановить работоспособность такой системы. {Но рецепт восстановления есть только в централизованном алгоритме.} То же самое с кольцом токенов. Данное логическое кольцо может быть разорвано при завершении любого из процессов в данном кольце. Значит, надо предпринимать действия по восстановлению логического кольца.

## 10.12 Транзакции

Модель транзакции пришла из бизнеса. Рассматривается некоторый переговорный процесс, когда стороны в процессе переговоров меняют условия договора, уточняют эти условия. Но когда договор принят или, как мы говорим, подписан, никакие изменения уже невозможны. Вот компьютерная транзакция построена на этой модели.

Один процесс объявляет, что хочет начать транзакцию с одним или более процессами. Эти процессы могут некоторое время создавать и уничтожает разные объекты, могут выполнять какие-то действия над этими объектами, то есть выполнять редактирование. Затем инициатор транзакции объявляет о желании завершить транзакцию. И если все процессы, участвующие в транзакции с ним соглашаются, то результат транзакции фиксируется.

Если же хотя бы один из процессов отказывается или он потерпел крах, завершился (нет его согласия на завершение транзакции), то все изменённые объекты возвращаются к исходному состоянию. Такое свойство транзакции называется **всё или ничего**. А возвращение объектов к исходному состоянию называется откатом.

Для реализации транзакций, у программистов должны быть соответствующий набор примитивов:

- 1) `begin_transaction` —
- 2) `end_transaction` — граница транзакции
- 3) `abort_transaction`
- 4) `read`
- 5) `write`

`begin` и `end` определяют границы транзакции.

Свойства транзакций определены соответствующей спецификацией:

- 1) Атомарность (`atomic`) — если транзакции и все составляющие её действия выполнены успешно, то транзакция фиксируется (`commit`). Если возникает хотя бы одна ошибка, выполняется откат (`rollback`).
- 2) Согласованность (`consistency`) — транзакция должна переводить данные из одного согласованного состояния в другое согласованное состояние. Например, если товар куплен, то это должно быть зафиксировано в системе складского учёта; В этой системе товар должен быть указан как проданный.
- 3) Изолированность (`isolation`) — транзакция изолирована от других транзакций и процессов. Данные не могут изменяться другой частью системы, которая не участвует в транзакции, можно сказать, что транзакция защищена
- 4) Устойчивость (`durability`, может называться постоянством) — при завершении транзакции все изменения фиксируются и никакой сбой не может привести к потере данных.

Существует два глобальных подхода к реализации транзакций.

Первый подход связан с созданием для всех взаимодействующих процессов локальной среды, в которой будут находиться копии файлов данных, над которыми выполняются действия транзакций. При транзакции все изменения выполняются над копиями. Когда транзакция фиксируется, то все сделанные изменения копируются в оригинал. Это приводит к большим накладным расходам, так как может быть необходимо хранить копии большого количества данных для каждого взаимодействующего процесса. Можно сказать, что в этом подходе каждый процесс имеет индивидуальное рабочее пространство.

Второй подход называется «списком намерений». В этом подходе модифицируются исходные файлы, но перед тем как они будут изменены, в журнал заносится исходное значение данных, вносится информация о транзакции, которая делает эти изменения, и новое значение данных. Только после успешной записи в журнал выполняются изменения исходных данных. Если транзакция фиксируется, то об этом делается запись в журнале регистрации, но записи не удаляются, а сохраняются. Если транзакция обрывается, то информация из журнала регистрации используется для приведения данных в исходное состояние (откат).



Наиболее распространённым протоколом (соглашением) для транзакций (для второго способа) используется двухфазный протокол фиксации транзакций. Рассмотрим вторую фазу.

{Первая фаза завершена: транзакция инициирована, значит, взаимодействующие процессы выполняют какие-то действия}

### 10.12.1 Вторая фаза

Позволяет фиксировать результат транзакций.

Эта фаза также состоит из двух фаз.

#### Первая фаза

Процесс, инициирующий транзакцию, он же и завершает транзакцию.

В конечном итоге координатор решает завершить транзакцию. Соответственно, он в свой журнал записывает «приготовиться» и рассылает сообщение «приготовиться» соответствующим процессам. Процессы, получив такое сообщение, если они готовы совершить транзакцию, в журнал записывают «готов» и только после этого посылаются ответное сообщение «готов». Координатор должен собрать все ответы. Если удалось собрать все ответы, то сначала делается запись в журнал «фиксировать». И только после этого посылаются сообщения взаимодействующим процессам.

#### Вторая фаза

Получив такое сообщение, процесс записывает журнал свой локальный и посылает, условно говоря, сообщение завершить. Опять же он {координатор?} должен собрать ответы. И только после этого транзакция будет завершена. Если не досчитался хотя бы одного сообщения, то транзакция прерывается, соответственно, выполняется отказ.

То есть создать какой-то протокол, это значит определить какую-то последовательность, обязательно для всех действий. А эта последовательность, естественно, выбирается на основе задачи, которую необходимо решить.

## 10.13 Тупики

Мы рассматривали с вами примеры, когда процессы захватывают семафоры в обратном порядке. При этом каждый процесс ждёт освобождения нужного ему семафора, удерживая уже захваченный. Ни один из процессов не может продолжить своё выполнения. При этом в системе они существуют, занимают выделенные им ресурсы, но при этом никакой работы не выполняют. Этот пример можно перефразировать следующим образом. То есть процессы запрашивают, запрашивают ресурсы, полученный ресурс удерживают и запрашивают следующий ресурс, который, как показано в данном примере, занят другим процессом.

1 P1:

1 P2:

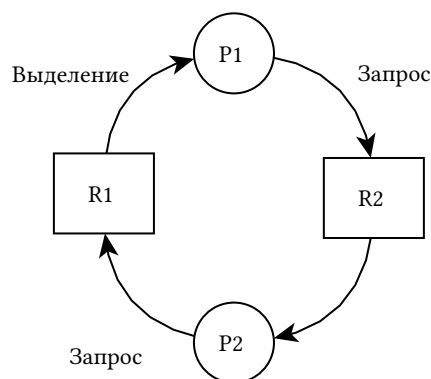
|   |             |
|---|-------------|
| 2 | ...         |
| 3 | Запросил R1 |
| 4 | Получил R1  |
| 5 | ...         |
| 6 | Запросил R2 |

|   |             |
|---|-------------|
| 2 | ...         |
| 3 | Запросил R2 |
| 4 | Получил R2  |
| 5 | ...         |
| 6 | Запросил R1 |

- 1) P1 запрашивает R1 и получает
- 2) P2 запрашивает R2 и получает
- 3) P1 запрашивает R2 и не может получить
- 4) P2 запрашивает R1 и не может получить

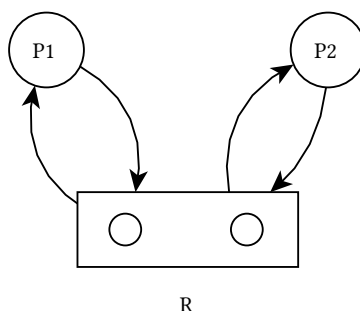
Процессы запрашивают ресурс, задерживают его и запрашивают следующий, который в данном примере занят другим процессом.

Наиболее наглядно тупики демонстрируются с помощью графов. Как правило, процесс обозначаются кружочками, а ресурсы — прямоугольниками.



Стрелка из вершины процесса — запрос ресурса, а стрелка из вершины ресурса к процессу — выделение. Перед нами двудольный (бихроматический) направленный граф — граф Хольта.

Ресурсы 1 и 2 — единичные. Возможна ситуация, когда ресурсы содержат несколько единиц. Тогда этот же граф представляется следующим образом:



Процесс P1 запросил и получил единицу ресурса R. Процесс P2 запросил и получил также единицу ресурса R. Больше нет свободных единиц ресурса R. Процесс P1 для того, чтобы продолжить своё выполнение, запрашивают ещё одну единицу. В

результате они не могут освободить занимаемые им единицы ресурса R и блокируют друг друга, они удерживают единицы ресурса R.

Явно видна **замкнутая цепь запросов**.

рисунок верхний

#### Определение 10.5 (Тупик / тупиковая ситуация)

Тупик или тупиковая ситуация — ситуация, которая возникает в результате монопольного использования разделяемых ресурсов, когда процесс, владея ресурсом, запрашивает другой ресурс, занятый непосредственно или через цепочку запросов другим процессом, ожидающим освобождение ресурса, занятого первым процессом.



Наиболее большую опасность тупики представляют для систем реального времени, для систем, которые предназначены для управления внешними системами, или устройствами, или процессами (внешними по отношению к самой системе). Как правило, это системы, которые управляют какими-то важными объектами, например, атомным реактором.

С точки зрения тупиков ресурсы делятся на два типа:

#### 1) Повторно используемые ресурсы

- 1.1) Аппаратная составляющая компьютера (процессоры, физическая память, устройства ввода/вывода)
- 1.2) Реентерабельные коды ОС
- 1.3) Объекты ядра (семафоры, программные каналы, разделяемая память, очереди сообщений)

Получение процессом такого ресурса никак не влияет на этот ресурс. В результате использования его характеристики не меняются, в систему он возвращается в том же виде, в котором был получен.

Таких ресурсов в системе очень много, причём они разнообразны.

Количество ресурсов ограничено, их количество постоянно для некоторого периода взаимодействия процессов (например, для семафоров).

#### 2) Потребляемые

##### 2.1) Сообщения

В процессе получения эти ресурсы перестают существовать.

Наибольшую проблему представляет в распределённых системах, когда может существовать большое количество процессов, создающих сообщения и принимающих их.

Любой процесс может создать и потребить любое количество сообщений

Существуют системы, которые попадать в тупик не могут {не в плане что никогда не попадут, а в плане что для корректной работы не должны попадать, как я поняла}.

{Есть системы, в которых тупик может возникнуть в комбинации повторно используемых ресурсов и потребляемых, но это отдельные случаи, и, как правило, в таких ситуациях не бывает неограниченного числа сообщений.}

#### **10.13.1 Условия возникновения тупика**

Была предложена стратегия Хейвендера, в которой сначала формулируется 4 условия возникновения тупика:

##### **1) Взаимоисключение (mutual exclusion)**

Процессы монопольно используют предоставляемые им ресурсы. Монопольное использование предполагает взаимоисключение: если один процесс получил в своё распоряжение ресурс, то другой процесс получить этот же ресурс не может, пока первый процесс его не освободит.

##### **2) Ожидание (hold and wait)**

Процессы удерживают занятые ими ресурсы, ожидая получения дополнительных ресурсов, чтобы продолжить своё выполнение.

##### **3) Не перераспределяемость (no preemption)**

Ресурсы нельзя отобрать у процессов до их завершения или освобождения этих ресурсов самим процессом.

##### **4) Круговое ожидание (circular wait)**

То, что видели на графе, это и есть круговое ожидание. Существует цепь процессов, в которой каждый процесс занимает необходимый ему ресурс для дальнейшего выполнения.

Часть из этих условий является необходимыми, часть – достаточными.

#### **10.13.2 Методы борьбы с тупиками**

##### **1) Предотвращение тупиков (исключение тупиков)**

Необходимо создать в системе такие условия, при которых возникновение тупиков невозможно.

##### **2) Обход тупиков (недопущение тупиков)**

Тупики теоретически возможны (для этого есть условия), но система выполняет такие действия, чтобы тупик не возник. Выполняемые действия позволяют обойти тупиковую ситуацию.

У этого метода есть классический способ обхода, который был предложен Дейкстрой — алгоритм банкира (раздел 10.13.4).

### 3) Обнаружение тупиков и восстановление работоспособности системы

Обнаружение всегда требует восстановления работоспособности системы попавшей в тупиковую ситуацию.

#### 10.13.3 Способы исключения тупиков

{В современных системах процессы создаются по мере надобности. Надо выполнить программу, загружаем её на выполнение. А ресурсы определяются по мере необходимости. То есть ваша программа начинает выполняться, ей для выполнения нужны ресурсы, и система эти ресурсы выделяет вашей программе.}

Процессы создаются по мере надобности, ресурсы выделяются по мере необходимости. Всё динамически.

#### Опережающее требование

Оно предполагает, что процесс указывает в заявке все необходимые ему в процессе выполнения ресурсы и не сможет начать выполняться, пока система все эти ресурсы ему не предоставит.

Здесь исключается условие «hold and wait». Процессу нечего ждать, он всё получает сразу.

#### Недостатки:

- 1) Процессы должны заранее знать свою необходимость в ресурсах
- 2) Ресурсы выделяются процессу задолго до их реального использования

Нельзя исключать, что имеется полностью детерминированный процесс, в котором всё это известно. Процесс указывает максимальное количество ресурсов каждого типа.

Такая ситуация возможна только в специальных системах (специального значения). В такой системе можно заранее подобрать необходимое количество ресурсов каждого типа, чтобы не возникало постоянного откладывания. В системах же общего назначения ни один процесс такой информации не имеет, и возможности получить ресурсы заранее нет.

#### Иерархическое распределение

Реализуется путём упорядочивания ресурсов. Ресурсы делятся на классы, и каждому классу присваивается номер. Процессы могут запрашивать ресурсы только из классов с большим номером, чем те ресурсы, которые они уже удерживают.

Здесь устраняется условие «круговое ожидание».

При этом номера классов должны ожидать наиболее вероятный порядок использования ресурсов процессами. Как правило самые дефицитные ресурсы помещаются

в классы с наибольшими номерами. Но какие-то процессы могут запрашивать или хотели бы запрашивать в обратном порядке. Тогда, если процесс запросил ресурс с большим номером, а потом ему требуется ресурс с меньшим номером, тогда ему предлагается освобождение ресурса с большим номером и запрос ресурсов в правильном порядке.

Предельным случаем такой ситуации будет, когда процесс всё время запрашивает ресурсы с меньшими номерами. Всё время будет освобождать уже полученные ресурсы и в итоге он получит сразу все ресурсы. В итоге он получит сразу все ресурсы. Получится первый вариант. Перед этим он будет долго запрашивать и освобождать ресурсы.

Построить классификацию ресурсов по номерам — сложная задача и далеко не однозначная.

Здесь устраняется неперераспределяемость

Если процесс, сделав запрос на нужный ему ресурс, не может его сразу получить, то он освобождает занимаемые им ресурсы (все или какую-то часть). Затем через какое-то время он опять начинает с того, что пытается получить нужные ему ресурсы. Это приводит к дополнительным накладным расходам (запросы, освобождение ресурсов). Любое освобождение ресурса связано с откатом процесса в точку до запроса освобождаемого ресурса.

#### **10.13.4 Алгоритм банкира**

Дейкстра усмотрел в этом методе действия, схожие с действием банкира. Банкир имеет какой-то капитал. К нему обращаются заёмщики, а банкир, выдав им сумму, рассчитывает на возвращение выданных денег. Часто заёмщик, чтобы вернуть взятые в долг деньги, нуждается в дополнительной сумме. Банкир должен проанализировать, кому из заёмщиков он может выделить дополнительные деньги, учитывая возможность возвращения выделенных сумм.

В качестве банкира выступает менеджер ресурсов.

Заёмщики — процессы, которые делают заявки на ресурсы. Чтобы менеджер мог проанализировать ситуацию, необходимо, чтобы процессы в заявках указывали свою максимальную потребность в каждом из ресурсов (максимальное количество единиц каждого ресурса). При этом процесс не может запросить единиц ресурса больше, чем указано в его заявке.

Анализируя возникающую в системе ситуацию, менеджер ресурсов на основе заявок процессов может принять решения, какому процессу выделять ресурсы. В такой системе число процессов фиксировано, число ресурсов фиксировано и это позволит менеджеру ресурсов принять решение.

| Процессы | Текущее распределение | Свободные единицы ресурса | Заявка |
|----------|-----------------------|---------------------------|--------|
| P1       | 4                     |                           | 12     |
| P2       | 2                     |                           | 4      |
| P3       | 4                     |                           | 8      |
|          | $\sum 10$             | 2                         |        |

Анализируя текущую ситуацию в системе, можно сделать вывод, что процесс P2 может гарантировано завершиться, так как он уже получил 2 единицы ресурса, его максимальная потребность составляет 4 единицы, то есть он может запросить только 2 единицы, и у системы эти единицы имеются.

Завершившись, процесс P2 вернёт системе занимаемые им ресурсы, и в сумме со свободными единицами система сможет удовлетворить максимальную потребность в ресурсах процесса P3, который также завершится и освободит занимаемые им ресурсы, которые в сумме со свободными единицами позволят системе удовлетворить максимальную потребность в единицах ресурса процесса P1.

В результате мы имеем последовательность процессов, которая может успешно завершиться. Такое состояние в системе называется **безопасным относительно тупика**, то есть существует последовательность процессов, каждый из которых может успешно завершиться, освобождая занимаемые им ресурсы, что позволит удовлетворить потребность других процессов в ресурсах.

| Процессы | Текущее распределение | Свободные единицы ресурса | Заявка |
|----------|-----------------------|---------------------------|--------|
| P1       | 5                     |                           | 12     |
| P2       | 2                     |                           | 4      |
| P3       | 4                     |                           | 8      |
|          | $\sum 10$             | 1                         |        |

Допустим, в системе возникла ситуация, когда процесс P1 получил 5 единиц и у системы осталась одна свободная единица ресурсов. Это состояние является **небезопасным**. Мы не можем найти последовательность процессов, которые гарантированно будут завершаться. Однако данное небезопасное состояние необязательно приведёт к тупику, так как процессы могут не запросить ранее заявленное максимальное количество ресурсов.

С другой стороны, безопасное относительно тупика необязательно приведёт так же при выделении ресурса к следующему безопасному состоянию. Если состояние



системы безопасно относительно тупика, то следующее состояние может быть небезопасным.

### **Алгоритм Дейкстры (сам непосредственно)**

{Формализация алгоритма} Формально, состояние является безопасным, если существует последовательность процессов такая, что:

- 1) Первый (не по номеру) процесс в найденной последовательности обязательно завершится, так как даже если он запросит максимальное заявленное количество единиц ресурса, у системы имеется необходимое количество свободных единиц для удовлетворения запроса.
- 2) Второй процесс в последовательности может завершиться, если завершился первый и вернул системе все занятые им единицы ресурса, что в сумме со свободными единицами позволит удовлетворить максимальную потребность в единицах ресурса второго процесса в последовательности. И так далее...
- 3)  $i$ -й процесс в последовательности может завершиться, если все предыдущие  $i - 1$  процессы успешно завершились и сумма освобождённых единиц ресурса сможет удовлетворить максимальную потребность  $i$ -го процесса в последовательности.

Таким образом, всякий раз, когда процесс делает новый запрос, менеджер ресурсов должен найти последовательность процессов, которая может завершиться, то есть где каждый процесс может успешно завершиться. Только в этом случае запрос процесса может быть удовлетворён. В результате проведения такого анализа можно гарантировать, что система не попадёт в тупик. Но для этого необходимо исследовать  $N!$  последовательностей.

В связи с этим алгоритм банкира имеет теоретический, а не практический интерес. Он показывает, что действительно можно обойти тупик, но затраты колоссальные.

Самый популярный — алгоритм банкира. Самый известный аппроксимирующий его алгоритм — алгоритм Хабермана.

### **10.13.5 Обнаружение тупиков**

Для обнаружения тупиков используется графовая модель Холта (когда когда начали рассматривать эту тему). Это бихроматический или двудольный направленный граф, в котором имеется два непересекающихся множества вершин. Множество вершин делится на два непересекающихся подмножества: множество процессов и множество ресурсов. Граф направленный, поскольку имеет значение направления дуги. Если дуга выходит из вершины-процесс в вершину-ресурс, это запрос. Если дуга выходит из вершины-ресурс в вершину-процесс - это выделение. Ни одна дуга не соединяет вершины одного и того же подмножества.

Рассмотрим классический пример такого двудольного графа.

Пусть у нас есть ресурс  $R$  и у этого ресурса три единицы. И скажем, два процесса  $P2$ ,  $P1$ . При этом процесс  $P1$  запросил и получил одну единицу ресурса  $R1$  и запрашивает

ещё одну единицу. Процесс P2 запросил и получил одну единицу ресурса R. Значит, вот это приобретение или получение - это запрос. При этом мы видим на этом простейшем графе, что запрос процесса P1 может быть удовлетворён, потому что у системы есть свободная единица ресурса R. То есть нет никакой проблемы. Но так бывает не всегда. Количество ресурсов всегда ограничено. Оно меньше числа процессов, процессы конкурируют в системе за ресурсы и вот могут попадать в такие ситуации, когда ни один из взаимодействующих процессов не может продолжить своё выполнение. Для обнаружения тупиковой ситуации используется метод редукции графа.

Рис. 21. граф

Метод редукции (сокращения) графа. Этот метод позволяет обнаружить тупиковую ситуацию.

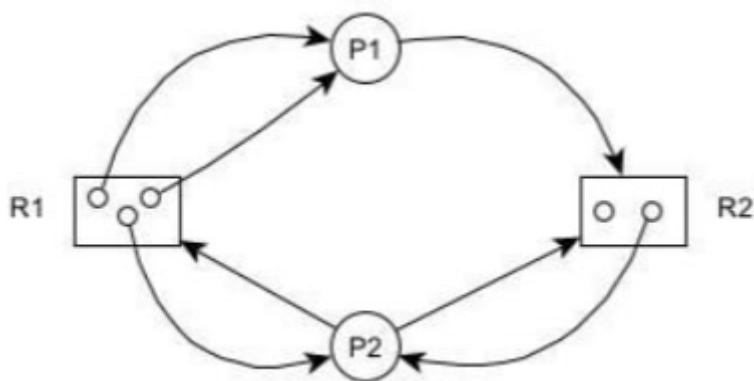


Рис. 22. Новый граф

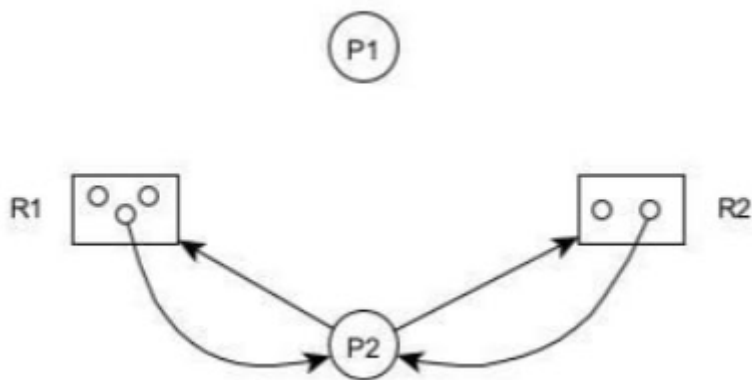


Рис. 23. Шаг 3



Рис. 24. Шаг 3

Ресурс R1 имеет три единицы, ресурс R2 – три единицы. При этом процесс P1 приобрёл две единицы ресурса R1 и запрашивает единицу ресурса R2. А процесс P2 приобрёл единицу ресурса R1, запрашивает ещё одну единицу ресурса R1 и приобрёл единицу ресурса R2 и запрашивает ещё одну единицу ресурса R2. У системы есть одна свободная единица ресурса R2. Сокращение графа в данном случае мы проведём по вершине P1. Значит, процесс P1 запрашивает единицу ресурса R2, и она есть у системы, она свободна. Следовательно, эта единица может быть выделена процессу P1. В результате процесс P1 может завершиться. Это в вот этой последовательности редуцирования графа выглядит следующим образом. То есть процесс P1 получив эту свободную единицу ресурса R2 сможет завершиться и освободить занимаемые им единицы ресурса R1. То есть в результате вершина P1 станет изолированной. По отношению к ресурсу P2 мы оставляем ту же ситуацию. То есть P2 закрашивает а единицу ресурса R1, владея одной единицей ресурса R1. И также, владея одной единицей ресурса R2, запрашивает ещё одну единицу ресурса R2. Но у системы после того, как завершился процесс P1, есть эти единицы свободные, значит процесс P2 сможет их получить и успешно завершиться. В результате и процесс P2 станет изолированной вершиной. В результате получаем полностью сокращаемый граф. И это говорит о том, что система не находилась в тупиковой ситуации.

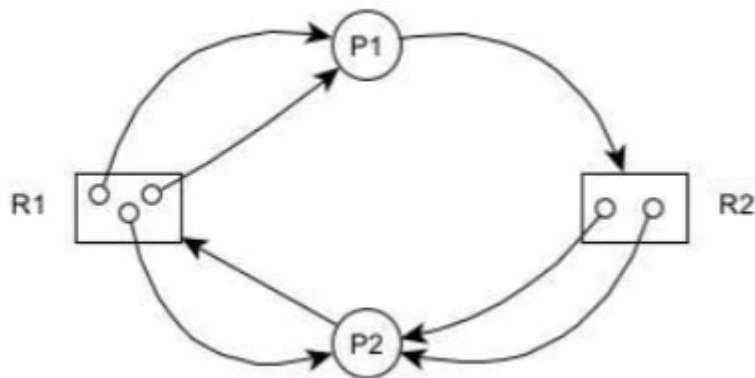


Рис. 25. Новый граф 2

Значит, мы видим, что процесс P2 приобрёл две единицы ресурса R2, но ему ещё нужна одна единица ресурса R1 и свободных единиц у системы нет. А процессу P1 нужна единица ресурса R2 и свободных единиц у системы нет. Видим, что возникла

замкнутая цепь запросов. Собственно, цикл, который возник в результате именно запросов. Круговое ожидание.

#### Теорема 10.6 (Теорема 1, о полностью сокращаемых графах)

Граф является полностью сокращаемым, если существует такая последовательность сокращений, которая устраняет все дуги. Если граф нельзя полностью сократить, то анализируемое состояние является тупиковым. (Состояние графа  $S$  является состоянием тупика тогда и только тогда, когда граф повторно используемых ресурсов в состоянии  $S$  не является полностью сокращаем.)

#### Теорема 10.7 (Теорема 2, о состоянии графа)

Цикл в графе повторно используемых ресурсов является необходимым условием тупика.

Тупик возникает в результате запроса процесса на ресурс, который не может быть немедленно удовлетворён, потому что у системы нет необходимого количества свободных единиц запрашиваемого ресурса.

#### Теорема 10.8

Если состояние системы  $S$  не является состоянием тупика, а состояние  $T$ , в которое перешла система из состояния  $S$ , является тупиковым состоянием, то операция, которая перевела систему из состояния  $S$  в состояние  $T$  является запросом.

Другими словами, тупик может возникнуть только в результате запроса процесса на ресурс. Если такой запрос удовлетворить нельзя, то, значит, нельзя сократить граф, то есть нельзя эту дугу удалить. Но в результате вот таких запросов всегда возникает замкнутая цепь запроса.

Чтобы программно исследовать состояние системы необходимо представлять граф. И наиболее очевидной формой представления графа является матричная форма. При этом необходимо иметь обязательно две матрицы: матрицу текущего распределения и матрицу запросов.

Матрица распределения. Каждый элемент матрицы определяет, сколько единиц ресурса выделено процессу  $P$ .

$$\begin{pmatrix} a_{11} & & a_{1m} \\ & a_{ij} & \\ a_{n1} & & a_{nm} \end{pmatrix}$$

Рис. 26. Матрица распределения ресурсов

$$\begin{pmatrix} b_{11} & & \\ & b_{ij} & \\ & & b_{nm} \end{pmatrix}$$

Рис. 27. Матрица запросов ресурсов

Если реализовывать процесс редукции графа с использованием этих двух матриц, это будет так называемый прямой алгоритм обнаружения.

В результате анализа нужно определить, может ли быть удовлетворён запрос. Для этого нужно иметь дополнительный вектор свободных единиц ресурсов. В системе это количество повторно используемых ресурсов определено, оно не может меняться динамически, поэтому нужно иметь вектор. И, соответственно, сравнивать с этим вектором, запросы процессов и на основе этой информации выполнять сокращение графа.

$$\begin{pmatrix} v_1 \\ \vdots \\ v_m \end{pmatrix}$$

Рис. 28. Вектор свободных единиц ресурсов

Метод обхода тупиков на основе аппроксимации алгоритма банкира. Разница заключается в том, что для реализации алгоритма Хабермана необходимо иметь информацию о заявках процесса на каждый тип ресурса. Тогда возможно выполнение алгоритма Хабермана. Если такой информации у системы нет, это будет реализация метода обнаружения тупиков.

#### 10.13.6 Алгоритм Хабермана

Рассмотрим алгоритм Хабермана. И в том, и в другом случае речь идёт о матрицах. Только на основе представления системы в виде матриц, можно выполнить определённые действия. В данном случае алгоритм выполняется вручную. Его можно реализовать программно, поскольку формализованы все необходимые действия.

Выполнение необходимых действий начинается с заданного состояния системы. При этом имитируется выполнение каждого процесса в плане запросов процессов на необходимое ему количество ресурсов. При этом процесс запрашивает максимальное количество ресурсов, которое ему необходимо. Если в результате анализа все процессы успешно завершаются, то состояние системы называется безопасным. Анализируется  $N$  процессов и  $M$  типов ресурсов. Для решения задачи необходимо иметь матрицу свободных (доступных) ресурсов.

Такая матрица называется `avail_matrix`. Обозначим её как  $A = (a_1, a_2, \dots, a_n)$ . В этом векторе содержатся свободные единицы каждого типа свободных ресурсов.

### Определение 10.9 (Матрица заявок)

Матрица заявок — матрица, в которой указана максимальная потребность каждого процесса в каждом типе ресурса (`claim_matrix`).

$$P = \begin{pmatrix} b_{11} & \dots & b_{1m} \\ \vdots & & \vdots \\ b_{n1} & \dots & b_{mn} \end{pmatrix} = \begin{pmatrix} B_1 \\ B_2 \\ \vdots \\ B_n \end{pmatrix}$$



### Определение 10.10 (Матрица распределений)

Матрица распределений (`allocation_matrix`) —

$$C = \begin{pmatrix} C_{11} & \dots & C_{1m} \\ \vdots & & \vdots \\ C_{n1} & \dots & C_{nm} \end{pmatrix} = \begin{pmatrix} C_1 \\ \vdots \\ C_n \end{pmatrix}$$

$c_{ij}$  — количество ресурсов  $r_j$ , выделенное процессу  $p_i$ .



При этом

- 1) все  $k$  для которых  $B_k \leq A$ , могут быть удовлетворены, но главное, что процесс не может требовать количество единиц ресурса больше, чем имеется в системе или больше, чем доступное количество;
- 2)  $C \leq B$  — указывает на то, что процесс не может запросить больше ресурсов, чем указано в его заявке;
- 3)  $\sum_{k=1}^n$  — в системе невозможно выделить больше ресурсов, чем имеется у системы;
- 4) матрица доступных ресурсов может быть сформирована следующим образом:

$$D = (d_1 \dots d_m) = A - \sum_{k=1}^n C_k$$

У системы имеется количество единиц ресурсов, которое представляется вектором  $A$ , но чтобы анализировать текущую ситуацию, мы должны определить, сколько в текущий момент в системе имеется свободных единиц каждого ресурса. Для этого мы выполняем вычитание и для каждого ресурса суммируем уже выделенное количество ресурсов. Мы получаем матрицу свободных единиц ресурсов в текущий момент времени.

- 5) Матрица возможных запросов:

$$E = \begin{pmatrix} e_{11} & \dots & e_{1m} \\ \vdots & & \vdots \\ e_{n1} & \dots & e_{nm} \end{pmatrix} = B - C = \begin{pmatrix} E_1 \\ \vdots \\ E_n \end{pmatrix}$$

Из заявки вычитаем полученные единицы, в результате формируем матрицу возможных запросов

6) Матрица текущих запросов (Current Request Matrix)

$$F = \begin{pmatrix} f_{11} & \dots & f_{1m} \\ \vdots & & \vdots \\ f_{n1} & \dots & f_{nm} \end{pmatrix} = \begin{pmatrix} F_1 \\ \vdots \\ F_n \end{pmatrix}$$

7) Можно определить предварительное состояние. Предположим, что удовлетворяются все запросы и проведём анализ ситуации

$$D \leftarrow \underbrace{D}_{\text{Доступно}} - \underbrace{F_i}_{\text{Запросы}}$$

$D$  доступно,  $F$  — запросы. В результате запросов  $D$  изменится, то есть мы заранее анализируем запросы.  $C_i$  так же будет изменено, если предположить, что запросы будут удовлетворены, то есть

$$C_i \leftarrow C_i + F_i$$

$E_i$  также изменится, т.к. что мы предполагаем, что запросы были удовлетворены и количество единиц ресурса, которое в последствии процесс может запросить, изменится

$$E_i \leftarrow E_i - F_i$$

### Алгоритм проверки безопасного состояния

- 1) Выбирается не завершившийся процесс  $p_i : E_i \leq D$ , то есть доступно больше ресурсов, чем требуется процессу  $P_i$ . Если такого процесса нет, то выполняется переход на шаг 3.
- 2)  $D \leftarrow D + C_i$ , процесс  $P_i$  получил запрошенный ресурс,  $P_i$  помечается как завершённый. В результате системе возвращаются занятые процессом  $P_i$  единицы каждого типа ресурса и переход на шаг 1.
- 3) Если все процессы отмечены как «завершённые», то система находится в безопасном состоянии, иначе — в опасном. Если система находится в опасном (небезопасном) состоянии, то запрос процесса блокируется, и состояние системы сбрасывается, то есть выполняется откат по пункту 7.

$$D \leftarrow D +$$

Дополнительно процесс отмеченный как завершённый на шаге 2, также возвращается в исходное состояние, то есть сбрасывается, и  $P_i$  повторно помечается как незавершённый.

*Пример.* Пусть имеются соответствующие значения, которые отражены в матрицах.

$$A = (2 \ 4 \ 3)$$

$$B = \begin{vmatrix} 1 & 2 & 2 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{vmatrix}$$

$$C = \begin{vmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{vmatrix} \begin{matrix} P_1 \\ P_2 \\ p_3 \end{matrix}$$

$$\underbrace{\begin{pmatrix} r_1 & r_2 & r_3 \\ 2 & 3 & 2 \end{pmatrix}}$$

$$D = (0 \ 1 \ 1) \Rightarrow$$

$$E = \begin{vmatrix} 0 & 0 & 2 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{vmatrix} = B - C$$

Пусть  $P_1$  делает запрос  $F_1 = (0 \ 0 \ 1)$ , то есть запрашивает единицу ресурса  $R_3$ . Проанализируем предварительное состояние. Оно определяется как

$$D = (0 \ 1 \ 0)$$

$$C = \begin{vmatrix} 1 & 2 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{vmatrix}$$

$$\underbrace{\hspace{1.5cm}}_{(2 \ 3 \ 3)}$$

$$E = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 0 \end{pmatrix}$$

ситуация вот эта поменялась на вот эту (обведена строка в овал и стрелочка такая типа на другуд строку ну я не нарисую тут етава)

В этом состоянии  $P_3$  может завершиться, так как  $E_3 \leq D$ . В результате  $P_3$  вернёт удерживаемые им ресурсы: единицу  $R_1$  и единицу  $R_3$  в пул доступных ресурсов. В результате

$$D = (1 \ 1 \ 1)$$



Это позволит удовлетворить потребность процессов  $P_1$  и  $P_2$ . Делается вывод предварительной оценки о том, что состояние безопасное. В результате запрос  $P_1$  является безопасным и может быть удовлетворён.

В данном подходе необходимо иметь матрицу заявок, которая отражает максимальную потребность в каждом типе ресурса.

*пример, который демонстрирует работу 3 метода обнаружения тупиков(?). 2 основные матрицы:*

- 1) матрица распределения;
- 2) матрица запросов.

Матрица распределения:

| p\r      | 1 | 2 | 3 |
|----------|---|---|---|
| 1        | 0 | 1 | 1 |
| 2        | 1 | 3 | 0 |
| 3        | 1 | 5 | 0 |
| $\Sigma$ | 2 | 9 | 1 |

Матрица запросов:

|   | 1 | 2 | 3 |
|---|---|---|---|
| 1 | 1 | 1 | 0 |
| 2 | 0 | 0 | 1 |
| 3 | 1 | 1 | 2 |

ещё таблица

Тогда можем определить количество ресурсов в системе. формула  $R_i$

В результате имеем 6 9 3. нам нужно проанализировать систему, есть ли такая последовательность процессов, которая может успешно завершиться. Это похоже на то, как формулировал алгоритм банкира Дейкстра, но это ничего не меняет, это позволяет проводить анализ. Мы видим, что второй процесс имеет строку запросов меньше  $\Phi$ , следовательно, он может завершиться, в результате освободив занимаемые им единицы ресурсов. Выполняем с матрицей следующие действия:

$$\begin{array}{ccc} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & 5 & 0 \end{array} \Rightarrow F = (5 \ 3 \ 2)$$

Теперь первый процесс имеет строку запросов меньше  $F$ . Также мы можем сократить по  $P_1$ .

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 5 & 0 \end{array} \Rightarrow F = (5 \ 4 \ 3)$$

Можно удовлетворить запрос третьего процесса, поскольку у системы имеется достаточное количество единиц каждого ресурса.

В результате:

$$\begin{array}{ccc} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{array} \Rightarrow F = (6 \ 9 \ 3)$$

То есть, анализируемое состояние не является тупиковым. Важна разница: там было безопасное/небезопасное, а тут тупик или нет. Мы сократили все дуги, матрица полностью сокращаемая, система не находится в тупике.

Существует 3 основных подхода для предотвращения тупиков:

1) Перезапустить систему

Плох тем, что будет потеряна вся работа всей совокупности процессов.

2) Последовательно удалять/завершать/отбирать ресурсы у процессов, которые попали в тупик. Очень возможно, что эти ресурсы позволят другим процессам, попавшим в тупик, продолжить выполняться.

Проблема может быть решена откатом. Процесс не завершается, а переводится в то состояния, в котором он был до запроса.

3) Отбирать ресурсы у процессов, которые не попали в тупиковую ситуацию. При этом в том предположении, что отобранные ресурсы позволят удовлетворить запросы в соответствующих ресурсов для процессов, которые попали в тупик. Какие процессы выбирать для завершения? Например, можно завершать самые низкоприоритетные процессы.

Все эти решения требуют скрупулёзного анализа программистом. Не существует систем, которые не могли бы попасть в тупик.

### 10.13.7 Алгоритм Chandy–Misra–Haas

Существует алгоритм обнаружения тупиков в распределённой системе. Этот алгоритм предлагает выполнять обнаружения тупика на основе пробного сообщения.

Если процесс выполняет запрос на ресурс, который занят, то процесс формирует сообщение-зонд или пробное сообщение и посылает это сообщение-зонд всем процессам, которые могут удерживать интересующие его ресурсы (имеется информация о взаимодействующих процессах). Каждое сообщение-зонд содержит следующую информацию:

- 1) ID процесса (PID), которые «блокирован», так как запрашиваемый ресурс занят. Слово «блокирован» — условное
- 2) ID процесса, который послал текущую версию зонда (particular version).

3) ID процесса, который должен получить это сообщение - зонд.

|           |             |            |
|-----------|-------------|------------|
| инициатор | отправитель | получатель |
|-----------|-------------|------------|

Когда процесс получает зонд, он проверяет, не ждёт ли он данный ресурс. Если процесс удерживает этот ресурс, то он сможет его освободить, когда закончит работать с ним. Если процесс ждёт этот ресурс, то он посылает сообщение-зонд всем процессам, которые удерживают ресурсы, которые он запрашивал. В результате будет модифицироваться сообщение-зонд, то есть будет изменяться ID отправителя и ID получателя.

Простейший пример данной ситуации:

рисунок

Рис. 29. рисуночик

Транзакции тоже в современном мире выполняются в распределённых системах, и оказывается что транзакции также могут попадать в тупиковые ситуации, что приведет к невозможности обслуживания запроса в удаленной системе, если в этой транзакции участвует некоторое количество процессов.

Для предотвращения тупиковой ситуации фактически имеется 2 алгоритма. В результате выполнения анализа системы, если обнаружится что система попадет в тупик, возникнет необходимость какую-то транзакцию убрать.

## 10.14 Транзакции

### 10.14.1 Алгоритм ожидание-смерть (wait-die)

Если транзакция старше чем  $T_2$ , то  $T_1$  может подождать, иначе если  $T_1$  младше, то  $T_1$  прерывается и затем должна быть перезапущена.

### 10.14.2 Алгоритм ожидание-ранение (wound-wait)

Если транзакция  $T_1$  старше  $T_2$ , то  $T_2$  прерывается и затем перезапускается, иначе если  $T_2$  младше, то  $T_1$  может ждать.

## 10.15 чтоо

Ресурс-память в системе имеет несколько типов, причём тип зависит от близости к процессору. Речь идёт об архитектуре Фон-Неймана. В соответствии с его концептуальной, процессор может выполнять только программы из оперативной памяти. Предполагается, что быстродействие оперативной памяти должно соответствовать быстродействию процессора. В вычислительной системе существует иерархия памяти. Она строится по отношению к процессору.

В старых системах были и магнитные барабаны, и магнитные диски. В наших системах есть вторичная память, как и раньше. И даже несмотря на изменение технологии производства, это — дисковая память. На персональных компьютерах

были стримеры — ленточно-протяжные устройства большого объема для запоминания большого объема данных. Сейчас вторичная память, которая рассматривается как блочное устройство, обладает огромными запоминающими мощностями.

К этой категории блочных устройств также относятся широко используемые флеш-накопители.

То, что мы называем оперативной памятью

|                    |   |
|--------------------|---|
| Внешняя память     |   |
| Оперативная память |   |
| Кэш                | ? |

Таблица 2. рисуночик НЭ БУДЭ

У процессора есть общий кэш L3, а так же у каждого ядра собственные L1, L2. Это называется вертикальным управлением передачи данных. Управление на конкретном уровне иерархии называется горизонтальным управлением. Управлением данных во вторичной памяти занимаются файловые системы.

Типы рассмотренных операционных систем:

1) Система однопрограммной пакетной обработки.

Все ресурсы выделялись одному единственному процессу, который загружался в ОЗУ целиком. Для таких систем распределение памяти называлось **одиноким непрерывным распределением**, причём в последовательные адреса. Это как раз было декларировано Фон-Нейманом в его статье, это одна из особенностей его архитектуры.

2) Операционная система мультипрограммной пакетной обработки.

Это означает, что в оперативной памяти одновременно находилось какое-то число программ.

## Статические разделы

рисунок

Рис. 30. рисуночик

...

Разделы определялись до начала работы системы и далее не менялись.

## Динамические разделы

При начале работы системы программы загружались, занимая адресное пространство, которое соответствовало их размеру. Никаких потерь памяти в начальный момент нет.

рисунок

Рис. 31. рисуночик

Должны быть как минимум 2 таблицы: таблицы свободных разделов и таблицы распределённых разделов ??.

Практически невозможно найти программу того же размера. В раздел можно загрузить программу такого же или меньшего размера. При этом возникают участки, в которые мы уже ничего загрузить не можем. Например, возможна такая ситуация, завершился процесс  $P_3$ , видим что свободные участки соседствуют. Возникает стремление их объединить, нужно отредактировать соответствующие таблицы, а это дополнительные действия системы по управлению памятью. Даже при наличии возможности объединения свободных разделов при постоянной загрузке и выгрузке программ в памяти возникнут участки очень небольшого размера, в которые невозможно загрузить ни одну программу. Это явление называется **фрагментацией памяти**.

При этом были сформированы стратегии выбора раздела для загрузки программ:

1) Первый подходящий.

Программа загружалась в первый подходящий раздел.

2) Самый тесный.

Искался раздел, наиболее точно соответствующий размеру программы.

3) Самый широкий.

Программа загружалась в самый большой раздел, самый широкий.

Стратегия была наиболее востребована, потому что ... в разделе остаётся достаточно памяти для загрузки ещё одной программы.

Вторая и третья стратегии требовали сортировки по размеру разделов для более быстрого поиска.

Если не перезагружать систему, нужно перемещать программы. Для этого нужны специальные аппаратные средства.

Для того, чтобы иметь возможность перемещать программы в памяти, нужно иметь соответствующие аппаратные средства. Рассмотрим ситуацию, когда программа перемещена на несколько байт.

рисунок

Рис. 32. рисуночик

Чтобы обратиться к операнду, необходимо выполнить преобразование адреса. Преобразование будет выполняться на каждой команде, а если команда косвенная, то даже несколько раз.

Как часто выполнять такое перемещение?

Все программы грузятся в память целиком. Было быстро замечено, что любая программа начинается с нулевого адреса. Соответственно в коде у нас только смещения.

Рис. 33. рисуночик

Если сначала компиляция, трансляция выполнялась в абсолютных адресах, это потребовало изменения подхода к компиляции. Это привело к возможности загрузки программы с любого адреса, изменения коснулись всего ПО. Осталась проблема связности выделения памяти.

На смену непрерывному распределению пришло не непрерывное 😊.👁️

Первым решением было **деление адресного пространства** процесса и адресного пространства физической памяти **на страницы**.

Страница памяти = фрейм.

Страницы программы могут быть загружены в любую страницу физической памяти. Таким образом был осуществлён отказ от непрерывности выделения памяти. Технически это решается с помощью таблиц страниц. Пока ещё не отказались от необходимости загружать в память всё адресное пространство, то есть все страницы процесса. Также в этот же период времени прорабатывался вопрос выделения памяти сегментами. Программа делилась не на страницы, которые имеют один размер, а на сегменты, что более соответствует логической структуре программ. Также с помощью таблиц осуществлялась возможность выполнения, работая с сегментами.

Возникла необходимость преобразования, причём на схеме мы также видим преобразование. Как часто выполняется такое преобразование? На каждой команде, а то и несколько раз. В результате увеличились затраты на формирование физического адреса. Отсюда 1 шаг до виртуальной памяти.

По архитектуре фон Неймана мы знаем, что один из основных пунктов был пункт о том, что программа должна находиться в физической памяти. Но может ли только часть программы находиться в физической памяти? Да, может, в этом и смысл схем управления виртуальной памятью. Это потребовало создания таблиц страниц для **каждого** процесса. Если распределять сегментами, то у каждого процесса должна быть своя таблица сегментов. В таблице указывается страница адресного пространства процесса.

Программа не должна целиком находиться в физической памяти. В ней должна находиться та часть программы, с которой производится работа в данный момент времени.

Это отправная точка идеи виртуальной памяти. Виртуальный — кажущийся, возможный.

Существует 3 схемы управления виртуальной памятью:

- 1) Страницами по запросу
- 2) Сегментами по запросу

### 3) Сегментами, поделёнными на страницы по запросу

От стремления избавиться от фрагментации, от необходимости выделения непрерывных адресных пространств в физической памяти, пришли к идеи виртуальной памяти.

Какой от этого выигрыш, если это так затратно? Увеличение мультизадачности (мультипрограммности (multitasking)), это значит, что больше одной программы может одновременно находиться в физической памяти. Они могут выполняться, ждать своей очереди, но это и есть мультизадачность.

Это огромные затраты, но от идеи использования виртуальной памяти никто не отказался.

## 10.16 Управление памятью страницами по запросу

Во всех трёх частях содержится общая часть: «по запросу», то есть тогда, когда процессор обращается к данным, отсутствующим в физической памяти. Реализуется это с помощью прерываний, в частности, если мы говорим об управлении памятью страницами по запросу, то это будет страничное прерывание. Причём страничное прерывание отнесено к исправимым исключениям.

В начальный момент времени, когда процесс порождён, то есть он получил идентификатор, часть его полей дескриптора проинициализирована, процессу выделяется физическая память. В современных системах это какое-то минимально необходимое для начала выполнения количество страниц физической памяти. Условно говоря, 3 страницы: сегмента кода, сегмента данных и сегмента стека.

После этого по диаграмме состояний, процесс переходит в очередь готовых процессов для получения процессорного времени. Получив процессорное время, процесс начинается выполняться команда за командой. В какой-то момент времени процессор обращается к данным, отсутствующим в физической памяти. Возникает страничное прерывание, в результате которого нужная страница будет загружена в физическую память, после чего процесс начнёт своё выполнение. К затратам добавляется обслуживание страничных прерываний, затраты ещё больше.

## 10.17

Существует 3 базовые схемы преобразования адресов в страничном распределении.

### 10.17.1 Прямое отображение

Процессы имеют виртуальные адресные пространства. Виртуальный адрес делится на две части: номер страницы и смещение. Обязательно должен быть регистр базового адреса таблицы страниц.

Каждый процесс имеет собственную таблицу страниц, в которой находятся дескрипторы страниц.

Фактически, ... это смещение к дескриптору страницы

... Смещение одно и то же, причём дерётся оно из программы. Выполняется преобразование и получается физический адрес.

При этом такая таблица страниц находится в физической памяти. То есть чтобы выполнить преобразование, сначала нужно обратиться к таблице страниц, получить физический адрес страницы и затем выполнить преобразование для получения физического адреса данных. Такое преобразование будет выполняться на каждой команде, а то и несколько раз — крайне затратные действия.

Обращение к физической памяти требует большого количество тактовых импульсов — циклы обращения к памяти. Это затратное действие. Стремясь уменьшить эти накладные расходы, разработчики систем, основываясь на своих технических знаниях, предложили использовать ассоциативную память.

### **10.17.2 Ассоциативное отображение**

Таблица страниц загружается в ассоциативную память.

Особенностью ассоциативной памяти является возможность получения данных за 1 такт за счёт схемы реализации ассоциативной памяти.

Важно понимать, почему эта схема не нашла своё физическое воплощение, а лишь частичное. Ассоциативная память очень дорогая, это всегда регистровая память.

#### **РИСУНОК**

Номер виртуальной страницы является ключом. При совпадении ключа формируется импульс, который позволяет считать информацию. Каждый разряд каждого регистра дополнительно имеет схему совпадения.

Мало того, что это регистровая память (количество схем увеличивается в 2 раза), также увеличивается количество соединений. Это говорит о дороговизне ассоциативной памяти.

Скорость доступа к памяти заметно вырастает, но обходится это очень дорого.

### **10.17.3 Ассоциативно-прямое отображение**

В физической памяти располагается таблица страниц. В процессоре есть регистр базового адреса таблицы страниц и ассоциативная память небольшого размера, в которое может находиться только небольшое количество дескрипторов страниц.

При этом в такой ассоциативной памяти хранятся только дескрипторы страниц, к которым были последние обращения. Связано это со свойством локальности наших программ. если обращение было к странице, то наиболее вероятно, что следующие обращения будут к этой же странице. Это следует из того, что команды, данные хранятся в памяти в последовательных адресах. При этом наиболее вероятной структурой является следование, выполнение программы именно команды за



командой. Ветвление выполняется в программах реже, сюда же относим и ветвление в цикле. Данные также хранятся в последовательных адресах. На странице никогда не выполняются все команды, как правило это от  $\frac{1}{3}$  до  $\frac{2}{3}$  команд. Даже кэш от до обеспечит скоростные показатели, составляющие 90% скоростных показателей при реализации полностью ассоциативной памяти .

Сначала страница ищется в ассоциативной памяти. Если её там нет, то обращение происходит к таблице страниц физической памяти. При этом будет выполнено замещение дескриптора. Кэш TLB — Translation Lookaside Buffer — ассоциативный по множеству четырёхнаправленный буфер.

Размеры программ растут, размеры таблиц страниц увеличиваются. Решением задачи увеличения размеров стала схема с гиперссылками.

#### **10.17.4 Схема с гиперстраницами**

##### **РИСУНОК**

Адресное пространство процесса делится на гиперстраницы (очень большие страницы), размер которых кратен одной странице, то есть это большая страница, содержащая другие страницы.

Это многоуровневая схема. Виртуальный адрес делится на 3 части: смещение, номер страницы, номер гиперстраницы. Теперь адресное пространство процесса описывается одной таблицей гиперстраниц и большим количеством таблиц страниц.

Вместо одного обращения к физической памяти мы получаем 2. Сюда так же нужно добавить ассоциативный кэш, которые сокращает затраты. К кэшу будет обращение по двум полям.

#### **10.18 Замещение страниц**

Процесс никогда не обращается одновременно ко всем своим страницам. Физическая память имеет ограниченный размер — это дефицитный ресурс.

Процесс в какое-то время обращается к странице, которая отсутствует в физической памяти, а свободный фрейм отсутствует, то есть все страницы физической памяти заняты. Изначально различали страницы виртуального адресного пространства и страницу физического адресного пространства, называемую фреймом. Что делать, если не хватает .... — надо выполнить замещение.

##### **10.18.1 Замещение случайной страницы**

Не является дискриминационным

Данный подход с равной вероятностью может быть вытеснена любая страница любого процесса, но также существует вероятность, что будет вытеснена только что загруженная страница или часто используемая. Реализация такого алгоритма малозатратна.

### 10.18.2 Алгоритм FIFO

Для реализации данного алгоритма каждому фрейму присваивается или временная метка, или организуется связанный список. Вытеснена будет страница, которая дольше всего находилась в физической памяти. Но также такой страницей не окажется только что загруженная страница, но может оказаться часто используемая страница. Алгоритм FIFO по своим характеристикам может приблизиться к первому алгоритму замещения страниц.

Но также было обнаружено, что алгоритм FIFO обладает свойством аномалии (аномалия алгоритма FIFO). Она заключается в том, что при увеличении объёма физической памяти, расчёт на то, что количество страничных прерываний сократится, не оправдывается. Количество страничных прерываний для некоторых траекторий страниц может увеличиться.

#### РИСУНОК С ПРИМЕРОМ

Траектория страниц — номера страниц, к которым выполняются обращения.

### 10.18.3 Алгоритм LRU

LRU — least recently used, page replacement

Алгоритм относится к классу стековых алгоритмов. Алгоритм наиболее полно отвечает свойству локальности наших программ, но в реальных системах не используется, так как требует колоссальных затрат на реализацию.

#### Свойство 10.11 (Свойство локальности)

Свойство является следствием архитектуры фон Неймана. Если обращение было к странице, то наиболее вероятно, что следующие обращения будут к этой же странице. Это связано с тем, что программа в своём адресном пространстве располагается в последовательных адресах (в виртуальном адресном пространстве команда за командой).

По этому алгоритму будет вытеснена страницу, к которой дольше всего не было обращений. Это требует изменения работы или с временными метками или со связным списком. При каждом обращении к странице должна быть отредактирована временная метка. Если это связный список, то при каждом обращении страница перемещается в конец списка (меняется адрес). Это крайне затратные действия в системе, потому что обращения происходят на каждой команде, а то и несколько раз.

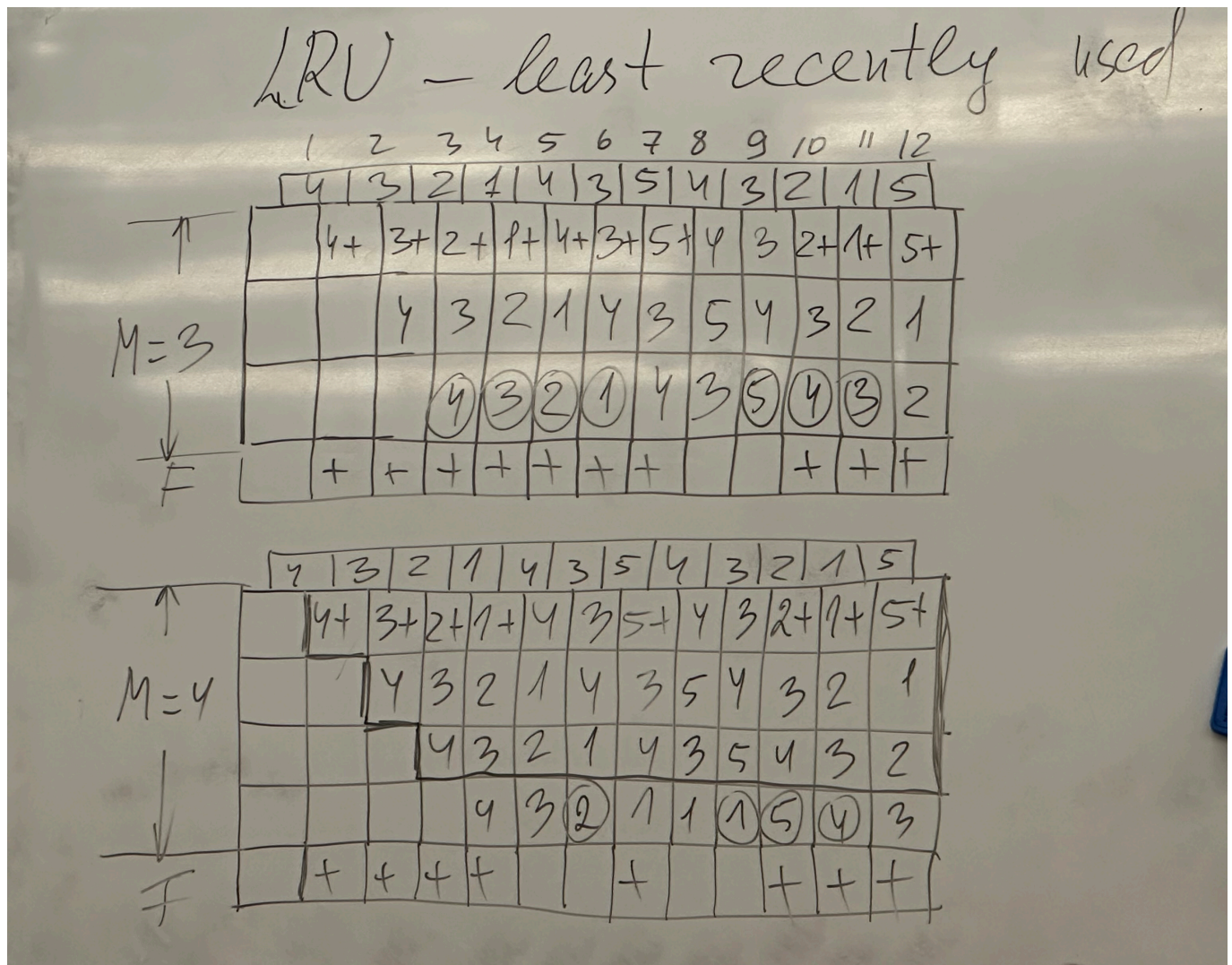


Рис. 34. рисунок

#### 10.18.4 Алгоритм LFU

LFU — least frequency used, page replacement (наименее часто используемая страница)

В этом алгоритме фактически контролируется частота обращения к странице. Это реализуется с помощью счётчика. Наименее используемой может оказаться только что загруженная страница. В результате она будет вытеснена, и поскольку она будет вытеснена, потому что не успела набрать счётчик обращений, её необходимо будет загрузить заново. Это большие накладные расходы, поэтому используется аппроксимация (алгоритм NUR).

#### 10.18.5 Алгоритм NUR

NUR — not used recently (страница, не используемая в последнее время)

Это аппроксимация LRU

Рис. 35. рисунок

Имеется бит, который изменяется при каждом обращении к странице. При загрузке бит устанавливается в 1 и с этого момента будет рассматриваться удаление.

Именно этот способ реально используется в системах. В дескрипторе странице имеется бит обращения access. Но кроме бита обращения вводится бит модификации.

2 бита, в результате возможны 4 ситуации:

1) 00

2) 01

Обращения не было, модификация была. Дело в том, что биты обращения время от времени сбрасываются в 0. При необходимости выгрузки страницы, страница для выгрузки ищется среди страниц, у которых бит обращения 0. Ввод бита модификации позволяет вытеснение выполнять более точно, и данное сочетание двух битов не абсурдно. В данном случае можно сказать, что модификация была до последнего сброса бита обращения в 0.

Возникает вопрос: какую страницу выгоднее вытеснять во вторичную память, модифицированную или не модифицированную? Не модифицированную, потому что её точная копия находится на диске и это не требует копирования. Копирование — сложный и затратный процесс.

3) 10

4) 11

Ну скажем, в эээ, (смотрит в листок) ХА-ХА ваще смеётся с объёмов памяти

## 10.19 Теория рабочего множества

В 1968 году Деннинг предложил в качестве локальной меры производительности взять число страниц, к которым обращается процесс за интервал времени  $\Delta t$ . Рабочее множество (working set) — функция  $W(t, \Delta t)$ . Величина показывает количество страниц, к которым обращается процесс. При этом выяснилось, что для эффективного выполнения процесса (процесс эффективно выполняется, когда нет страничных прерываний, то есть нет блокировок в ожидании загрузки нужной страницы, если в памяти находятся все страницы, к которым процесс обращается за некоторый интервал  $\Delta t$ , то есть всё рабочее множество).

Исследование IBM было проведено, потому что несмотря на затраты, никто не отказался от реализации идеи виртуальной памяти, потому что это существенно повышает уровень мультизадачности. При этом возникает вопрос: сколько физической памяти потенциально нужно предоставлять каждому процессу, то есть сколько физических страниц понадобится процессу для его эффективного выполнения, чтобы он смог загрузить в память своё рабочее множество. Эта теория отвечает на этот вопрос.

В результате исследований было подтверждено наличие рабочего множества. При этом в начальный момент времени, когда процесс порождён,?

Процессу, чтобы он мог начать выполняться, выделяется минимальное необходимое количество страниц. Скажем, ему выделяется по одной странице кода, стека и данных. После этого процесс переходит в состояние готовности. Получив процессорное время, программа начинает выполняться, и процесс начинает интенсивно подгружать нужные ему страницы. Будут возникать страничные прерывания, система будет загружать нужные страницы в память.

В результате в какой-то момент времени процесс загрузит в память своё первое рабочее множество. Этот интервал  $\Delta t$ , в течение которого процесс будет обращаться к одним и тем же страницам, ни предсказать, ни измерить невозможно. Затем процесс перейдёт к своему второму множеству.

Существуют такие интервалы во времени жизни процесса, когда процесс выполняется без страничных прерываний. Для этого процесс должен иметь возможность загрузить в память всё своё рабочее множество, то есть количество физических страниц которые выделены процесс, например, как квота, должно позволять загружать ему рабочее множество. Если процесс не может загрузить в память своё рабочее множество, возникнет интенсивная подкачка одних и тех же страниц в результате страничных прерываний. Это получило название **trashing**.

В Windows рабочее множество — это квота. Каждому процессу изначально устанавливается квота — 10 физических страниц. Любая система контролирует страничных прерываний. Если число страничных прерываний у процесса резко возрастает, это значит, что процесс не может загрузить в память всё своё рабочее множество. В этом случае квота может быть увеличена. Для этого нужно иметь возможность отложить процесс на какое-то время, пока система не сможет добавить к его квоте ещё некоторое количество страниц.

### 10.19.1 hit rate

?? TLB Hit Rate? hit rate paging — доля обращений, не потребовавших смена страницы.

Первый график отражает влияние изменения размера страниц при сохранении объёма оперативной памяти на число прерываний.

график, похожий на  $\cap$   
Рис. 36. первый график

Мы видим, что число страничных прерываний растёт, сначала не очень интенсивно, затем резко, а потом падает. Рост числа страничных прерываний связан с тем, что чем больше страница, тем меньшее количество команд будет выполнено на этой странице. Когда страница достигнет размера, сопоставимого с размером команды, число страничных прерываний сократится.

Второй график — кривая времени жизни, график зависимости длительности периода между прерываниями.

### Рис. 37. Рисунок

Время выполнения процесса — суммарное время, полученное процессом.

Перегиб происходит из-за того, что в некоторый момент времени в оперативной памяти процесс загрузил всё своё рабочее множество. Интервал между обращениями к отсутствующей странице называли временем жизни.

### Рис. 38. цилиндр 70-х годов прошлого века

Программа в любом варианте находится во вторичной памяти. Когда она не выполняется, это просто файл. Когда мы запускаем программу на выполнение, это процесс, его адресное пространство так же находится во вторичной памяти.

Любая программа считает, что она начинается с нулевого адреса, внутри неё есть смещения. Система хранит информацию о свободных, занятых кадрах, есть системная таблица, всё в ядре.

## 10.20 Какая-то тема про чото

Во всех современных системах используется страничная память. Но раньше для Unix, для PDP-11 был написан с сегментированной памятью. Для того же 32-х разрядного процессора

Флаг Copy-On-Write решил

### 10.20.1 Официальное заявление на 10.12.2025 🚀

Рязанову в детстве пинали родители (официальное заявление от неё).

Слово хер — не ругательное. В древнерусском языке оно означало крест (официальное заявление). Похерить — перекрестить.

## 10.21 Управление сегментами по запросу

### Рис. 39. Сегментно-страничная организация памяти

Сегмент - логическая единица деления памяти, поскольку его размер определяется размером программного кода. Также имеется в процессоре регистр базового адреса, но это уже таблица сегментов. В таблице дескрипторов сегментов обязательным полем является его размер. Поскольку необходимо контролировать выход за границы сегмента. Если адрес не превышает размер, то выполняется адресация.

Мы говорим о вирт памяти, управление ... по запросу. Может возникнуть ситуация когда нет в памяти свободных блоков, и необходимо выполнить замещение. Замещение сегментов выполняется аналогично замещению страниц. Но возникают дополнительные проблемы. Если когда мы замещаем страницу, то мы замещаем на идентичную страницу (размер). Но в случае сегментов, может быть такое что загруженный сегмент будет меньше выгруженного. Может быть такое, что сегмент находится



далеко. Необходимо выполнить перемещение. В классическом Unix была реализована сегментная адресация.

Для них установить права доступа для работы с сегментами было проще, меньше накладных расходов.

В старых учебниках расписывалось, что требовалось хранить информацию, объем которой возрастал с увеличением ...

Сегмент - «модуль» и права доступа устанавливались целиком для этого модуля, а не страницы. Это требовало большого объема информации.

Затрудняется своппинг сегментов. Была предложена схема: поделить сегменты на страницы. Размер сегмента кратен размеру страницы. В управлении памятью все равно остается понятие сегмента: сегмент кода, данных, тд.

понятно, что жти сегмнты также будут кратны размеру страницы, но управление памяти – современное понятие. Была попытка соединить достоинства двух схем управления памятью: страничными и сегментированной. Это стало называться сегментно-страничной организацией памяти.

В 32-х разрядных системах на базе Intel были аппаратно реализованы две независимые схемы управления памятью:

- 1) сегментами по запросу;
- 2) страницами по запросу.

Именно в 32-х разрядных системах появилось понятие плоской памяти (flat). Это связано с тем аппаратным обеспечением управления памяти, которое было реализовано в этих процессорах. В защищённом режиме кроме рассмотренных регистров (общего назначения, адресов, флагов, сегментных) было добавлено ещё 2 сегментных регистра:

- 1) CS
- 2) SS
- 3) DS
- 4) ES
- 5) FS
- 6) GS

Эти регистры играли роль селекторов. Кроме этих регистров в процессорах Intel были включены регистры адресов:

- 1) GDTR — 32-разрядный (Global Descriptor Table Register)  
Описывает сегменты физической памяти
- 2) IDTR — 32-разрядный (Interrupt Descriptor Table Register)
- 3) LDTR — 32-разрядный (Local Descriptor Table Register)

Описывают сегменты виртуального адресного пространства процесса.

Эти регистры сохранены в 64-х разрядных процессорах для обратной совместимости (Compatibility mode). Размер сегмента в 32-х разрядных процессорах не мог превышать 4 Гб. Для размера сегмента было отведено 20 разрядов ( $2^{20} = 1\text{Гб}$ ) — размер шины адреса в реальном режиме, но память можно было измерять в байтах, а можно в страницах по 4 Кб  $= 2^{12}$  Байт ( $2^{12+20} = 2^{32} = 4\text{ Гб}$ ). В процессоре имеется несколько регистров: CR0, FLAGS.

В силу ограничения адресации было введено понятие флэт память.

Рис. 40. Чета 3 схема

Виртуальный адрес делится на 3 части. Максимальное смещение определяется возможным размером сегмента. Сегмент делится на страницы. В результате у одного сегмента будет таблица страниц. Если у нас много сегментов, будет соответствующее количество таблиц страниц.

Смещение делится на 2 части: страница, смещение в странице.

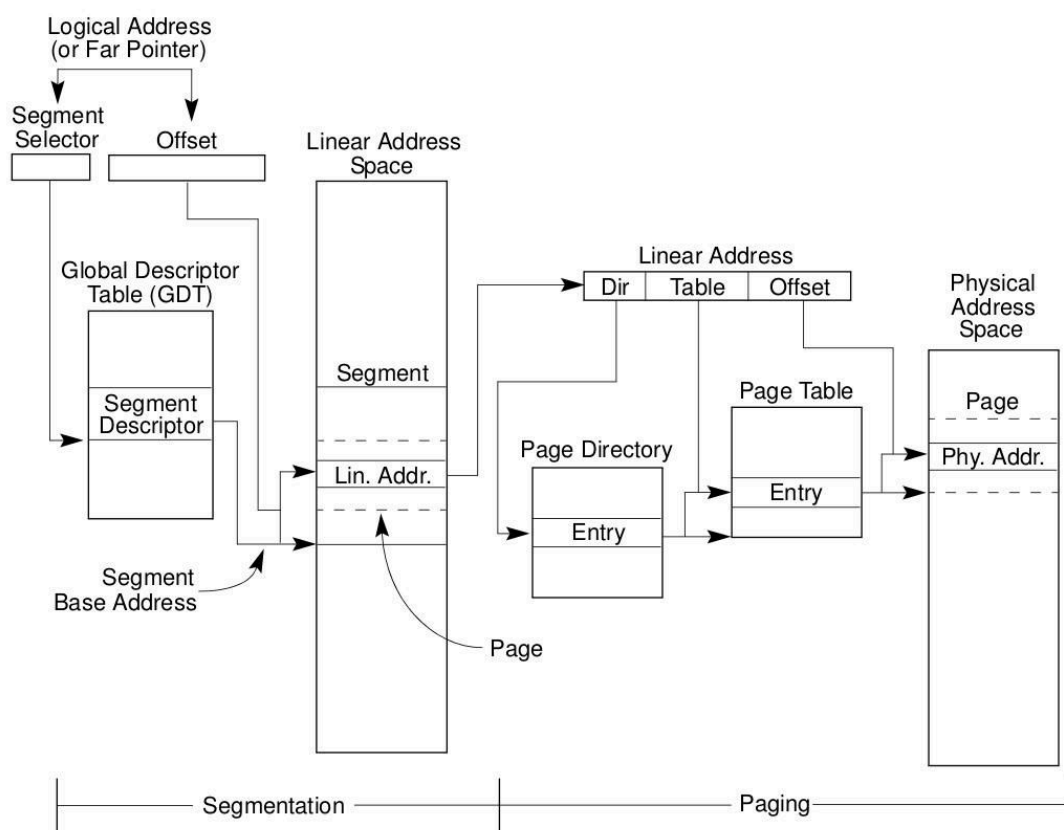


Рис. 41. Схема трансляции логического адреса в физический

Отложенное связывание возникло, когда программы стали содержать большое количество модулей. При этом конкретный запуск ПО стал приводить к тому, что выполняются не все части кода большой программы, это зависит от начальных параметров, от заданных значений. Перед разработчиками всегда стояла задача эффективного использования памяти. Исходя из того, что программа может выпол-



няться не полностью, возникла идея DLL (динамически загружаемая библиотека, Dynamic Link Library). Она возникла давно.

Отложенное связывание связано с тем, что не для всех модулей программы создаются дескрипторы сегментов, не все модули описаны дескрипторами в таблице сегментов. В результате обработка прерывания по особому случаю при связывании приведёт к тому, что сегменту с символическим именем будет выделен дескриптор в таблице сегментов (присвоен номер строки в таблице сегментов) и будут заполнены поля структуры (поля сегментов дескриптора?), то есть по случаю прерывания, когда такой дескриптор в таблице сегментов отсутствует. Прерывания по особому случаю в сегменте возникнет в том случае, если для данного сегмента не создана таблица страниц.

В результате обработки этого прерывания создаётся таблица страниц для сегмента, и адрес таблицы страниц заносится в дескриптор сегмента. Прерывание по особому случаю в странице возникнет так же, как при страничном распределении, когда процессор обращается к странице, отсутствующей в памяти.

Возникает сначала страничное прерывание, при выполнении которого окажется, что нет таблицы страниц, содержащей дескриптор этой страницы, чтобы загрузить его в память. Может оказаться, что и нужный сегмент не имеет соответствующего дескриптора.

Виртуальный адрес - 64-х разрядный. Все таблицы имеют размер 4 КБ (одна страница). Это 12 разрядов, то есть смещение 2 в 12.

- 1) С 0 по 11 бит — смещение.
- 2) С 12 по 20 бит — номер страницы.
- 3) С 21 по 29 бит — в директории
- 4) С 30 по 38 бит — смещение в таблице страниц PDP
- 5) С 39 по 47 бит — смещение PML4
- 6) С 48 по 63 бит — Зарезервировано

Page Map Level 4 — 512 дескрипторов. Размер таблицы  $512 \cdot 8 = 4$  КБ.

Это смещения в четырёхуровневой таблице страниц.

Начальный адрес этой таблицы находится в регистре CR3.

рисунок

Такая схема позволяет адресовать 256 ТБ адресного пространства. При этом указатель виртуальный адрес может в данной схеме занимать только 48 разрядов. На базе процессоров Intel существует возможность формировать адреса, используя 57 бит. В этом случае добавляется ещё одна таблица с 5 уровнями и адресация возрастает до 128 ПБ.

Каждый дескриптор имеет разрядность 2 в 64 байт? Рассмотрим дескриптор таблицы страниц

рисунок

Без соответствующей аппаратной поддержки, такая схема пожирала бы процессорное время. В процессорах интел это 3 кэша и кэш ТЛБ. L1 и L2 находятся непосредственно в ядрах, то есть принадлежат процессору. L3 является обдим кэшем. TLB — Translation Lookaside Buffer. Все кэши являются частично ассоциативными.

Для первых 32-х разрядных процессоров такие кэши назывались 4-х направленными ассоциативными множествами.

 Пропущена лекция 17-й недели

## 10.22

Системный вызов переводит систему в режим ядра.

Системный вызов является один из типов прерываний в системе прерываний.

- interrupt — аппаратное прерывание
- system call / program interrupt — системный вызов

рисунок

Рис. 42. Рисунок

- Пунктиром в этой диаграмме означает передачу данных.
- Любой системный вызов означает вызов функции, параметры получают фактические параметры, то есть происходит передача данных.
- SVC supervisor call/ В результате вызова API read/write. Система переходит в режим ядра.
- Разрыв на схеме означает, что процессор отключается и не управляет внешним устройством.

У ядра есть слой функций, который вызывается сразу непосредственно при переходе в режим ядра, например, switch.

Поскольку приложение запросило ввод/вывод, мы рассмотрим чтение/запись в файл. Он находится на внешнем запоминающем устройстве. В результате выполнения кода ядра будет вызван драйвер (в книге написано «программа ввода/вывода»).

Драйвера входят в код ядра, являются его частью. Мы говорим о низкоуровневом драйвере. Драйвер сформирует из переданных параметров код, который по шине данных будет передан на контроллер, скажем, диска. В результате это будет **инициализация ввода/вывода** (start IO). Процессор отключается.

В соответствии с принципом, в IBM360 был реализован принцип распараллеливания функций, то есть передача управления внешним устройствам, чтобы по тем

временам освободить центральный процессор от задачи управления медленными внешними устройствами.

В результате передаче по шине данных команд контроллеру внешних устройств, он начинает управлять работой внешнего устройства, процессор отключается от управления внешним устройством.

Прерывания от внешних устройств формируются контроллерами прерываний по завершении операций ввода/вывода. Контроллер сформирует сигнал прерывания, который придёт на выделенную ножку, сформируется INT. В конце цикла выполнения каждой команды процессор проверяет наличие сигнала на выделенной ножке. Если он есть, процессор формирует INT A, ..., формируется вектор прерываний, ....

Устройство выполнит запрос. Драйвер вызовет callback-функцию. Драйвер переведёт полученные данные в соответствующий формат. Приложение получит данные, но сначала оно должно быть разблокировано.

Приложение разблокируется системой, когда система передаёт ему данные из ядра в буфер пользователя. Это в общей классификации ввода/вывода называется **синхронным блокирующим вводом/выводом**.

Обработка начинается системным вызовом, а заканчивается (почти) аппаратным прерыванием.

В Linux мы видели непрерываемый сон (D, Device). Такую блокировку прервать нельзя. Смысл в том, что приложение должно получить данные, но как прервать сон, как сохранить состояние, чтобы в него вернуться, чтобы приложение получило требуемые данные? Такого места нет, поэтому придумали `uninterruptable sleep`.

## 10.23 Ядра

Существует два типа ядер:

- 1) Монолитные
- 2) Микроядра

Linux и Unix позволяют пользователю включать в монолитное ядро нужную функциональность.

## 10.24 Микроядра

Смысл в том, что операционная система имеет микроядра, и часть функций реализована в виде отдельных процессов, которые выполняются, возможно, в собственных адресных пространствах. Разработчикам удалось вывести ввод в режим пользователя.

Имеется 4 кольца защиты. Для указания уровня привилегий отведено 2 бита. В наших системах работают два кольца: 0 (ядро) и 3 (пользователь).

Часть кода, которая вынесена из ядра, работает по модели клиент-сервер. Все низкоуровневые операции, то есть операции непосредственно с аппаратным обес-

печением остаются в микроядре. Фактическое выделение физической страницы — низкоуровневое действие. А определение, какую страницу можно вытеснить на диск — высокоуровневое.

рисунок

Рис. 43. Рисунок

Сервер памяти решает, какую область какому процессу выделить. Если работает с виртуальной памятью, процессу выделяется виртуальное адресное пространство, в котором затем выделяются регионы. Этим может заниматься сервер памяти.

Так же с файлами. Файловая система многоуровневая, какие-то действия, связанные с управлением файлами, могут быть вынесены в сервер файловой системы. Обращение к файлу будет выполняться путём обращения к внешнему устройству.

Сервера находятся в режиме пользователя, как и приложения. В результате микроядро должно предоставить пользователю те же возможности, что и монолитные системы.

Приложение формирует запрос. Этот запрос через микроядро адресует соответствующему серверу. Отработав запрос приложения, сервер формирует ответ, который через микроядро придёт в приложение. Например, в результате вызова `fork()` мы получаем результат, который проверяем на -1.

В дополнение к низкоуровневым действиям микроядро должно обеспечить возможность взаимодействия, причём сервер для другого сервера может быть клиентом. Взаимодействие в микроядерной архитектуре выполняется путём передачей сообщений. Передача должна быть ..., клиент должен быть уверен, что сообщение дошло, что запрос обслуживается.

Для QNX введена функция `reply`, чтобы подчеркнуть, что это отправка ответа на сформированный запрос.

Цитата из книги:

Основан ли Windows 2000 на микроядрах? Windows 2000 не является ОС, основанной на микроядрах. В ОС на микроядерной архитектуре ...

...

Системы построенные только по принципу микроядра не практичны из-за слишком низкой эффективности. (*из-за слишком большого времени ответа на действия пользователя*)

— Какая-то книга

## 10.25 Операционная система Mach

Сама концепция явилась прогрессивной. Микроядерная архитектура активно используется в системах специального назначения.

Почему? Если у нас сервер, скажем, планирования вынесен из ядра, то мы можем переписать код этого сервера, не затрагивая ядро операционной системы. Микроядерная архитектура, исходя из её особенностей, позволяет вносить изменения в соответствующие коды операционной системы. Другое дело, что они выполняются в режиме пользователя. Если же мы попытаемся изменить монолитное ядро, например, если нам недостаточно использовать те средства, которые предоставляются, нам придётся перекомпилировать ядро, а это сложная задача.

Ядро Mach управляет пятью основными абстракциями:

### 1) Процессы

Именно процесс владеет ресурсами. Он **не владеет приоритетом**, но имеет его, ему он назначается. Для пользовательских процессов значение приоритета изменяется.

### 2) Потоки

В Unix потоки имеют те же приоритеты, что и процессы. Если дополнительных процессов не создаётся, main рассматривается как один поток. Если создаются, то новые потоки рассматриваются как вспомогательные, но для системы это всё потоки. Приоритеты потоков меняются в зависимости от того, на чём поток был блокирован, сколько времени ожидал в очереди готовых потоков, но поскольку базовым понятием является процесс, никто не переписывает всё на потоки.

Потоки — часть кода программы, которая может выполняться параллельно с другими частями программы. Они не имеют собственного адресного пространства и выполняются в адресном пространстве процесса.

Поток владеет аппаратным контекстом. При выполнении они запрашивают ресурсы, но владельцем ресурсов будет процесс.

С памятью возникают проблемы, связанные с потоками.

### 3) Объекты памяти

Память страничная, есть пул страниц в виртуальном адресном пространстве.

При этом также реализованы страничные прерывания. Если при обращении к объекту оказалось, что он не находится в физической памяти, возникнет страничное прерывание.

В отличие от монолитных систем, ядро Mach для загрузки страницы форсирует зарпос, который поступит на сервер памяти. Сервер выполнит соответствующий запрос, может вызвать ещё какой-то код ядра. В итоге нужная страница будет

загружена в физическую память. Процесс, обратившийся к объекту памяти, продолжит своё выполнение, поскольку это исправимое исключение.

При этом в современных Linux, они загружают сразу несколько страниц (пул) с тем расчётом, что дальше будет обращение к следующим страницам. Это не всегда происходит так. Тогда происходит проигрыш, поскольку в память загружены неиспользуемые страницы.

#### 4) Порты

Это средство обмена сообщениями, то есть порт поддерживает упорядоченный список сообщений. Процесс владеет каким-то количеством портов для обмена сообщениями с серверами или с другими процессами, поскольку параллельные процессы взаимодействуют.

Для взаимодействия реализована возможность, чтобы процесс могут другому процессу получать или посылать сообщения по определённом порту, разделяя его.

#### 5) Сообщения

Поддерживаются задачи и нити (threads). Под задачей понимает процесс, фактически занимаемый адресное пространство и набор системных ресурсов.

## 11 Системы реального времени (RTOS, Realtime OS)

В POSIX-1 даётся определение системы реального времени и самого реального времени.

### Определение 11.1 (Реальное время в ОС)

Реальное время в ОС — способность операционной системы обеспечить требуемый уровень сервиса в определённый промежуток времени. Как правило, это системы специального назначения.

Система должно обеспечить ответ на полученный запрос за строго отведённый интервал времени. Это **жёсткое реальное время**.



В мире наибольшее число систем относится к системам специального назначения. Там самые объёмные и сложные задачи программирования.

Как правило, такие системы обслуживают сложные внешние системы, состоящие из большого количества подсистем, состоящих из большого количества датчиков, которое должно контролировать соответствующее ПО, которое написано именно для управления конкретным внешним объектом.

Время ответа системы должно быть меньшим или равным периоду поступления запросов на результат обслуживания.

Если какие-то подсистемы позволяют какие-то небольшие отклонения от периода, отведённого на ответ, это называется **jitter** (дрожание).